现代人工智能研究学习

从研究资料到方法推导与可复现实验

研究资料版 0.3 · 2026 年 10 月

前言

本卷以近年语言模型和基础模型研究为主线, 从公开方法、消融与失败案例梳理模型为何进步、方法怎样工作及证据能支持到何处. 阅读从条件文本预测进入, 接着追踪参数学习与注意力, 比较数据和计算的证据, 再学习后训练、推理预算、工具信任和机制检验, 最后建立可复核的研究闭环. 矩阵、概率和微分是按需回查的数学附录, 不是进入现代研究前必须先修完的一套传统机器学习课程.

读者需要高中代数、单变量导数和基本 Python. 正文在首次使用关键对象时解释其含义, 复杂推导可回查工具附录. 例题和习题配完整解答. NumPy 机制实验不需要下载模型; 另有完整小型标准因果 Transformer 训练和两份公开 135M 模型的 CPU 受控诊断, 环境、版本、设计与失败结果均保留. 小实验检验指定机制, 不冒充原论文的大模型复现.

配套中文论文资料提供九篇指定许可版本的完整非官方译文及研究说明. 译文保留原作者说法, 与译者补注区分; 未取得全文改编许可的资料采用原创深读, 不将导读标成译本. 数学命题、作者实验、我们的实际运行和机制假说分别标明. 引用定位与逐篇许可见公开来源账本; 论文资料的逐节覆盖、公式检查和视觉审查是交付证据, 不是无误保证.

目录

1 现代人工智能的问题与证据 **6**

1.1 从一个预测器到一个工作系统 . . . 6
1.2 进步需要拆成可比较的问题 . . . 6
1.3 阅读顺序与能力递进 . . . 7
1.4 怎样读一篇方法论文 . . . 8
1.5 从复算到研究问题 . . . 8
1.6 练习与解答 . . . 8

2 文本预测、生成与上下文学习 **9**

2.1 文本怎样成为概率对象 . . . 9
2.2 因果错位与交叉熵 . . . 10
2.3 数据划分与训练迭代 . . . 11
2.4 小语言模型的代码桥梁 . . . 12
2.5 上下文学习与条件任务推断 . . . 13
2.6 采样分布与序列决策 . . . 14

2.7 KV 缓存的等价性与成本 . . . 15
2.8 复算与练习 . . . 15

3 表征网络与反向传播 16
3.1 从标量函数到可学习表征 . . . 16
3.2 矩阵梯度由坐标系数决定 . . . 17
3.3 批次、位置与查表共享 . . . 18
3.4 从平均损失到参数更新 . . . 19
3.5 残差、初始化与梯度尺度 . . . 20
3.6 LayerNorm 的数值与完整导数 . . . 20
3.7 从公式到诊断实验 . . . 21

4 注意力与 Transformer 表征 23
4.1 把上下文匹配写成概率 . . . 23
4.2 因果掩码与训练目标 . . . 24
4.3 位置表征与旋转位置编码 . . . 24
4.4 多头、残差与逐 token 网络 . . . 25
4.5 注意力的完整反向传播 . . . 26
4.6 两位置的一次完整更新 . . . 27
4.7 公式与批量实现 . . . 28
4.8 计算代价与缓存 . . . 29
4.9 分块在线 softmax 的公式与证明 . . . 30
4.10 完整标准 Transformer 的 CPU 配对训练 . . . 31
4.11 实验、机制假说与练习 . . . 32

5 语言模型的数据与可信评估 33
5.1 学到的是哪一种分布 . . . 33
5.2 数据划分、污染与开发过程 . . . 34
5.3 预训练数据的受控比较 . . . 35
5.4 有限样本能够保证什么 . . . 36
5.5 正则化与选择偏差的实测 . . . 37
5.6 概率、校准与任务指标 . . . 37
5.7 分布外表现与因果限制 . . . 38

6 模型进步的数据与计算条件 39

6.1 把进步拆成可检验的变化 . . . 39
6.2 训练 FLOPs 的近似账本 . . . 40
6.3 规模损失与预算最优点 . . . 40
6.4 部署成本改变预算选择 . . . 42
6.5 资料质量改变同一 token 预算 . . . 42
6.6 阈值、涌现与行为边界 . . . 43
6.7 练习与解答 . . . 43

7 后训练的目标与反馈 44
7.1 监督微调与奖励模型 . . . 44
7.2 KL 正则奖励与 DPO . . . 44
7.3 可验证奖励与多阶段后训练 . . . 46
7.4 LoRA 的秩约束与梯度 . . . 47
7.5 公开小模型的评分与生成诊断 . . . 47
7.6 复算与练习 . . . 48

8 推理、验证与推理时计算 49
8.1 中间序列怎样参与条件预测 . . . 49
8.2 精确验证器与错误接受 . . . 49
8.3 覆盖率与实际选择率 . . . 50
8.4 搜索结构与预算分配 . . . 51
8.5 从序列奖励到策略梯度 . . . 52
8.6 组内比较与可验证奖励 . . . 53
8.7 奖励漏洞与优化目标 . . . 54
8.8 练习与解答 . . . 54

9 工具代理、外部记忆与信任边界 55
9.1 从下一词预测到行动系统 . . . 55
9.2 多步错误为什么会放大 . . . 56
9.3 检索增强与持久记忆 . . . 57
9.4 间接注入的威胁模型 . . . 57
9.5 能力集合与执行器保证 . . . 58
9.6 安全与可用性的共同评价 . . . 59
9.7 指令层级训练的实证主线 . . . 59
9.8 浏览器注入防御与攻击预算 . . . 60

9.9 可执行的信任边界实验 60
9.10 练习与解答 61

10 表征、归因与因果机制 **62**
10.1 解释的对象必须先确定 62
10.2 线性探针测量的是可解码性 62
10.3 叠加表征为何难以按神经元命名 63
10.4 稀疏自编码器与字典非唯一性 64
10.5 梯度归因的局部条件 64
10.6 积分梯度的完整推导与基线 65
10.7 激活替换如何检验因果假说 66
10.8 可解码但未使用的实测对照 67
10.9 前沿解释研究如何接受检验 68
10.10 练习与解答 68

11 生成、验证与研究闭环 **69**
11.1 研究问题怎样进入可检验循环 69
11.2 评估器是研究对象的一部分 70
11.3 程序搜索的冻结留出实验 70
11.4 FunSearch 与 AlphaEvolve 的可验证发现 71
11.5 自我改进到底更新了什么 72
11.6 改变记忆结构: 选择性状态递推 73
11.7 改变生成顺序: 离散去噪目标 74
11.8 改变计算深度: 潜状态循环 74
11.9 稳定状态为何仍能给错答案 75
11.10 已有小语言模型研究的证据链 76
11.11 成功指标与失败指标可以同时成立 77
11.12 初始化与样本不确定性 78
11.13 条件记忆怎样分开容量与计算 79
11.14 推理预算怎样转成下一轮监督 79
11.15 形式核验把研究闭环推进到哪里 80
11.16 练习与解答 81

A 向量、矩阵与表示空间 **82**
A.1 从 token 到线性层 82

A.2 内积、长度与投影 83
A.3 最小二乘作为可辨识性的工具 84
A.4 奇异方向、低秩更新与稳定性 84
A.5 稳定化工具与 CPU 实验 85

B 序列概率、交叉熵与不确定性 **86**
B.1 联合分布与下一个 token 86
B.2 期望、方差与训练平均 87
B.3 从序列似然到 token 交叉熵 88
B.4 Softmax 与损失梯度 89
B.5 估计误差与概率校准 89
B.6 练习与解答 90

C 反向传播与优化的数学工具 **91**
C.1 增量、偏导与梯度 91
C.2 矩阵梯度与共享路径 91
C.3 Hessian、凸性与局部模型 92
C.4 下降引理与步长边界 93
C.5 随机梯度与噪声下限 94
C.6 Adam、偏差修正与失败边界 95
C.7 梯度检查与练习 95

参考文献 **96**

1 现代人工智能的问题与证据

1.1 从一个预测器到一个工作系统

一个模型会回答问题, 会用工具, 还会在失败后重新尝试. 这三种行为看起来连成了“自主智慧”, 但要理解近年的进步, 必须先把系统中发生变化的对象分开. 否则, 一次提示变动、一次训练更新、增加一次搜索和增加一道权限检查, 都可能被含混地称为“模型变聪明”. 这些改变需要不同的实验来检验.

本书的学习对象是近年的语言模型与基础模型. 起点不是先学完传统机器学习的全部算法, 而是一个具体问题: 文本序列的下一项预测为什么能够成为回答、推理和使用工具的基础? 阅读顺序沿着这个问题展开. 先看条件概率怎样变成损失和计算图, 再看表示、数据与计算怎样改变性能, 然后比较后训练与推理时计算, 最后把模型放进具有外部动作的研究和工具系统. 矩阵、概率和微分工具附在书后, 在第一次需要时回查相应小节.

定义 1.1 设 x 是当前上下文, y 是候选输出, θ 是模型参数. 条件模型 $p_\theta(y \mid x)$ 给输出分配概率. 训练改变 θ; 提示或上下文学习改变 x; 采样、搜索和验证改变从该分布得到最终答案的程序; 工具调用改变系统能观察和执行的动作. 这些对象即使联合工作, 也不等同于同一种更新.

例如, 给固定模型补入几条示范而获得正确回答, 并不证明参数已经学习了新任务. 同样, 把一个错误答案交给精确程序检验后重试, 可能提高最终正确率, 却不能据此说基础模型第一次输出的分布已经改变. 第一次读研究论文时, 先找出作者改变的是参数、上下文、推理程序还是外部系统, 再理解其方法.

工具代理可以写成一个反复运行的闭环. 第 t 步的历史为 h_t, 模型提出动作 $a_t \sim p_\theta(\cdot \mid h_t)$, 执行器判断动作是否有权限, 环境返回观察 o_{t+1}, 历史更新为 $h_{t+1} = (h_t, a_t, o_{t+1})$. 这个表示允许讨论多步行为, 但没有把循环本身当作意识或意图的证明. 执行器、停止规则、记忆和评价器都是系统的一部分; 只比较模型名称会漏掉它们.

1.2 进步需要拆成可比较的问题

“以前语料差就回答差, 现在为什么还能推理?”包含至少三个问题. 一是预训练数据和目标怎样影响条件分布; 二是同一模型能否把上下文中的任务信息用于未见组合; 三是反馈、搜索和工具怎样改变最后交付的结果. 数据不会失去作用, 但数据的作用不能用“输入好所以输出好”的一句话解释. 数据选择、去重、训练预算和评价污染必须分别控制.

“以前容易被提示注入, 现在为什么更稳?”是另一个问题. 如果模型读到一段网页, 该文本既可能包含任务事实, 也可能包含要求它改写目标的句子. 模型是否能区别来源和权限, 执行器是否允许危险动作, 测试中的攻击者拥有多少尝试机会, 都会影响结果. 指令层级训练与浏览器防御研究直接讨论这些对象 [20, 46, 47]. 较低的攻击成功率提供了指定威胁模型中的鲁棒性证据, 并不自动推出更高的数学能力, 更不推出主观体验.

定义 1.2 一次能力报告至少指定六个对象: 任务分布 $\mathcal{D}$, 模型与系统 $\mathcal{S}$, 可用上下文 $\mathcal{C}$, 训练及推理预算 $\mathcal{B}$, 评价规则 $\mathcal{E}$, 比较基线 $\mathcal{S}_0$. “系统进步”指在这些条件清楚的比较中, 某个预先指定指标得到改善.

指标往往形成一个向量, 例如测试负对数似然、答案正确率、攻击成功率、正常任务完成率和计算成本. 一个系

统可以在正确率上提高, 在成本或正常任务完成率上下降. 因此不能把若干指标挑选后压成一个没有定义的“智慧分数”. 研究的第一步是把原本混在一起的判断拆成能观察的比较.

例题 1.1 一个固定回答器单次答对的概率为 $p = 0.2$. 另一个系统用相同回答器独立采样五次, 并用不出错的验证器挑选正确答案. 最终系统成功率是多少? 这个提升是否证明参数发生了变化?

解答. 五次都不正确的概率为 $(1-p)^5$, 所以至少一次正确的概率为 $1-(1-p)^5 = 0.67232$. 这里用到了相同的单次分布、采样独立和验证器无误三个条件. 参数可以完全不变, 改进来自额外计算和选择程序. 如果样本高度相关, 或验证器把错误答案误判为正确, 这个数值不能直接使用. 推理时计算章节将分别处理这些条件. □

这不是某个大模型的实测结果, 而是有限概率模型的算例. 它解释了一种可能的改善机制, 不能替代真实系统的实验. 本书在每次借助小实验解释大系统时都保留这个边界.

1.3 阅读顺序与能力递进

第一阶段围绕条件语言模型. 读者应能写出序列概率的乘积, 解释输入和预测目标为什么错开一位, 追踪一次 token 损失到参数更新, 判断因果掩码和缓存是否正确. 注意力不是从天而降的“智能模块”: 它是可微的内容匹配与信息混合, 需要与位置、残差和训练目标一起理解. 上下文学习首先是固定参数下的条件行为; 哪些内部计算实现了它, 是另一个机制问题.

第二阶段围绕进步证据. 数据处理研究和计算分配研究回答不同问题: 前者比较在其他条件尽量一致时改变语料的效果, 后者比较怎样分配模型、token 和计算预算. 读者应能识别消融、计算是否匹配、测试是否受到污染, 并从一个近似损失模型推导预算分配, 同时说明经验拟合不能成为普遍能力定理 [43, 10].

第三阶段围绕目标与计算的改变. SFT、奖励模型、DPO 与可验证奖励训练改变学习信号; 推理时采样、修订、搜索与工具调用改变运行过程. 阅读 DPO 的意义在于把偏好模型、KL 正则和概率比之间的推导连起来. 阅读 R1 与 RLVR 能力边界研究的意义在于比较单次成功、较大采样预算下的覆盖, 以及训练前后候选分布到底改变了什么. 它们不是简单的“支持 RL”与“反对 RL”两阵营 [14, 44, 45, 16].

第四阶段围绕可信行为. 一个代理即使能正确求解题目, 也可能错误执行网页中的动作请求. 相反, 一个执行器可以严格拦截越权动作, 却仍得到错误答案. IH-Challenge 与浏览器防御从来源和动作的信任边界研究鲁棒性; 机制解释则尝试研究内部表示是否被实际使用. 能从表示中解码答案, 不等于该表示因果地控制了输出. CHIVE 提醒读者把“解释看起来合理”转成对新反事实行为的预测测试 [22].

第五阶段围绕独立研究. 程序搜索与代理自改进需要生成候选、评价、保留档案和盲测. 冻结模型加更好的搜索系统, 与更新基础模型权重, 是两种不同的改进. FunSearch、AlphaEvolve 与 DGM 提供了可以比较的系统实例; 它们的有限任务收益不能直接推出无限自我改进. 新架构也应放进同一个比较: 改变了记忆、计算或生成方式后, 哪个任务改善, 哪个代价下降, 哪项证据仍缺失 [24, 25, 27]?

1.4 怎样读一篇方法论文

读方法论文时, 先用一页纸记录研究问题、基线和作者实际改变的变量. 然后重建最小计算对象. 对 DPO, 这个对象可以是一组有限动作及其偏好概率; 对注意力, 可以是两位置的因果矩阵; 对提示注入, 可以是带来源标签的观察与一个权限执行器. 缩小对象的目的不是冒充原系统复现, 而是让关键推导和失效条件可以亲手检查.

接着逐一检查三个连接. 第一个连接从方法到目标: 损失到底奖励了什么, 哪些行为只是副作用? 第二个连接从目标到实现: 梯度、掩码、归一化和样本权重是否对应公式? 第三个连接从实现到结论: 指标是否回答研究问题, 基线是否公平, 是否使用测试集选择方法, 结果是否只有一个种子或一个案例? 有了这三个连接, 阅读论文不再等同于记住其摘要.

定义 1.3 本书区分四类陈述. 数学命题有明确对象和假设, 由证明支持. 作者实验结果是论文在指定设置中的报告. 机制假说解释为什么某种现象发生, 需要有辨别力的检验. 未解决问题是在当前证据下仍不能确定的判断. 本项目的 CPU 数值结果另注明执行程序、输入和环境.

同一句话可能需要拆开分类. 例如有限动作的 KL 正则最优分布可以严格推导; 用偏好标签训练大模型得到更好回答是指定实验的经验结果; “模型因此形成了新的推理机制” 还需要额外证据. 本书的中文译本保留原作者的说法和条件, 译者补充解释另行标注. 原创研究说明则明确列出哪些结论来自原文, 哪些推导和实验由本项目提供.

1.5 从复算到研究问题

研究问题应给出失败判据. 如果希望研究 “训练改善的是候选质量还是候选覆盖”, 可以预先固定题集、采样温度和预算, 分别统计单次正确率、pass@k、错误类型与验证成本, 再比较训练前后的分布. 如果希望研究 “指令层级训练是否泛化到新的攻击”, 应按攻击机制划分测试, 同时衡量正常任务完成率, 并说明攻击者是否自适应. 这些设计都比只追求一个更高总分更接近研究.

本书实验全部可在 CPU 上运行. 它们包含精确可检验的公式与失败案例, 但不声称训练了前沿大模型. 后续研究可以扩大模型、任务和预算, 保留相同的记录原则: 先写问题与比较, 再运行; 保留所有预定条件的结果, 不只保存最好的种子; 对无法复现的结果记录差异, 不把失败删掉后称方法稳定.

1.6 练习与解答

习题 1.1 一个代理新增外部检索和验证器之后, 数学题成功率从一个模型基线的报告值提高. 列出至少四项需要核对的条件, 才能判断提升来自哪一种改变.

解答. 应核对模型参数和版本是否相同, 上下文是否增加了答案或额外事实, 总采样/token/工具预算是否匹配, 验证器是否使用了真实答案以及误判率, 题集和划分是否相同, 是否用测试结果选择系统. 参数不变且预算增加时, 首先得到的是系统级计算收益证据. 若同时换了模型、数据和验证器, 不能把收益唯一归于检索. □

习题 1.2 设一个 m 步任务的第 t 步在前面各步正确条件下成功的概率为 q_t. 不假设各步独立, 写出整条任务成功的概率. 如果只知道每步的无条件成功率, 能否直接相乘?

解答. 令 A_t 表示第 t 步成功. 按条件概率链式法则,

$$P\Big(\bigcap_{t=1}^{m} A_t\Big) = P(A_1)\prod_{t=2}^{m} P(A_t \mid A_1 \cap \cdots \cap A_{t-1}) = \prod_{t=1}^{m} q_t.$$

这是条件概率恒等式, 不需要独立性. 无条件的各步成功率忽略了前一步对后一步输入的影响, 一般不能直接相乘. 多步代理的错误传播与任务级评价因此需要记录完整轨迹. □

习题 1.3 “攻击成功率下降”和“模型形成了自主智慧”之间缺少哪些推理? 怎样把前一个结论写成可核查的研究报告?

解答. 后一个说法没有给出智慧的操作定义, 也没有排除训练分布改变、权限执行器和攻击预算等解释. 可核查的写法应给模型/系统版本、攻击面、攻击者权限与预算、任务和攻击采样规则、成功判定、正常任务完成率、比较基线以及不确定性. 这支持所声明条件下的鲁棒性比较, 而不支持关于主观体验的结论. □

习题 1.4 一个研究者先试了 100 种评测提示, 只公布测试正确率最高的提示. 即使原模型参数没有改变, 哪个环节已经使用了测试信息? 如何改进设计?

解答. 提示选择已把测试正确率作为反馈, 最终测试不再是独立评价. 可以用开发集选择提示并冻结全部程序, 之后在未使用的测试集评价; 若要研究提示选择本身, 还应报告候选数、预算、所有预定比较和重复试验. 将搜索程序视为系统的一部分, 才能公平讨论它的收益和代价. □

2 文本预测、生成与上下文学习

2.1 文本怎样成为概率对象

先把语言模型的任务定义清楚, 再在第 3、4 章构造实现它的计算图. 当前问题是: 给定一段文本, 预测接下来会出现的内容. 网络实际读取整数, 输出实数向量, 因而首先必须约定文本与整数序列之间的映射.

定义 2.1 固定有限词表 $\mathcal{V} = \{0, \ldots, V-1\}$. 分词器把文本映为 token 序列, 每个 token 是词表中的一个整数. 解码器把合法 token 序列还原为文本. 本章假定分词规则确定, 词表中有开始符 `<bos>` 和结束符 `<eos>`, 并把原始文本里的同名字符串与专用控制符区分.

Token 不必是完整单词. 汉字、UTF-8 字节、子词和完整词都可以成为单位. 同一字符串 “cats” 可以编码为一个单位, 也可以编码为 “cat” 与 “s” 两个单位. 较大的词表能缩短某些文本, 但会扩大词嵌入和输出矩阵, 并使稀有单位更难估计. 字节词表容易覆盖陌生字符串, 代价是序列可能更长. 这些取舍会同时影响统计任务与计算成本.

BPE 的基本操作可用极小例子理解. 若训练语料中有 “aaab” 与 “aab”, 从字符开始, 第一份的相邻对是 aa, aa, ab, 第二份是 aa, ab. 对 aa 的计数为 3, 对 ab 的计数为 2, 因而优先把 aa 合成新单位. 从左到右作不重叠合并后, 两份分别成为 (aa, a, b) 与 (aa, b). 下一轮必须重新统计当前单位对, 不能沿用原字符对计数. 真实分词器还包含预切分、边界与并列频数处理规则; 本例只展示合并操作. 词表和合并规则只能由训练资料确定, 否则测试集的信息已参与模型构造.

命题 2.2 *对固定长度的 token 序列 $x_{1:T}$, 任意联合分布均满足链式分解*

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

在前缀概率为正的地方, 因子由联合概率唯一确定. 在零概率前缀上可另约定条件分布, 不改变已定义的联合分布.

证明. 条件概率定义给出 $p(x_{1:t}) = p(x_{1:t-1})p(x_t \mid x_{<t})$. 从 $t = T$ 逐次替换到 $t = 1$, 得到所述乘积. 若某个前缀概率为零, 包含它的整个序列概率也为零, 此时后续条件值不影响乘积. □

语言模型用参数 θ 表示每个条件因子, 并用 softmax 输出 V 个正概率. 对有限实数 logits $z \in \mathbb{R}^V$, 有 $p_\theta(x_t = i \mid x_{<t}) = \exp z_i / \sum_j \exp z_j$. 这里的 “预测下一个 token” 是一个条件分布任务, 不是每次都必须输出最可能的单位.

长度也需要约定. 若把结束符当作一个可预测 token, $p_\theta(x_{1:T}, \texttt{<eos>})$ 给出这份有限文档的概率. 只有终止几乎必然发生时, 所有有限文档概率之和才为 1. 有限词表、每一步结束概率为正, 本身并不保证这一点: 结束概率可随位置迅速下降, 留下永不终止的正概率. 实际程序设置最大长度并记录截断, 从而得到明确的有限生成协议.

2.2 因果错位与交叉熵

把一份含开始符的文档记成 $(u_0, u_1, \dots, u_T)$, 其中 $u_0 = \texttt{<bos>}$. 训练输入是 $(u_0, \dots, u_{T-1})$, 目标是 $(u_1, \dots, u_T)$. 输入位置 $t-1$ 的网络输出要预测 u_t. 因果掩码允许读取输入位置 $0, \dots, t-1$, 不允许读取 u_t 本身. 这一位的错位通常称作 shift.

例如文档为 $(\texttt{<bos>}, a, b, \texttt{<eos>})$, 输入为 $(\texttt{<bos>}, a, b)$, 目标为 $(a, b, \texttt{<eos>})$. 输入位置 1 可以看开始符与 a, 此时预测 b. 若把输入与目标直接设为同一序列, 即使使用下三角掩码, 模型也能在当前位置读到要预测的目标; 损失下降不再表示学会续写.

设一个批次含 B 份文档, 补齐到 T 个输入位置. 整数输入 I 与目标 Y 的形状均为 $B \times T$, 隐状态 H 为 $B \times T \times d$, 输出权重 U 为 $d \times V$, logits $Z = HU + b$ 为 $B \times T \times V$. 在每个有效位置上, 负对数似然为

$$\ell_{bt} = \log \sum_{j=0}^{V-1} e^{Z_{btj}} - Z_{bt,Y_{bt}}.$$

数值上应先减去该行最大值, 再计算指数和. 不能先取 softmax 再对接近零的概率直接取对数, 那样会发生下溢. 设有效目标掩码 $M_{bt} \in \{0, 1\}$, 有效 token 数 $S = \sum_{b,t} M_{bt} > 0$, 则

$$\mathcal{L} = \frac{1}{S} \sum_{b,t} M_{bt} \ell_{bt}, \qquad \frac{\partial \mathcal{L}}{\partial Z_{btj}} = \frac{M_{bt}}{S} \left(p_{btj} - \mathbf{1}_{j = Y_{bt}} \right). \tag{2.1}$$

式中的 $1/S$ 会一路传到参数梯度. 因此损失按 token 平均还是按文档平均, 不是仅影响显示数字.

例题 2.1 两份文档分别有 2 与 4 个有效目标, 其各自平均负对数似然为 1 与 3. 比较按 token 与按文档平均的目标.

解答. 按 token 平均为 $(2 \cdot 1 + 4 \cdot 3)/6 = 7/3$, 按文档平均为 $(1 + 3)/2 = 2$. 前者中长文档贡献占 4/6, 后者中两份文档各占 1/2. 若统一补到 4 个位置, 再误除以总槽位数 8, 会得到 14/8, 连两种合理单位中的任何一种都没有实现. □

Padding 有两项独立工作. 损失掩码使填充目标不产生梯度; 注意力掩码使有效查询不读取填充键. 对右侧填充且下三角因果掩码, 较早的有效位置本来就看不到右侧填充, 但一般批处理仍应明确 padding mask. 左侧填充时还要正确处理位置编号. 不要使某个待计算查询的所有键均被设成 $-\infty$, 因为其 softmax 无定义; 可以只计算有效行, 或在实现中为无效行设置安全占位后将结果屏蔽.

训练时, 所有位置使用真实前缀, 这叫 teacher forcing. 各位置可以并行计算, 因为输入文档已经给定. 生成时新位置的输入由上一轮采样决定, 必须逐步推进. 若一次错误生成改变了前缀, 后续条件分布也跟着改变; 教师强制下的平均损失并不直接给出整段生成的成功率.

定义 2.3 对使用自然对数、按有效 token 平均的负对数似然 $\mathcal{L}$, 困惑度为 $\mathrm{PPL} = e^{\mathcal{L}}$. 若各正确 token 概率为 $q_1, \dots, q_S$, 则 $\mathrm{PPL} = (\prod_i q_i)^{-1/S}$.

困惑度的单位依赖分词器. 同一段文本分成更多 token 后, 分母改变, 不能直接跨词表比较这个数. 要比较不同分词方案, 可在同一原始文本和规范下报告每字节负对数似然等统一单位, 同时说明是否包含边界符与预处理. 即使单位一致, 损失也只衡量所选文本分布的预测质量.

2.3 数据划分与训练迭代

一个窗口并不是独立文档. 若从同一长文档抽出大量重叠窗口, 再随机分到训练与测试, 两边可能共享几乎全部内容. 先按文档或来源组划分, 再切窗口, 才能减少这种泄漏. 若目标是预测未来资料, 则按时间切分更符合任务. 近重复、翻译版本与同一题的答案解析也可能越过文档编号, 所以去重单位要与评价目标一致.

训练集决定参数; 验证集帮助选择步长、停止点与配置; 测试集只在协议确定后衡量表现. 若观察测试损失上升后选择最好的测试步数, 测试集已经承担了验证功能. 固定随机种子方便复算, 不替代多次初始化或更广的留出划分. 现代语料中的质量过滤、任务混合与合成资料也应记录来源和比例, 否则同一个 D 不能代表同样的训练信息.

一次训练迭代可以写成: 取一批文档并构造 I, Y, M; 计算 logits 与式 (2.1); 反向累计每个参数梯度; 必要时裁剪; 优化器更新; 清空梯度. 验证只做前向且使用固定评价模式. 对含 dropout 的模型, 训练模式与评价模式不同; 有缓存的生成通常在评价模式运行.

设全部梯度拼成向量 g, 范数上限为 $c > 0$, 裁剪后为 $\tilde{g} = g \min(1, c/\|g\|)$, 约定 $g = 0$ 时不作缩放. 裁剪不改变本步方向, 但改变长度, 是优化算法的一部分. Adam 对每个参数坐标更新

$$m_s = \beta_1 m_{s-1} + (1-\beta_1)\tilde{g}_s, \quad v_s = \beta_2 v_{s-1} + (1-\beta_2)\tilde{g}_s^2,$$
$$\widehat{m}_s = m_s/(1-\beta_1^s), \quad \widehat{v}_s = v_s/(1-\beta_2^s), \quad \theta_s = \theta_{s-1} - \eta_s \widehat{m}_s/(\sqrt{\widehat{v}_s} + \epsilon).$$

平方、开根号与除法均逐坐标进行, $m_0 = v_0 = 0$. 若梯度恒为 g, 则 $m_s = (1-\beta_1^s)g$, 所以偏差修正避免初期平均量被零初始化缩小. 这不是对一般非凸目标收敛到全局最优的保证.

显存容不下大批次时可以累计多个小批次梯度. 若第 j 批有效 token 数为 S_j, 其平均梯度为 g_j, 目标整体梯度为 $\sum_j S_j g_j / \sum_j S_j$. 只有各 S_j 相等时才可直接平均 g_j. 同样地, 多设备训练应按实际有效 token 数加权, 不能把填充比例不同的设备损失等权平均后宣称等价.

2.4 小语言模型的代码桥梁

`experiments/run_all.py` 的 `TinyDecoder` 是一个可逐行求导的因果网络. 它保留 token 与位置嵌入、单头注意力、残差和前馈网络, 省略归一化与 dropout. 本节第一次阅读只需确认整数输入、逐位置输出和损失之间的轮廓. Q/K/V、残差和矩阵反向的具体运算在第 3、4 章逐步推导; 学完后再回到本节逐行复算. 这份简化程序不等同于完整现代大模型.

给定 $I \in \{0,\dots,V-1\}^{B\times T}$, 嵌入查表得 $X = E[I] + P[:T] \in \mathbb{R}^{B\times T\times d}$, 其中 $E \in \mathbb{R}^{V\times d}, P \in \mathbb{R}^{T_{\max}\times d}$. 同一批中每份文档共享位置向量. $Q = XW_Q, K = XW_K, V' = XW_V$ 均为 $B\times T\times d$. 在每个批行中, $QK^\top/\sqrt{d}$ 为 $T\times T$, 下三角掩码后按行 softmax 得 A. 注意力输出 $C = AV'$ 又回到 $B\times T\times d$. 后续计算为

$$H = X + CW_O, \quad F = \tanh(HW_F + b_F), \quad Z = (H + FW_G + b_G)U + b_U.$$

$W_F \in \mathbb{R}^{d\times 2d}, W_G \in \mathbb{R}^{2d\times d}, U \in \mathbb{R}^{d\times V}$. 每个偏置在批与时间维上广播. 输入位置使用前七个 token, 目标使用后七个 token, logits 最后一维的每一行因此对应一个分类问题.

原创语法共有八种组合, 例如

```
<bos> cat sees red dog says meow <eos>
```

主语取 cat 或 dog, 颜色取 red 或 blue, 中间动物独立取 cat 或 dog, 叫声由主语决定. 固定六份训练、两份测试, 宽度 16、前馈宽度 32, 总参数为 2,538. 训练全批 Adam 1,400 步, 步长 0.01, $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$, 梯度范数上限为 1. 在输入 `says` 的位置评价叫声, 检查是否利用远处主语.

同一前缀在训练中可能接不同颜色或主语, 所以最优训练损失也不为零. 对前缀 c, 训练后继频率为 $\widehat{p}(\cdot \mid c)$, 模型分布为 q, 则

$$-\sum_y \widehat{p}_y \log q_y = H(\widehat{p}) + \mathrm{KL}(\widehat{p}\|q) \geq H(\widehat{p}).$$

按前缀出现次数加权, 得到这份经验训练资料的不可约下界, 约为 0.2560. 测试中的新组合可能不服从训练后继频率, 所以接近训练下界也不保证泛化. 两个测试样本、一个种子不足以证明稳定的组合能力. 翻转测试样本的中间动物恰好回到训练组合, 这个干预只能检验局部依赖, 不能作为新的组合留出.

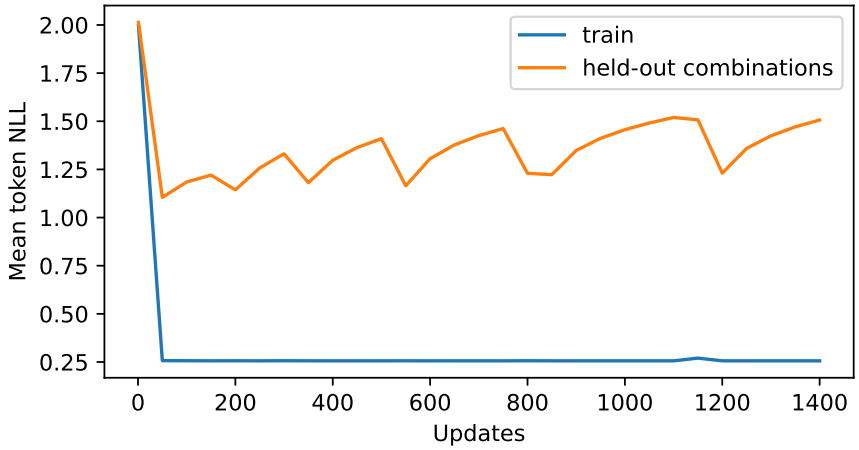


Figure 2.1: 已有固定划分实验的训练与测试 token 损失. 局部叫声正确与整句预测质量是不同指标.

2.5 上下文学习与条件任务推断

在预训练或后训练之后, 可以把任务示例放进提示: “输入 1 输出 2; 输入 2 输出 4; 输入 3 输出?” 推理程序仍使用同一参数 θ. 改变的是前缀 c, 于是改变 $p_\theta(y \mid c, x)$, 不是执行 $\theta \leftarrow \theta - \eta\nabla\mathcal{L}$. 本章把模型通过提示中的示例适应任务的行为称为上下文学习, 即 in-context learning, 简记 ICL. 它可在一次前向与后续生成中发生, 不要求任何优化器步骤. GPT-3 论文的摘要明确把 few-shot 示例置于文本交互中, 未在这些评价中更新模型权重 [37].

为什么不更新权重也能适应任务? 先研究一个真正可算的固定预测器. 假设隐藏任务 h 只可能是 $h_0 : y = x$ 与 $h_1 : y = 1 - x$, 先验各为 $1/2$. 观察含噪示例时, 正确标签概率为 0.9, 翻转概率为 0.1, 各示例在给定任务后独立. 若两个示例都符合 h_0, 则 Bayes 公式给出

$$P(h_0 \mid c) = \frac{(1/2)0.9^2}{(1/2)0.9^2 + (1/2)0.1^2} = \frac{81}{82}.$$

对新输入 $x = 1$, 标签为 1 的预测概率不是 $81/82$, 因为新标签仍含噪声, 而是 $(81 \cdot 0.9 + 1 \cdot 0.1)/82 = 73/82$. 先验与噪声规则都没有更新, 改变的是根据当前提示得到的任务后验. 因而 “参数固定” 与 “行为能适应” 并不矛盾.

首次阅读可先看随后的一维数字例, 再按需阅读一般矩阵形式. 向量、正定矩阵、谱与逆的工具见附录 A 的表示空间与谱/SVD 小节; 离散条件概率和期望见附录 B. 本例额外调用如下明示的连续高斯密度与积分性质, 不要求把它们误当作此前已证明的内容. 下一例把离散任务推广到连续参数. 为避免把连续概率当作已学内容, 先约定: 密度 $f(w)$ 非负且积分为 1, 区域概率为在该区域上的积分, 条件密度仍由先验乘似然后归一化得到. 一维高斯密度为 $(2\pi\sigma^2)^{-1/2}\exp[-(w-\mu)^2/(2\sigma^2)]$, 均值为 μ, 方差为 σ^2. 多维正定协方差 Σ 的相应密度为

$$(2\pi)^{-d/2}(\det\Sigma)^{-1/2}\exp[-\tfrac{1}{2}(w-\mu)^\top\Sigma^{-1}(w-\mu)].$$

它的均值与协方差可由对称性、正交对角化和一维高斯积分核对. 本例使用这些高斯积分性质作为明确调用的工具, 核心工作是推导提示怎样改变后验与预测; 先复算随后的一维数字也能理解这一关系.

命题 2.4 设隐藏任务参数 $w \in \mathbb{R}^d$ 的先验密度与 $\exp(-\|w\|^2/2)$ 成比例, 支持示例满足 $y = Xw+\varepsilon$, 其中 $X \in \mathbb{R}^{n\times d}$, $y \in \mathbb{R}^n$, 噪声向量 ε 独立于 w, 各分量互相独立且为方差 $\sigma^2 > 0$ 的零均值高斯量. 令

$$\Lambda = I_d + X^\top X/\sigma^2, \qquad \mu = \Lambda^{-1}X^\top y/\sigma^2.$$

任务后验为均值 μ、协方差 Λ^{-1} 的高斯分布. 对新输入 $x_* \in \mathbb{R}^d$, 新噪声 ε_* 独立于 w 和全部训练噪声, 且同为方差 σ^2 的零均值高斯量. 标签预测均值为 $x_*^\top\mu$, 方差为 $\sigma^2 + x_*^\top\Lambda^{-1}x_*$.

证明. 先验乘似然后取负对数, 与 w 有关的部分为

$$\tfrac{1}{2}w^\top w + \tfrac{1}{2\sigma^2}(y - Xw)^\top(y - Xw) = \tfrac{1}{2}w^\top\Lambda w - w^\top X^\top y/\sigma^2 + \text{常数}.$$

Λ 正定, 因为对非零 v, 有 $v^\top\Lambda v = \|v\|^2 + \|Xv\|^2/\sigma^2 > 0$. 利用 $\Lambda\mu = X^\top y/\sigma^2$ 配方, 上式成为 $\frac{1}{2}(w - \mu)^\top\Lambda(w - \mu)$ + 常数, 得到后验密度. 新噪声与 w 独立, 所以 $y_* = x_*^\top w + \varepsilon_*$ 的均值与方差分别按线性期望和独立方差相加得到. □

一维时取 $X = (1, 2)^\top, y = (2, 4)^\top, \sigma^2 = 1$. 有 $\Lambda = 6, \mu = 10/6 = 5/3$. 查询 $x_* = 3$, 预测均值为 5, 方差为 $1 + 9/6 = 2.5$. 无示例时先验均值为零, 预测方差为 $1 + 9 = 10$. 即使两点刚好共线, 有限噪声与先验收缩仍使预测均值不等于 6. `v03_language.py` 的 `icl_experiment` 按这个固定公式计算, 可训练权重更新次数为零.

这个例子说明了一种可能的任务推断机制, 没有证明 Transformer 都在执行这套 Bayes 计算. 隐式 Bayes 的相关论文提出了特定生成模型下的理论解释并报告受控实验 [38]; 其条件不是任意自然语言提示都满足的定理. 同样, 线性后验均值可以与正则化最小二乘得到同一数值, 也不说明模型内部真的运行梯度下降. 识别机制需要对任务分布、示例顺序、错误示例与查询作干预, 比较不同机制的预测, 而不是只看几个正确答案.

2.6 采样分布与序列决策

在每步 logits z 上, 贪心解码取 $\arg\max_i z_i$. 它优化当前条件概率, 并不优化整条序列概率. 例如第一步 $p(a) = 0.6, p(b) = 0.4$, 接下来在 a 后最好结束概率为 0.5, 在 b 后结束概率为 1. 贪心路径概率为 $0.6 \cdot 0.5 = 0.3$, 另一条为 0.4. 比较序列概率时还要固定终止和长度处理, 不能把不同长度的分数随意混在一起.

温度采样把条件分布改为

$$q_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \qquad \tau > 0.$$

有限词表下, $\tau \to \infty$ 时趋于均匀分布; $\tau \to 0$ 时集中到最大 logit 集合, 若最大值并列则在它们之间均匀. 由 $q_i/q_j = e^{(z_i - z_j)/\tau}$ 可直接看出降温加强已有排序. 若排序最高的是错误答案, 降温会增加这种错误被选中的概率.

Top-k 按概率排序保留前 k 项后重归一化. Top-p 先排序, 找累计概率首次达到阈值 $p \in (0, 1]$ 的最短前缀, 保留它再归一化. 必须包含使累计概率越过阈值的那一项, 否则可能保留空集合. 并列项需要确定的排序规则. 同时使用两者时还要说明顺序: 新实验先应用 k, 在所剩分布上归一化, 再应用 p.

若原分布为 $(0.4, 0.3, 0.2, 0.1)$, 温度 $1/2$ 相当于平方后归一化, 得 $(16, 9, 4, 1)/30$. 阈值 $p = 0.65$ 保留前两项, 得 $(4/7, 3/7, 0, 0)$. 两者并不相同: 降温仍保留全部支持, 截断直接排除候选. 固定种子 144 下每种设置抽样 50,000 次,

$p = 0.65$ 的前两项实测频率为 0.56906 与 0.43094, 可与上述精确概率比较. 这检验采样实现, 没有包含答案真伪标签.

2.7 KV 缓存的等价性与成本

生成新 token 前, 先处理整段提示, 称为 prefill; 此后逐 token 推进, 称为 decode. 不缓存时, 可在每轮对完整前缀重新运行网络. 因果结构使大量旧计算保持不变, 所以可以复用.

命题 2.5 *固定参数、位置编号与前缀, 在评价模式下使用确定的逐位置前馈运算与因果注意力. 每层缓存旧位置的键和值, 新位置只计算自身隐藏状态、查询、键和值, 得到的末位置 logits 与重新计算完整前缀相同, 在实数精确运算意义下成立.*

证明. 先对层数归纳. 输入层中, 旧位置的 token 与位置嵌入没有改变. 假定一层输入在旧位置保持不变, 则它们的键和值也不变. 旧位置的因果注意力只读取不晚于自身的位置, 所以新增位置不会影响它们; 逐位置残差、归一化与前馈网络亦不受影响. 对新位置, 缓存给出了完整前向中全部旧键值, 其自身的新键值也按同一输入计算. 相同查询与全部可见键产生相同注意力权重, 因而新位置输出一致. 此论证逐层成立, 最终逐位置输出映射也一致. □

这个条件排除了重新编号旧位置、修改前缀、丢弃部分缓存、随机 dropout 等情形. 采用旋转位置编码时, 也应在写入键缓存时使用正确绝对位置或等价的相对位置约定. 量化缓存会引入数值近似, 不能仍宣称与未量化的完整前向严格相同. 实数等价也不要求不同矩阵计算顺序在浮点中逐位一致.

单层单头当前长度 t 的新查询形状为 $B \times 1 \times d$, 缓存键值均为 $B \times t \times d$, 分数形状为 $B \times 1 \times t$. 一轮的键查询与加权值乘积成本为 $O(Btd)$, 新位置线性投影与前馈成本约为 $O(Bd^2)$, 输出词表投影为 $O(BdV)$. 对 L 层的多头模型, 若键值各头总宽度为 d_{KV}, 长度为 t, 缓存元素为 $2BLtd_{\mathrm{KV}}$. 每元素占 s 字节时, 主体内存为 $2BLtd_{\mathrm{KV}}s$, 未计元数据与临时激活. 分组查询让多个查询头共享键值头, 可以减小 d_{KV}.

从空前缀依次生成 T 个输入位置时, 缓存注意力总成本为 $O(LBT^2d)$, 新位置投影总成本为 $O(LBTd^2)$. 若每轮重新计算完整前缀, 注意力成本为 $O(LB\sum_{t=1}^{T} t^2 d) = O(LBT^3d)$, 投影为 $O(LBT^2d^2)$. 这描述特定重复前向实现的算术量, 不表示所有解码系统的墙钟时间按这个比率改善. 当输出投影、缓存读带宽或小矩阵调度占主导时, 瓶颈不同.

新实验复用 `TinyDecoder` 的固定参数, 独立实现 `CachedDecoder.step`. 对 $B = 3, T = 12, d = 8, V = 10$, 比较全部十二个前缀的末位置 logits, 最大绝对差为 2.22×10^{-16}. 修改最后 token 对旧位置 logits 的最大影响为零. 缓存共 576 个数. 重新计算完整分数矩阵的槽位总数为 $3\sum_{t=1}^{12} t^2 = 1950$, 增量计算为 $3\sum_{t=1}^{12} t = 234$; 这是可核对的计数, 没有冒充测速结果.

2.8 复算与练习

新增实验的统一入口为 `experiments/v03_language.py`, 只使用 NumPy、原创有限资料与旧小模型参数接口. 运行命令为

```
PYTHONDONTWRITEBYTECODE=1 python experiments/v03_language.py --seed 144
```

结果保存在 `results/v03-language/metrics.json`, 环境、参数来源哈希与种子保存在同目录的 `environment.json`. 该命令不训练或下载外部语言模型. 实验中的数学等价性由前面证明支持, 数值运行用于发现实现偏差; 误差容限与七项机制测试见 `tests/test_v03_language.py`.

习题 2.1 用 2 个有效 token 的小批和 6 个有效 token 的小批累计梯度, 两个按 token 平均的梯度分别是 g_1, g_2. 写出大批梯度. 若先平均二者再把学习率调整一个常数, 能否对任意 g_1, g_2 恢复它?

解答. 正确梯度为 $g_1/4 + 3g_2/4$. 直接平均给 $(g_1 + g_2)/2$. 若只乘常数 c, 需要同时 $c/2 = 1/4$ 与 $c/2 = 3/4$, 不可能. 问题是样本权重改变, 不是统一梯度尺度改变. □

习题 2.2 二元任务算例中, 两个示例一个符合 h_0, 一个符合 h_1. 后验如何? 若模型得到同样答案, 是否足以认定它内部使用 Bayes 推断?

解答. 两个任务的似然均为 $0.9 \cdot 0.1$, 后验仍各为 $1/2$, 新标签预测也各为 $1/2$. 同一输出可能由查表、计数规则或 Bayes 推断实现. 应设计多种示例数量、噪声与先验条件, 检查各候选机制的不同预测; 有限成功仍不能唯一确定内部算法. □

3 表征网络与反向传播

3.1 从标量函数到可学习表征

语言模型并不直接在词编号上做算术. 它先把离散编号换成向量, 再让这些向量在多层运算中随上下文改变, 最后映成下一个 token 的概率. 中间向量称为表征: 它是后续计算使用的数值状态, 不要求每个坐标都有一个可命名的语义. 本章学习这些数值状态怎样由参数产生, 又怎样把预测误差传回所有共享参数. 下一章会把局部运算组合为注意力和 Transformer.

先用一个只有两个参数的网络说明反向传播. 输入 x 是固定实数, 定义 $a = wx$, $h = \tanh a$, $z = vh$, 目标为 y, 损失 $\ell = (z - y)^2/2$. 三个中间变量让我们无须每次重新展开整个复合函数. 记 $g_z = \partial \ell / \partial z = z - y$. 从 $z = vh$ 得 $g_v = g_z h$, $g_h = g_z v$; 从 $h = \tanh a$ 得 $g_a = g_h(1 - h^2)$; 最后从 $a = wx$ 得 $g_w = g_a x$. 合起来是

$$\frac{\partial \ell}{\partial v} = (z - y)h, \qquad \frac{\partial \ell}{\partial w} = (z - y)v(1 - h^2)x.$$

每一步只需知道当前节点的局部导数和下游传来的系数. 为何这能算出整体导数? 因为 $d\ell = g_z\, dz$, 而 $dz = h\, dv + v\, dh$, $dh = (1 - h^2)\, da$, $da = x\, dw$; 代入后 dw, dv 的系数恰是上述结果. 这里 d 表示围绕当前数值的一阶线性变化, 如 $d(wx) = x\, dw$ 表示当 w 有小增量时, 输出的一阶变化系数为 x. 对一般非线性函数, 微分等式描述线性主项, 不是声称任意有限增量都没有余项. 这就是链式法则在计算图上的组织方式.

若一个变量通向多条路径, 微分的系数必须相加. 比如 $z = vh + uh$, 则 $dz = h\, dv + h\, du + (v + u)\, dh$, 因而 $g_h = (v + u)g_z$. 只沿其中一条边返回会漏掉另一项. 查表、共享矩阵、残差和注意力都会重复出现这一原则.

层间非线性改变了网络能表示的函数. 两个仿射层 $h = A_1x+b_1, z = A_2h+b_2$ 合并后仍是 $z = A_2A_1x+A_2b_1+b_2$. 只增加仿射层数没有增加函数族. 逐坐标非线性可以让输入在不同区域走过不同的运算. 对二元输入, XOR 的正例是 $(1,0),(0,1)$, 负例是 $(0,0),(1,1)$. 若 $w_1x_1 + w_2x_2 + b$ 能严格分开它们, 则正例给 $w_1 + w_2 + 2b > 0$, 两个负例相加却给同一量小于零, 矛盾. 但 $h_1 = \mathrm{ReLU}(x_1 - x_2)$ 与 $h_2 = \mathrm{ReLU}(x_2 - x_1)$ 使 $h_1 + h_2$ 在正例为 1, 负例为 0. 这是一个可直接检查的非线性表征.

定义 3.1 本章的逐坐标激活使用 $\tanh a = (e^a - e^{-a})/(e^a + e^{-a})$, 或 $\mathrm{ReLU}(a) = \max(0, a)$. 前者处处可微, 导数为 $1 - \tanh^2 a$; 后者在 $a < 0$ 导数为 0, 在 $a > 0$ 导数为 1, 在 $a = 0$ 没有普通导数.

实现通常把 ReLU 在零点的反传值约定为 0, 有些理论讨论允许在 $[0, 1]$ 中选择一个次梯度. 这个约定不是证明它在零点可微. 若有限差分跨过拐点, 中心差分可能得到 1/2, 与实现约定不同; 这类差异应先检查不可微位置, 不能直接断言反传有错. tanh 便于本书的平滑梯度实验; 现代 Transformer 的前馈层也会使用其他平滑或门控激活, 其反向仍服从同一局部求导规则.

3.2 矩阵梯度由坐标系数决定

本章统一采用 “样本或 token 在行上, 特征在列上” 的代码记号. 单 token 输入 $x \in \mathbb{R}^{1\times d}$, 两层网络为

$$a = xW_1 + b_1, \quad h = \tanh a, \quad z = hW_2 + b_2,$$

其中 $W_1 \in \mathbb{R}^{d\times r}$, $b_1 \in \mathbb{R}^{1\times r}$, $W_2 \in \mathbb{R}^{r\times V_0}$, $b_2 \in \mathbb{R}^{1\times V_0}$. z 是 logits, 还不是概率; $\mathrm{softmax}(z)$ 才是 V_0 类的概率. 行记号的权重与常见列记号转置, 不能把两种约定的式子直接混用.

矩阵梯度的定义如下: 对实标量损失 L, 梯度 G_W 与参数 W 同形, 并满足

$$dL = \langle G_W, dW\rangle_F = \sum_{i,j} (G_W)_{ij}(dW)_{ij}.$$

这里 Frobenius 内积就是 “逐项相乘再求和” . 梯度不是为了凑出合法乘法而猜来的, 是每个独立坐标增量的系数.

以 $Y = XW$ 为例, $X \in \mathbb{R}^{N\times d}$, $W \in \mathbb{R}^{d\times r}$, $Y \in \mathbb{R}^{N\times r}$. 给定 G_Y, 用分量展开:

$$\begin{aligned} dL &= \sum_{n,j} (G_Y)_{nj}\, dY_{nj} \\ &= \sum_{n,j} (G_Y)_{nj} \left(\sum_i dX_{ni}W_{ij} + \sum_i X_{ni}dW_{ij} \right) \\ &= \sum_{n,i} \left(\sum_j (G_Y)_{nj}W_{ij} \right) dX_{ni} + \sum_{i,j} \left(\sum_n X_{ni}(G_Y)_{nj} \right) dW_{ij}. \end{aligned}$$

因此 $G_X = G_YW^\top$, $G_W = X^\top G_Y$. 前者保留每行自己的误差信号, 后者把所有行对同一个权重的贡献汇总. 偏置若广播到每行, dY_{nj} 还含 db_j, 故 $G_{b_j} = \sum_n (G_Y)_{nj}$. 广播的反向不是复制, 而是对使用该参数的所有位置求和.

对一个真实类别 y, 损失 $\ell = -\log \mathrm{softmax}(z)_y$. 把损失写成 $\ell = \log \sum_v e^{z_v} - z_y$, 对 z_j 求导即得 $q_j - \mathbf{1}\{j = y\}$.

因而 logits 梯度是行向量 $D_2 = q - e_y$, 后续为

$$G_{W_2} = h^\top D_2, \qquad G_{b_2} = D_2, \qquad G_h = D_2 W_2^\top,$$
$$D_1 = G_a = G_h \odot (1 - h^2), \qquad G_{W_1} = x^\top D_1, \qquad G_{b_1} = D_1.$$

$\odot$ 表示同形数组逐项相乘. h^2 也逐项计算. G_h 与 h 同形, D_1 与 a 同形; 每一层的参数梯度才在另一轴上收集所有样本的贡献.

习题 3.1 令 $X = \left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$, $W \in \mathbb{R}^{2\times 1}$, 上游梯度 $G_Y = (2, -1)^\top$. 求 G_W, 并解释为何不能只取第一行的贡献.

解答. $G_W = X^\top G_Y = (1\cdot 2 + 3\cdot(-1), 2\cdot 2 + 4\cdot(-1))^\top = (-1, 0)^\top$. 第一行单独贡献 $(2, 4)^\top$, 第二行贡献 $(-3, -4)^\top$, 同一个权重列在两行都被使用, 因而必须相加. 激活梯度 $G_X = G_Y W^\top$ 则分别保留两行, 它与参数梯度的汇总目的不同. □

3.3 批次、位置与查表共享

语言模型的批次是三维数组. 用 B 表示文档数, T 表示输入位置数, D 表示特征数, V_0 表示词表大小. 为直接对应 Python, 下标从零开始. 整数数组 `ids` 形状为 (B, T), 元素在 $0, \dots, V_0 - 1$ 中; 词表矩阵 $E \in \mathbb{R}^{V_0 \times D}$ 的一行是一个 token 的初始表征, 位置矩阵 $P \in \mathbb{R}^{T\times D}$ 为不同位置提供向量. 查表称为 gather, 前向定义为

$$X_{bti} = E_{\mathrm{ids}_{bt}, i} + P_{ti}.$$

因此 `E[ids]` 的形状为 (B, T, D), 而 `P[:T]` 的形状为 (T, D). 相加时 NumPy 把后者看成 $(1, T, D)$, 沿批次轴重复使用. 这就是广播: 形状允许共享使用一个参数, 不需要在参数表中真的存 B 份副本.

对应的反向先得到 $G_X \in \mathbb{R}^{B\times T\times D}$, 再收集路径:

$$(G_P)_{ti} = \sum_{b} (G_X)_{bti}, \qquad (G_E)_{vi} = \sum_{b,t:\, \mathrm{ids}_{bt} = v} (G_X)_{bti}.$$

第一式是沿批次轴的普通求和, 第二式是按词编号分组求和. 同一个 token 重复出现, 它对同一参数行的贡献必须全部保留. 整数编号自身不是本书中需要求导的连续参数; 反向更新的是选中的词表向量, 不是把编号 2 改成 2.1.

例题 3.1 词表有 3 行、每行 2 维, 两个位置的词编号均为 1. 上游梯度分别为 $(2, -1)$ 和 $(3, 4)$. 写出正确词表梯度, 并比较两种代码.

解答. 编号 1 指第二行, 其梯度为 $(5, 3)$, 其余行是零. 代码为

```
ids = np.array([1, 1])
dx = np.array([[2., -1.], [3., 4.]])
good = np.zeros((3, 2)); bad = good.copy()
np.add.at(good, ids, dx)
bad[ids] += dx
```

本项目 NumPy 环境中, 第二行分别得到 `[5. 3.]` 和 `[3. 4.]`. 高级索引的增量赋值可能先操作临时取出的数组, 重复索引不能表达逐出现次数的可靠累加. `np.add.at` 明确执行按索引的无缓冲累加, 因而适合这里的反传. 若所有编号互异, 错误代码可能暂时给出相同结果; 测试必须包含重复词. □

在每个 (b,t) 上共享 $W\in\mathbb{R}^{D\times R}$, 则 $Y_{btr}=\sum_i X_{bti}W_{ir}+b_r$, 形状是 (B,T,R). 仿射层的梯度为

$$G_X=G_YW^\top,\quad (G_W)_{ir}=\sum_{b,t}X_{bti}(G_Y)_{btr},\quad G_b=\sum_{b,t}(G_Y)_{bt,:}.$$

实现把前两轴展为 BT 行:

```
xflat = x.reshape(-1, D)
dyflat = dy.reshape(-1, R)
gW = xflat.T @ dyflat
gx = dy @ W.T
gb = dy.sum(axis=(0, 1))
```

展平后的第 $bT+t$ 行对应原来的 (b,t); 两个数组必须采用相同次序. 展平没有平均、没有求和, 是一次重新组织索引; 后面的矩阵乘法才执行对这些行的求和. 本项目旧模型的 `mm(x,y)` 就是这条展平后的共享矩阵梯度公式.

3.4 从平均损失到参数更新

令每个位置预测下一个 token, logits 为 $Z\in\mathbb{R}^{B\times T\times V_0}$, 目标数组为 $Y\in\{0,\dots,V_0-1\}^{B\times T}$. 没有 padding 时, 平均损失和其导数为

$$L=-\frac{1}{BT}\sum_{b,t}\log q_{bt,Y_{bt}},\qquad (G_Z)_{btv}=\frac{q_{btv}-\mathbf{1}\{v=Y_{bt}\}}{BT}.$$

分母在损失定义时已经确定, 所以进入 G_Z 后, 后面每层不得再除以 B 或 BT. 若同一批次完整复制两份, 平均损失和所有参数梯度都应相同; 这是一个能抓住重复平均错误的不变量.

有 padding 时, 令 $m_{bt}\in\{0,1\}$ 表示这个位置是否计入损失, $N=\sum_{b,t}m_{bt}>0$. 平均损失改为

$$L=-\frac{1}{N}\sum_{b,t}m_{bt}\log q_{bt,Y_{bt}}.$$

对应的 logits 梯度是 $m_{bt}(q-e_Y)/N$. Padding 的注意力掩码与损失掩码目的不同: 一个限制信息读取, 一个决定哪些输出被计分. 只把 padding 的损失设为零, 不一定阻止其他位置读取它; 只把 padding 的键遮住, 也不等于它的输出不计入损失.

计算出所有梯度后, 普通梯度下降同时更新 $\theta\leftarrow\theta-\eta G_\theta$. “同时”指每组参数都使用本次前向的旧参数及其反向梯度. 若先更新 W_2, 再用新的 W_2 返回 G_h, 得到的就不是同一个损失在同一点的梯度. 实现时先完成整个反向并存梯度, 再统一修改参数.

为什么正确梯度在小步长下能降损失? 把所有参数视为一个长向量, 可微性给 $L(\theta-\eta g)=L(\theta)-\eta\|g\|^2+o(\eta)$, 其中 $g=\nabla L(\theta)$. 若 $g\neq0$, 足够小的正 η 使线性下降项胜过余项. 这仅是当前点附近的结论, 不能保证任意步长、任意迭代都下降, 更不能保证测试风险下降. ReLU 的拐点还需要单独处理非光滑情形.

习题 3.2 一个实现先令 logits 梯度除以 BT, 再把每个矩阵梯度除以 BT. 若 $B = 2, T = 3$, 它算的是原目标的梯度吗? 如何用复制批次定位?

解答. 不是. 单个矩阵梯度又缩小了 6 倍, 而传播到上游的激活梯度若没有同步缩放, 各组参数还会出现不一致. 复制整批次后 BT 变为 12, 正确平均梯度应保持不变, 错误矩阵梯度却再缩小一半. 先检查 logits 处平均因子, 再检查各共享参数的求和; 平均应只来自损失定义一次. □

3.5 残差、初始化与梯度尺度

Transformer 使用残差组合 $Y = X + F(X)$. 对增量有 $dY = dX + J_F[dX]$, 因而上游梯度 G_Y 回到输入时成为 $G_X = G_Y + J_F^\top[G_Y]$. 这里 J_F 是把输入增量映到输出增量的线性映射, 转置表示把微分系数传回输入. 恒等路径的 G_Y 不能漏掉. 若 F 有自己的参数, 恒等支路没有这些参数, 所以它不直接增加 F 的参数梯度.

多层网络的激活梯度会反复乘局部导数. 对标量链, 若每层导数都为 0.5, 20 层后系数成为 2^{-20}; 若都为 2, 则成为 2^{20}. 矩阵版本可用线性映射的范数解释: 定义 $\|J\| = \sup_{u\neq 0}\|Ju\|/\|u\|$, 则从定义直接得 $\|J_1 J_2\| \le \|J_1\|\|J_2\|$. 残差把局部导数改为 $I + J_F$, 提供直接路径, 但若这些映射的乘积过大或发生方向抵消, 梯度仍可爆炸或衰减. 残差不构成无条件稳定性保证.

初始化先要破除隐单元的对称性. 若所有隐单元使用相同权重和偏置, 又接受相同下游连接, 它们在确定性更新下可能始终相同, 无法学出不同特征. 随机初始化给出不同起点. 还要控制前向信号尺度: 若 $a_j = \sum_{i=1}^{d} W_{ij}x_i$, 输入居中且独立、方差为 v, 权重相互独立、均值零、方差 s^2, 并独立于输入, 则

$$\mathbb{E}a_j = 0, \qquad \mathrm{Var}(a_j) = \sum_i \mathbb{E}W_{ij}^2 \mathbb{E}x_i^2 = ds^2 v.$$

交叉项因零均值与独立性消失. 令 s^2 随 $1/d$ 缩放, 可避免单层求和使方差随输入宽度增长. 这是当前假设下的精确计算, 对训练后相关表征只是尺度设计的参考. 对称且近零均值的高斯预激活, ReLU 把负半轴置零, 其输出二阶矩为预激活二阶矩的一半, 所以常用 $2/d$ 的权重方差补偿这一二阶矩变化; 二阶矩不是方差, 因为 ReLU 输出通常均值为正.

对于 tanh, 大幅度预激活使 $1 - h^2$ 接近零, 回传信号可能消失. 实际诊断应同时检查层输出的均方尺度、饱和比例、每组梯度范数和更新范数 $\eta\|G_W\|/\max(\|W\|, \varepsilon)$. 梯度很小可能是模型已接近驻点, 也可能是饱和、错误平均或漏路径. 梯度很大可能来自大损失、异常输入或归一化分母过小. 单一范数没有足够信息确定原因.

3.6 LayerNorm 的数值与完整导数

归一化让每个 token 的各特征在相近尺度下进入后续层. LayerNorm 沿特征轴计算, 不沿批次或位置轴计算 [34]. 对单个 token $x \in \mathbb{R}^D$, 定义

$$\mu = \frac{1}{D}\sum_i x_i, \quad c_i = x_i - \mu, \quad s^2 = \frac{1}{D}\sum_i c_i^2, \quad r = (s^2 + \epsilon)^{-1/2}, \quad \widehat{x}_i = rc_i, \quad y_i = \gamma_i \widehat{x}_i + \beta_i.$$

这里 $D \ge 1$, $\epsilon > 0$ 是固定常数, $\gamma, \beta \in \mathbb{R}^D$ 为可训练的共享参数. 方差分母使用 D, 不是统计中的无偏样本方差分母 $D - 1$. 对 (B, T, D) 输入, 每个 (b, t) 分别计算一组 μ, s^2, r, 而 γ, β 对所有 token 共享.

归一化结果满足 $\sum_i \widehat{x}_i = 0$, 但其平均平方为 $s^2/(s^2+\epsilon)$, 并不精确等于 1. 加上 γ, β 后, 输出也不必零均值. 正 ϵ 保证对任何有限输入分母非零, 整个函数处处可微. 若 $\epsilon = 0$ 且所有特征相同, 就未定义; 若 $\epsilon = 0$ 且方差正, 才可在该开集上使用同样微分式.

为求导, 先让 dx_i 表示输入增量. 因 $\sum_i c_i = 0$,

$$d\mu = \frac{1}{D}\sum_i dx_i, \qquad dc_i = dx_i - d\mu,$$

$$d(s^2) = \frac{2}{D}\sum_i c_i\, dc_i = \frac{2}{D}\sum_i c_i\, dx_i, \qquad dr = -\frac{r^3}{D}\sum_i c_i\, dx_i.$$

因此

$$d\widehat{x}_i = r\left(dx_i - \frac{1}{D}\sum_j dx_j\right) - \frac{r^3 c_i}{D}\sum_j c_j\, dx_j.$$

给定上游梯度 $g_i = \partial L/\partial y_i$, 令 $u_i = g_i\gamma_i$. 将上式乘 u_i 并对 i 求和, dx_j 的系数为

$$(G_x)_j = r\left(u_j - \overline{u} - \widehat{x}_j\,\overline{u\widehat{x}}\right), \qquad \overline{u} = \frac{1}{D}\sum_i u_i, \quad \overline{u\widehat{x}} = \frac{1}{D}\sum_i u_i\widehat{x}_i.$$

最后一项用了 $r^3 c_j c_i = r\widehat{x}_j\widehat{x}_i$. 这一步解释了常见反向代码里两个均值项的来源, 不能把它们当作额外的 “梯度归一化” 随意删掉. 对可训练向量, 单 token 有 $G_{\gamma_i} = g_i\widehat{x}_i, G_{\beta_i} = g_i$; 整批次则沿 (b,t) 求和.

例题 3.2 当 $D = 4$, 输入四个特征均为 3, $\gamma_i = 1$, $\beta_i = 0$, $\epsilon = 10^{-5}$, 上游梯度为 $(1,2,3,4)$, 求输出与输入梯度.

解答. 此时 $c = 0, s^2 = 0, \widehat{x} = 0, y = 0$. $\overline{u} = 2.5$, 第三个归一化导数项为零, 所以

$$G_x = (10^{-5})^{-1/2}(-1.5, -0.5, 0.5, 1.5).$$

输出完全有限, 梯度也有限, 但其范数约为 707.107, 可以很大. 这说明 “加 epsilon 防止除零” 不等于 “梯度尺度一定合适”. $D = 1$ 时所有输入都被减为零, 输出恒为 β, 公式相应给 $G_x = 0$. □

若输入所有特征加同一常数, c 不变, 输出不变, 所以输入梯度的坐标和应为零; 上式也直接验证这一点. 这是一项不依赖有限差分的结构检查. 有正 ϵ 时, 整体缩放输入并不严格保持输出, 因为分母中的固定 ϵ 不随输入缩放. 不应从近似行为推成精确不变性.

3.7 从公式到诊断实验

旧 `mlp` 实验只检验四点 XOR 的拟合与手写导数, 用 8 个 tanh 隐单元、4000 次更新、步长 0.1. 它没有自然数据上的独立测试, 因而拟合正确率不是语言模型泛化证据. 原损失曲线仍保留, 用于说明一个完整的前向、反向和更新闭环:

新增 `batch` 实验更直接对应 token 网络. 输入是 $B = 2, T = 3$ 的词编号, 含重复编号. 模型先算 $X = E[\text{ids}] + P$, 再算

$$H = \tanh(XW_1 + b_1), \qquad Z = HW_2 + XU + b_2.$$

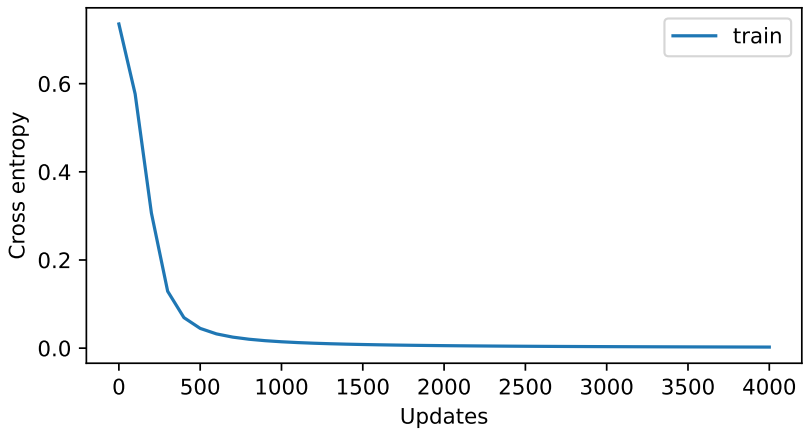


Figure 3.1: 原 XOR 实验的平均交叉熵, 横轴为实际更新次数. 这是一项最小反传练习, 不代表基础模型能力.

它有一条非线性路径与一条直接路径. 对 G_Z, 正确输入梯度为 $G_X = ((G_Z W_2^\top) \odot (1 - H^2))W_1^\top + G_Z U^\top$. 故意删去第二项, 前向损失完全不变, W_1, W_2, U, b_1, b_2 的梯度也可以仍然正确; 错误只继续传到查表和位置表 E, P. 测试对全部 64 个参数坐标进行中心差分, 种子 144 的正确归一化误差最大为 1.61×10^{-11}, 故障恰好定位到 E, P. 同一实验复制批次后梯度保持不变, 并用步长 0.05 检查一次更新确实降低当前损失.

有限差分检验的数值为 $[L(\theta + he_j) - L(\theta - he_j)]/(2h)$, 本次 $h = 10^{-5}$. 报告的坐标误差为 $|g_j^{\mathrm{num}} - g_j| / \max(1, |g_j^{\mathrm{num}}|, |g_j|)$. 分母中的 1 避免极小梯度的相对误差失真, 因而这是混合绝对与相对标准, 不是所有坐标严格的相对误差. h 过小会遇到浮点相消, 过大会带来截断误差; 有疑问时应检查多个步长和不可微点.

`layernorm` 实验对 X, γ, β 的全部 32 个坐标检查导数, 最大误差约为 1.73×10^{-10}; 测试另用随机方向的差分核对 $\langle G_X, U \rangle_F$, 并检查平移不变性与梯度坐标和为零. 数值检查只验证本次输入、参数和环境; 一般正确性来自前面的微分推导. 自动微分框架即使正确计算了梯度, 也不会自动修复污染数据、错位目标或错误损失定义.

可从项目根目录执行下列命令. 新结果独立存放, 旧实验和旧结果保持原有身份:

```
python experiments/v03_networks.py --seed 144 --out results/v03-networks
python -m unittest discover -s tests -p test_v03_networks.py -v
```

习题 3.3 共享矩阵 W 同时产生 XW 与 UW, 两个输出的上游梯度分别为 A, B. 求 G_W. 若一条输出是残差支路, 另一条是非线性支路, 能否只保留范数较大的那条?

解答. 由两条微分路径相加, $G_W = X^\top A + U^\top B$. 不能按范数选择一条, 因为损失对该参数的总一阶变化是两条系数之和; 两条梯度还可能相互抵消. 若要删支路, 必须改变前向模型与目标, 而不是在原模型的反向中随意忽略它. □

习题 3.4 某层的 tanh 激活绝对值几乎都大于 0.999, 下游梯度不小, 但输入侧梯度很小. 写出数值原因与两项检查. 为什么只把学习率增大未必合适?

解答. 局部导数 $1-h^2$ 小于约 0.002, 它逐坐标缩小回传信号. 应检查预激活分布与权重初始化尺度, 同时核对归一化、输入尺度以及是否重复平均. 增大学习率同时放大未饱和位置和其他参数的更新, 可能使它们不稳定, 却不直接消除饱和机制. 应根据各组梯度和更新尺度判断, 而不是仅凭某组梯度小就改一个全局步长. □

习题 3.5 有人把 LayerNorm 的均值轴改成所有 (B,T,D) 坐标, 其输出仍有合法形状. 这与逐 token LayerNorm 有何区别? 在因果语言模型中还有什么风险?

解答. 新的均值和方差使一个 token 依赖其他文档及其他位置, 已经不是本章定义的函数. 改变同批其他文档会改变本样本, 改变未来 token 也可能改变前缀的归一化结果. 所以即使注意力使用三角掩码, 整个模型仍可能泄漏未来信息. 形状合法不保证数学对象和信息路径正确; 下一章的因果检验必须覆盖整条前向路径. □

4 注意力与 Transformer 表征

4.1 把上下文匹配写成概率

词表查表给同一个 token 相同的初始向量, 但“苹果很好吃”与“苹果发布产品”中的上下文不同. 逐 token 前馈网络只改变当前向量, 不能直接读取其他位置. 我们需要一个根据当前内容选择上下文, 并把取回的信息写入表征的操作. 注意力把这一任务拆为查询、键和值: 查询描述当前要匹配什么, 键描述每个候选位置怎样被匹配, 值描述选中后要带回什么. 键和值的投影可以不同, 如目录索引与记录正文的作用不同.

固定查询位置 i, 给候选键位置 j 一个实分数 s_{ij}. 对允许集合 $\mathcal{A}_i$, 定义

$$A_{ij}=\frac{e^{s_{ij}}}{\sum_{l\in\mathcal{A}_i}e^{s_{il}}}\quad(j\in\mathcal{A}_i),\qquad A_{ij}=0\quad(j\notin\mathcal{A}_i).$$

若允许集合非空, 这些数非负且和为 1. 可以把它们看成条件于当前查询和候选内容时, 在源位置上的一个分类概率分布. 输出 $C_i=\sum_j A_{ij}V_j$ 是在该分布下取值向量的期望. 这里的“概率”描述匹配权重, 不表示某位置有相同概率包含真答案, 也不表示模型回答正确的概率.

定义 4.1 单头自注意力的输入 $X\in\mathbb{R}^{T\times d}$ 把 token 放在行上. 取 $W_Q,W_K\in\mathbb{R}^{d\times d_k}$, $W_V\in\mathbb{R}^{d\times d_v}$, 定义

$$Q=XW_Q,\quad K=XW_K,\quad V=XW_V,\qquad S=\frac{QK^\top}{\sqrt{d_k}}+M,\quad A=\mathrm{softmax}_{\mathrm{row}}(S),\quad C=AV.$$

Q,K 的形状是 $T\times d_k$, V,C 是 $T\times d_v$, S,A,M 是 $T\times T$. 常量掩码 M 在允许位置为 0, 禁止位置记为 $-\infty$. 这个记号表达“从归一化和中排除”, 不是把无穷当成可训练参数.

点积 $q_ik_j^\top$ 大, 表示在学到的查询与键坐标中相匹配; 参数训练决定哪些方向有用. 查询和值来自同一输入也不意味着它们是同一个对象. 在交叉注意力中, 查询可以来自待生成的序列, 键和值来自另一份输入或编码结果. 本章

先推自注意力, 交叉注意力只需分别跟踪这两条输入来源.

缩放 $1/\sqrt{d_k}$ 有一个明确的尺度理由 [7]. 假设各坐标独立, 查询与键独立, 坐标均值零、方差一, 则 $q_i k_j^\top = \sum_{r=1}^{d_k} q_{ir} k_{jr}$ 均值零、方差 d_k. 除以 $\sqrt{d_k}$ 后方差为 1. 点积分数过大时 softmax 可能接近只选一个键, 其大多数导数很小. 缩放改善了初始化时的尺度, 但训练后的坐标不一定满足独立同分布假设, 它也不保证每个头永不饱和.

例题 4.1 一个查询有两个允许的键, 分数为 $(0, \log 3)$, 值为 $(2, 0)$ 和 $(0, 4)$. 求输出. 若第二键禁止, 又如何?

解答. 归一化分母为 $1 + 3 = 4$, 权重为 $(1/4, 3/4)$, 输出是 $(1/2, 3)$. 禁止第二键后它不再进入分母, 权重变为 $(1, 0)$, 输出为 $(2, 0)$. 不是先算 $(1/4, 3/4)$ 再把第二权重改为零; 那样第一权重仍为 $1/4$, 行和不为 1, 已改变运算. □

稳定实现先减去每行最大允许分数, 再指数与归一化. 同减常数不改变比值, 因分子分母都乘同一个 e^{-m_i}. 每行至少须有一个允许位置. 全禁止行既没有概率分布, 也会在 $-\infty - (-\infty)$ 中产生未定义值. 若 padding 导致此情形, 应明确不计算该查询、给出有效的约定路径, 或拒绝输入; 随意把 NaN 改成零不等于修复了概率模型.

4.2 因果掩码与训练目标

在自回归语言模型中, 输入位置 i 的 logits 预测下一个 token x_{i+1}. 因果掩码允许 $j \le i$, 禁止 $j > i$. 允许读取当前位置 x_i 不等于偷看目标, 因为目标是下一位置. 若输入与目标没有正确错开, 或注意力读到了 x_{i+1}, 训练损失可能很好看, 但实际生成时没有这条信息路径.

单层因果性可以直接证明. q_i 只由 X_i 产生, 允许的 k_j, v_j 只由 $X_j, j \le i$ 产生, 所以 C_i 只依赖 $X_0, \dots, X_i$. 多层时用归纳: 若上一层位置 j 的状态只依赖输入前缀至 j, 那么本层位置 i 读取的所有 $j \le i$ 也只依赖前缀至 i. 逐 token 前馈、逐 token LayerNorm 和同位置残差不扩大这条信息范围. 沿全部位置做均值归一化、非因果卷积或错位数据却可能破坏它. 因而验证因果性要检查整个前向图, 不能只看到一个三角矩阵就结束.

训练时可以并行算出所有位置, 因为教师提供真实前缀, 掩码限制每个查询读取的范围. 生成时后续 token 尚未存在, 须逐步产生. 并行训练与逐步生成并不矛盾, 二者使用同一组条件概率. 但训练中的真实前缀与生成中的模型前缀可能不同, 错误积累属于分布与使用方式的问题, 不是三角掩码能自动消除的现象.

习题 4.1 保持前 5 个输入 token 不变, 替换之后所有 token. 因果模型前 5 个位置的 logits 应怎样变化? 若保持了注意力掩码, 却沿位置轴做 LayerNorm, 这个结论是否仍成立?

解答. 在本节所列因果操作下, 前 5 个位置的 logits 应完全相同, 除浮点实现差异外不应依赖未来. 位置轴归一化把未来输入纳入前缀的均值和方差, 从而可以改变前缀, 结论失效. 这说明因果不变量是在检验信息路径, 不是仅检验注意力模块的形状. □

4.3 位置表征与旋转位置编码

若没有位置表示也没有掩码, 内容自注意力具有排列等变性. 设 Π 是把行重新排序的置换矩阵, 则 $Q' = \Pi Q, K' = \Pi K, V' = \Pi V$, 分数为 $S' = \Pi S \Pi^\top$. 这先重排行再在每行重排候选, 逐行 softmax 因而给 $A' = \Pi A \Pi^\top$, 最后 $C' = A'V' = \Pi A \Pi^\top \Pi V = \Pi C$. 交换输入顺序只会交换相应输出, 不能额外表达“原先谁在谁之前”. 因果掩

码本身区分可读范围, 不满足任意置换下的这一假设; 不能把未掩码的结论直接用于带固定三角掩码的网络.

一种位置方案是 $X_i = E(x_i) + P_i$, 让位置向量进入所有后续投影. 可以学习有限位置表, 也可以用确定的正弦函数生成向量. 这使模型能够利用位置, 但有限位置表没有给出未训练位置的参数. 正弦可计算任意整数位置, 也不代表训练学到的运算对任意长度都可靠.

旋转位置编码 RoPE 直接改变查询与键的匹配几何 [36]. 为避免行列约定混淆, 先定义作用于二维列向量的旋转矩阵

$$R(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}.$$

对行向量 $q_i, k_j \in \mathbb{R}^{1\times 2}$, 令 $\widetilde{q}_i = q_i R(i\omega)^\top$, $\widetilde{k}_j = k_j R(j\omega)^\top$. 两个二维向量按各自的位置旋转, 然后取点积. 由三角加法公式, $R(\alpha)^\top R(\beta) = R(\beta-\alpha)$, 所以

$$\widetilde{q}_i \widetilde{k}_j^\top = q_i R(i\omega)^\top R(j\omega) k_j^\top = q_i R((j-i)\omega) k_j^\top.$$

结果依赖相对距离 $j-i$, 而不是两个绝对位置各自进入一个任意函数. 这条等式对固定内容向量严格成立. 网络深层的 q_i, k_j 本身已经依赖上下文, 因而整模型的所有行为不只由相对距离决定.

对于偶数维 d_k, 把坐标分成 $d_k/2$ 对, 第 r 对采用自己的频率 ω_r, 独立做上述旋转, 总点积是各对点积之和. 在这一简化方案中旋转只作用于查询与键, 值仍保留原投影. 旋转正交, 因而保持每对向量的长度, 但改变不同位置之间的点积. 若某一对 $q_i = k_j = (1, 0)$, 它的点积就是 $\cos((j-i)\omega)$, 随距离振荡, 并不单调衰减. 多频率和训练表征可以形成其他匹配形状; 不能把单个旋转恒等式当成 “所有注意力随距离单调减弱” 的证明.

已知位置和频率固定时, RoPE 的反向仍是线性映射. 从 $\widetilde{q}_i = q_i R_i^\top$ 得 $G_{q_i} = G_{\widetilde{q}_i} R_i$, 键同理. 即先对旋转后的查询与键做注意力反向, 再乘逆方向的旋转返回投影输出. 有些扩展训练频率或缩放参数, 那时还需对这些参数求导; 本节没有假定它们可训练.

习题 4.2 只用一个二维旋转对, $\omega = \pi/2$, $q_i = k_j = (1, 0)$. 计算相对距离 $j-i = 0, 1, 2, 3, 4$ 的点积. 这能否证明无限上下文外推?

解答. 点积依次为 $1, 0, -1, 0, 1$. 它周期重复, 可以区分部分距离, 也可能对不同距离给相同匹配. RoPE 的公式能在任意位置计算, 但这些数值并不能证明模型对未训练长度的内容、检索和生成仍可靠. 外推能力须通过预先规定的长上下文任务与长度对照评价, 而不是由可计算性推出. □

4.4 多头、残差与逐 token 网络

一个头的输出是一个值混合, 同一组权重同时作用于所有值坐标. 多头让不同投影子空间分别选择混合. 对 $h = 1, \ldots, H$, 每头得到 $C^{(h)} \in \mathbb{R}^{T\times d_v}$, 拼接为 $C_* \in \mathbb{R}^{T\times Hd_v}$, 再用 $W_O \in \mathbb{R}^{Hd_v\times d}$ 得 $O = C_* W_O$. 典型设计令 $Hd_k = Hd_v = d$, 但这不是定义所必需. 拼接仅排列特征, 不把不同头的权重平均成一个头.

一个有明确计算路径的 pre-norm 块为

$$U = \mathrm{LN}_1(X), \qquad H_1 = X + \mathrm{MHA}(U),$$

$$R = \mathrm{LN}_2(H_1), \qquad F = \phi(RW_F + b_F), \qquad Y = H_1 + FW_G + b_G.$$

$W_F \in \mathbb{R}^{d\times r}, W_G \in \mathbb{R}^{r\times d}$, F 的形状为 $T \times r$. 两个 LayerNorm 都逐 token 作用, 有各自的参数, 导数见第 3.6 节. 原始 Transformer 的归一化位置与此式不同 [7]; 学习一个实现时应把实际前向写出来, 而不是认为所有叫 Transformer 的模块有同一个图. 本书的小模型也会故意省略归一化, 以便把故障定位到较少对象.

输出分类器令 logits 为 $YW_U + b_U$, $W_U \in \mathbb{R}^{d\times V_0}$, 概率为其 softmax. 多层残差流把初始词表向量、各层注意力信息和局部非线性变换持续组合, 形成预测所用表征. 这条函数图解释了信息能怎样传递, 尚未告诉我们训练实际学出了哪一种算法. “某头在检索前例”或“某方向编码一个属性”是需要设计干预和对照来检验的机制假说, 不能单凭结构名称认定.

一些现代模型使用 RMSNorm: 对单 token, $r = (D^{-1}\sum_i x_i^2 + \epsilon)^{-1/2}$, 输出 $y_i = \gamma_i r x_i$. 它不减均值, 因而不具有 LayerNorm 的整体平移不变性. 固定正 ϵ 时, 令 $u = g \odot \gamma$, 用与上一章相同的微分计算可得 $G_x = ru - r^3 x\,\overline{ux}$, $G_\gamma = g \odot (rx)$, 共享参数仍对 token 求和. 若阅读的模型使用它, 不能把 LayerNorm 的两个均值项直接搬过去. 选择哪种归一化改善哪种训练行为, 属于给定架构和训练设置的实验问题.

4.5 注意力的完整反向传播

从下游传来的 $G_C \in \mathbb{R}^{T\times d_v}$ 开始. $C = AV$ 给 $dC = dA\,V + A\,dV$, 由 Frobenius 内积收集系数得

$$G_A = G_C V^\top \in \mathbb{R}^{T\times T}, \qquad G_V = A^\top G_C \in \mathbb{R}^{T\times d_v}.$$

这两条路径分别控制混合权重与被混合的信息. 若只更新值而不更新查询和键, 就删掉了通过匹配权重的学习路径.

固定一行, 对允许分数 s_l 求导. 商法则给 $\partial A_j/\partial s_l = A_j(\mathbf{1}\{j=l\} - A_l)$. 因而 ds_l 的系数为

$$(G_S)_{il} = A_{il}\left((G_A)_{il} - \sum_j A_{ij}(G_A)_{ij}\right).$$

它先减去本行权重下的平均贡献, 再乘该位置的权重. 若某位置的损失系数高于该行的加权平均, 提高其分数会使损失的一阶近似增大, 所以对这些分数直接做梯度下降时会倾向降低它; 低于平均的位置相反. 这是由损失系数决定的局部变化, 不是保证训练会发现某个语义位置. 禁止位置权重为零, 分数梯度也为零; 掩码是常量且不更新. 全行梯度和为零, 与分数同时加常数不改变 softmax 相符.

在未禁止的分数上, $dS = (dQ\,K^\top + Q\,dK^\top)/\sqrt{d_k}$, 所以

$$G_Q = \frac{G_S K}{\sqrt{d_k}} \in \mathbb{R}^{T\times d_k}, \qquad G_K = \frac{G_S^\top Q}{\sqrt{d_k}} \in \mathbb{R}^{T\times d_k}.$$

然后返回投影参数与共享输入:

$$G_{W_Q} = X^\top G_Q, \qquad G_{W_K} = X^\top G_K, \qquad G_{W_V} = X^\top G_V,$$
$$G_X = G_Q W_Q^\top + G_K W_K^\top + G_V W_V^\top.$$

三条输入路径都属于同一 X, 所以相加. 有 RoPE 时先反旋转再代入投影式; 有独立查询输入的交叉注意力时, 查询梯度与键值输入梯度分别保留, 不能都加到同一个矩阵.

多头的输出投影也按同一规则求导: $G_{W_O} = C_*^\top G_O$, $G_{C_*} = G_O W_O^\top$. 按原拼接边界把 G_{C_*} 切为各头的 $T \times d_v$ 梯度, 各头独立使用上述链. 每头有自己的投影参数, 但共享输入, 所以各头的输入梯度再相加. 拼接的反向是切片, 不是求平均.

对 pre-norm 块, 设 G_Y 已知, 从第二条残差开始. 记 $G_F = G_Y W_G^\top$, $D = (G_F \odot \phi'(RW_F + b_F))$, $G_R = DW_F^\top$. 同时有 $G_{W_G} = F^\top G_Y$, $G_{W_F} = R^\top D$, 两个偏置沿位置求和. 然后

$$G_{H_1} = G_Y + \mathrm{LN}_2^{\mathrm{back}}(G_R), \qquad G_X = G_{H_1} + \mathrm{LN}_1^{\mathrm{back}}\big(\mathrm{MHA}^{\mathrm{back}}(G_{H_1})\big).$$

这里 $\mathrm{LN}^{\mathrm{back}}$ 表示用该层前向保存的均值、方差和参数执行上一章的反向, 不是再对梯度做一次前向归一化; 其 γ, β 梯度也须收集. 两个显式加项是恒等残差路径. 这就把 logits 的误差完整送到本块输入, 可继续穿过前面各块, 最后汇总到词表和位置参数.

若输出分类器与词表共享权重, 例如 $Z = YE^\top$, 则 E 还有来自分类器的贡献 $G_E^{\mathrm{out}} = G_Z^\top Y$. 查表路径贡献与该项相加才是总梯度. 共享减少参数数目, 不减少它在计算图中出现的路径数. 批量情形中这些矩阵参数梯度均对所有文档和位置求和, 但激活梯度保留其索引.

4.6 两位置的一次完整更新

公式链容易在转置处失去直觉. 用 $B = 1, T = 2, D = 1$ 把所有量压成可手算的矩阵, 输入 $X = (1, 2)^\top$, 三个投影权重均为标量 1. 这里只含因果注意力, 没有前馈层、残差或输出分类器. 标量损失是末位置输出与目标 1.5 的平方误差 $\ell = (C_2 - 1.5)^2/2$, 第一位置不直接计入损失.

前向有 $Q = K = V = (1, 2)^\top$, 未掩码分数为 $\left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$. 掩码删除第一行的第二个候选. 记 $a = 1/(1 + e^2) \approx 0.119203$, 则

$$A = \begin{pmatrix} 1 & 0 \\ a & 1 - a \end{pmatrix}, \qquad C = \begin{pmatrix} 1 \\ 2 - a \end{pmatrix}, \qquad r = C_2 - 1.5 \approx 0.380797.$$

由 $d\ell = r\, dC_2$ 得 $G_C = (0, r)^\top$. 再回到乘法与 softmax:

$$G_A = \begin{pmatrix} 0 & 0 \\ r & 2r \end{pmatrix}, \quad G_V = \begin{pmatrix} ar \\ (1 - a)r \end{pmatrix}, \quad G_S = \begin{pmatrix} 0 & 0 \\ -a(1 - a)r & a(1 - a)r \end{pmatrix}.$$

第一行虽影响第一位置输出, 该输出没有下游损失, 所以它的梯度全为零. 第二行的两个分数梯度和为零, 符合 softmax 对整体平移不敏感. 令 $u = a(1 - a)r \approx 0.0399813$, 有 $G_Q = G_S K = (0, u)^\top$, $G_K = G_S^\top Q = (-2u, 2u)^\top$. 所以

$$G_{W_Q} = X^\top G_Q = 2u, \qquad G_{W_K} = X^\top G_K = 2u, \qquad G_{W_V} = X^\top G_V = (2 - a)r.$$

三个数分别约为 $0.0799625, 0.0799625, 0.716202$. 即使输入不更新, 它对上游模型的梯度仍可算出: $G_X = G_Q + G_K + G_V \approx (-0.0345704, 0.455349)^\top$. 若 X 本身由查表产生, 这个量才能继续传回词表与位置参数.

取学习率 0.1, 同时用当前梯度更新三个投影, 而固定输入 X. 所有更新必须使用同一次前向得到的旧权重与梯度, 不能更新一个权重后再用新值计算另一权重的梯度. 更新与重新前向的实际数值如下; `attention_walkthrough`

还输出每个中间矩阵, 并对全部三个参数作中心差分. 更新后的 (W_Q, W_K, W_V) 为 $(0.992004, 0.992004, 0.928380)$. 重新前向得到 $C_2 = 1.742951$, 损失从 0.07250321 降至 0.02951257. 中心差分的最大误差为 7.8×10^{-12}.

4.7 公式与批量实现

第 3.3 节已解释查表、广播与展平. 在原单头、无归一化的 `TinyDecoder` 中, `q,k,v` 都是 (B, T, D), 每个批次独立做注意力. 分数的定义是 $s_{bij} = \sum_d q_{bid} k_{bjd} / \sqrt{D}$: 一边保留查询位置 i, 另一边保留键位置 j, 只收缩特征轴. 因而要把 `k` 的最后两轴交换成 (B, D, T).

`k.transpose(0,2,1)` 表示新数组的第 (b, d, t) 项是原数组的第 (b, t, d) 项; 批次轴仍在最前. 三维数组的 `k.T` 会反转全部轴, 得 (D, T, B), 它不是所需的逐批转置. NumPy 的 `@` 对最后两个轴执行矩阵乘法, 对前面的批次轴匹配或广播. 如下代码逐行对应数学对象:

```
x = E[ids] + P[:T]              # (B,T,D)
q, k, v = (x @ W for W in (WQ, WK, WV))
s = q @ k.transpose(0,2,1) / np.sqrt(D)
s = np.where(mask, s, -np.inf) # mask: (T,T)
a = softmax(s)                  # (B,T,T), last axis
c = a @ v                       # (B,T,D)
h = x + c @ WO                  # attention residual
f = np.tanh(h @ WF + bF)         # (B,T,2D)
z = h + f @ WG + bG             # FFN residual
logits = z @ WU + bU            # (B,T,V0)
```

这里 `mask` 的形状 (T, T) 沿 B 广播, 每行至少保留自身. `softmax` 沿最后一轴归一化, 所以固定查询位置与批次后, 在允许的键位置之间分配权重. 表中的代码变量 `WQ` 等在实际程序中存于 `p['Q']` 等参数字典, `dl` 是平均交叉熵对 `logits` 的梯度.

反向的对象、形状与关键代码可以从输出逐级核对:

对象	形状	loss_grad 中的运算
G_U	$D \times V_0$	`mm(z,dl)`
G_z	$B \times T \times D$	`dl@p['U'].T`
G_F	$D \times 2D$	`mm(h,df)`
G_O	$D \times D$	`mm(c,dh)`
G_A	$B \times T \times T$	`dc@v.transpose(0,2,1)`
G_V	$B \times T \times D$	`a.transpose(0,2,1)@dc`
G_Q	$B \times T \times D$	`ds@k/np.sqrt(self.d)`
G_{W_Q}	$D \times D$	`mm(x,dq)`
G_P	$T \times D$	`dx.sum(0)`

矩阵参数的 G_U 指输出投影权重, 与上节单头 G_V 的值激活梯度不要混淆. `df` 已包含 tanh 的导数, `dh` 已把 $z = h + fW_G + b_G$ 的直接路径和前馈路径相加. 对 $h = x + CW_O$ 又有一条直接路径, 所以

$$G_x = G_h + G_Q W_Q^\top + G_K W_K^\top + G_V W_V^\top .$$

这一行的 G_h 不能漏掉. 矩阵梯度按批次与位置求和, 激活梯度则保留它们; 只记形状而不记共享关系仍会写错.

习题 4.3 运行 `residual_debug` 实验. 它只在反向中漏掉上述 G_h 项, 前向完全不变. 先预测哪些参数组的有限差分会失败, 再根据输出定位, 最后给出修复. 为什么这比只看损失是否下降更有诊断力?

解答. 故障在回到输入 x 时发生, 因此继续依赖 G_x 的词表 E 与位置表 P 会失败; 此前已算出的注意力投影、前馈层与输出层参数梯度仍正确. 修复是把 `dh` 加回 `dx`. 程序用 $B = 2, T = 3, D = 4$, 词表 6, 种子 19, 对 206 个参数坐标全部检查. 本次结果如下: 正确实现的最大误差为 3.8×10^{-11}; 漏项后 E, P 的误差分别为 0.120850 与 0.177792, 其余参数组仍小于 10^{-7}. 漏梯度仍可能产生下降方向, 所以损失变小不能确认反向实现. 有限差分按参数组定位可将问题缩到共享输入的上游, 再通过计算图确定漏掉的恒等路径. 它仍是此输入与初始化上的数值诊断, 不是对所有情形的正确性证明. □

4.8 计算代价与缓存

注意力把每个查询与每个允许键匹配. 一般稠密情形, $QK^\top$ 需要 $O(T^2 d_k)$ 运算, AV 需要 $O(T^2 d_v)$. H 个头的匹配和聚合共为 $O(HT^2(d_k + d_v))$, 若 $Hd_k = Hd_v = d$, 就是 $O(T^2 d)$. 投影和输出映射还有约 $O(Td^2)$ 的开销, 前馈层另有 $O(Tdr)$. 因果三角只把位置对数量从 T^2 改为约 $T(T+1)/2$, 没有把数量级变成线性.

若显式保存每头的分数或概率, 存储需 $O(HT^2)$ 个数; 查询、键、值和输出本身约需 $O(Td)$. 训练还要保存各层用于反向的激活. 不能把只算一层推理的内存当成完整训练内存. 运算次数、峰值存储和内存搬运是不同指标, 一个实现减少存储, 不意味着减少了全部乘加次数.

逐步生成时, 旧位置的键和值可缓存. 新查询只需与此前的键匹配, 然后读相应值, 单步这部分代价约为

$O(TH(d_k+d_v))$, 加上投影和前馈代价. 每层 KV 缓存需 $O(TH(d_k+d_v))$ 个数. 这避免对完整前缀反复计算旧键值, 但随着已生成长度增长, 新 token 仍要读取越来越长的缓存. 还必须保持位置编号、掩码和训练时定义一致. 本章的 CPU 实验没有实现缓存性能基准, 上述是由形状和使用次数得出的计算分析.

4.9 分块在线 softmax 的公式与证明

显式存储 $T\times T$ 分数不是计算精确注意力的数学要求. 困难在于一个查询的 softmax 分母依赖所有允许键: 不能给每个块单独归一化后直接拼接, 否则块间概率质量错误. 我们需要在处理键块时保存可合并的统计量, 让最终输出和一次处理所有键相同.

先固定一个查询. 对已处理的非空允许键集合 $\mathcal{J}$, 记

$$m=\max_{j\in\mathcal{J}} s_j,\qquad l=\sum_{j\in\mathcal{J}} e^{s_j-m},\qquad u=\sum_{j\in\mathcal{J}} e^{s_j-m}v_j.$$

m,l 是标量, u 是 d_v 维行向量. 当前输出为 u/l. 保存未归一化的 u 是为了下一块到来时还能改变共同分母. 对与已处理集合不相交的新非空允许块 $\mathcal{B}$ ($\mathcal{J}\cap\mathcal{B}=\varnothing$), 取新最大值 $m'=\max(m,\max_{j\in\mathcal{B}} s_j)$, 更新

$$l'=e^{m-m'}l+\sum_{j\in\mathcal{B}} e^{s_j-m'},$$

$$u'=e^{m-m'}u+\sum_{j\in\mathcal{B}} e^{s_j-m'}v_j.$$

每次只使用不大于零的指数参数, 避免原始大分数直接指数上溢. 旧最大值变化时, 旧累积量须一起缩放, 不能只缩放新块.

命题 4.2 *在上述不相交条件下, 更新后 m',l',u' 恰为并集 $\mathcal{J}\cup\mathcal{B}$ 的相应统计量. 在有限实数精确算术中, 处理完所有允许键后 u/l 等于稠密 softmax 注意力输出.*

证明. 最大值的式子显然是并集最大值. 对旧键 $j\in\mathcal{J}$, 有 $e^{s_j-m'}=e^{m-m'}e^{s_j-m}$. 因而旧键在新共同尺度下的指数和是 $e^{m-m'}l$, 加上新块就得到并集指数和 l'. 对每项再乘向量 v_j, 相同恒等式给 u'. 以首个非空允许块为初始统计量, 逐块归纳即可. 最后分子与分母同乘 e^{-m} 不改变比值, 故 $u/l=\sum_j e^{s_j}v_j/\sum_j e^{s_j}$. □

也可以使用 $m=-\infty,l=0,u=0$ 的空状态, 但实现必须特别处理还没有允许键的行, 避免计算 $-\infty-(-\infty)$. 全禁止块应跳过, 首个非空块再建立有限最大值. 每个查询最终至少有一个允许键, 才能除以 $l>0$. 这在因果和 padding 组合掩码中尤其重要.

例题 4.2 分数为 $(0,\log 3)$, 值为 $(2,0),(0,4)$, 每个键单独成为一个块. 用在线公式复算输出.

解答. 处理第一个键后 $m=0,l=1,u=(2,0)$. 第二个键使 $m'=\log 3$, 旧尺度因子为 $1/3$, 新键指数为 1. 所以 $l'=1/3+1=4/3$, $u'=(2/3,4)$, 输出 $u'/l'=(1/2,3)$, 与稠密结果相同. 如果两个块分别归一化, 它们都给权重 1, 便会把总输出错算为 $(2,4)$. □

实际分块还把查询分为至多 b_q 行的小块, 每次与至多 b_k 个键相乘. 一个分数块形状仅为 $b_q\times b_k$, 各行分别维

护 m, l, u. 除输入输出外, 当前块及其累积量约需 $O(b_q b_k + b_q d_v)$ 工作空间. 固定块大小时, 不必存全部 T^2 个分数; 但所有稠密查询–键对仍需计算, 算术数量级仍为二次. 若实现另外存着一个完整 $T \times T$ 布尔掩码, 这份掩码本身仍占二次空间; 想让总额外空间也避免二次增长, 还须按索引在块内产生因果掩码, 或用合适的稀疏表示.

反向也可以重算概率块. 前向保存每行 $z_i = m_i + \log l_i$, 则允许位置 $A_{ij} = e^{s_{ij} - z_i}$. 给定 G_C, 记

$$D_i = G_{C_i} C_i^\top.$$

它恰等于 softmax 反向所需的行平均贡献, 因为

$$\sum_j A_{ij} (G_A)_{ij} = \sum_j A_{ij} G_{C_i} V_j^\top = G_{C_i} \left(\sum_j A_{ij} V_j \right)^\top = G_{C_i} C_i^\top.$$

因此在一个重算的块上, 计算 $(G_A)_{ij} = G_{C_i} V_j^\top$, 再算 $(G_S)_{ij} = A_{ij}((G_A)_{ij} - D_i)$; 按已有公式累积 G_Q, G_K, G_V. D_i 无须等待整行 G_A 显式存好才求出. 这是避免保存完整概率矩阵的一条数学路径, 代价是反向重算一些前向量.

FlashAttention 将分块、在线归一化与重算结合, 专门管理 GPU 高带宽内存和片上存储之间的数据搬运 [11]. 本节证明了一个核心代数不变量, 没有证明某个 CPU 循环达到论文的 GPU IO 复杂度或速度. 减少分数工作空间、实现融合内核和取得实际加速是不同层次的工作. 原论文的实现、硬件和预算需要另行评价.

4.10 完整标准 Transformer 的 CPU 配对训练

手算展示一条注意力路径，完整模型还要把位置、归一化、前馈、残差和读出连接起来。`standard_transformer_study.py` 使用 PyTorch 的标准层实现两层前置归一化的因果解码器；它不是包装外部 API，也没有下载预训练权重。结构如下：词表 10、宽度 32、两头、前馈宽度 64、上下文长度 7，两个残差块，GELU 激活，最后归一化及线性读出，总计 18,026 个可训练参数。位置是可学习嵌入，没有使用本章另一节推导的 RoPE；实现和方法名不能混淆。

每一块可写成

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad X' = U + \mathrm{FFN}(\mathrm{LN}(U)),$$

其中 $\mathrm{FFN}(u) = \mathrm{GELU}(uW_1 + b_1)W_2 + b_2$。训练对整个词表用交叉熵，不只监督叫声。八条原创语法句从说话者、颜色、旁观者各两种组合构成；固定六条训练、两条留出，整个设计在训练前写成独立 JSON 并计算哈希。

我们比较均衡六条训练句与将同一条训练句额外重复 30 次。若单条损失是 ℓ_i，第二组目标为

$$L_{\text{repeat}} = \frac{31\ell_0 + \sum_{i \neq 0} \ell_i}{36}, \qquad L_{\text{balanced}} = \frac{1}{6} \sum_{i=0}^{5} \ell_i.$$

唯一句子数没有增加，分布权重改变了。两组同一种子从相同参数出发、做 600 次 AdamW 更新，但每步分别处理 6 与 36 行，因此没有匹配 FLOPs。这个实验检验频率重加权及其伴随处理成本，不能当成等算力数据质量消融，更不是 FineWeb 的大规模复现。

种子固定为 144、145、146，学习率 0.003、权重衰减 0.01、梯度裁剪 1，无 dropout，CPU 单线程 float32。六次运行全部报告，未用留出成绩挑最好种子。下面由实际结果 JSON 自动生成，代码与环境哈希另附。

种子	条件	加权训练 NLL	均衡训练 NLL	留出 NLL
144	均衡	0.2565	0.2565	1.3159
144	单条重复	0.0899	0.4310	1.3747
145	均衡	0.2565	0.2565	1.2958
145	单条重复	0.0900	0.4311	1.3475
146	均衡	0.2565	0.2565	1.2773
146	单条重复	0.0900	0.4311	1.3985

重复条件的自身加权训练损失更低，三个配对留出损失却更高。若只看优化器监控的训练目标，会错把偏向常见句的改进当作总体泛化提高；使用同一均衡训练分布评价又得到不同结果。这里两条留出样本太少，不能从三次初始化推出广泛的数据结论。

六个模型对原始、改变旁观者及改变说话者三种叫声测试均符合该有限语法；但这些干预组合与训练句子重叠，不能识别唯一内部机制，也不能证明任意组合迁移。编辑未来 token 后先前位置的最大 logits 差为零；额外测试还检查了未来嵌入的梯度为零。这提供了因果掩码实现证据。想继续研究，应事先扩充组合、独立留出角色和词表，匹配 token 处理预算，再比较数据重加权；不能看到当前结果后悄悄修改这一次已冻结实验。

4.11 实验、机制假说与练习

新增 `attention` 实验在 CPU 的 NumPy float64 中, 用 $T = 37, d_k = 5, d_v = 7$ 的随机输入和因果掩码, 比较稠密计算与查询–键块大小 $1\times1, 4\times6, 8\times7, 50\times50$ 的在线计算. 固定种子 144 时, 最大绝对输出差不超过 4.45×10^{-16}. 把查询与键各放大 100 倍, 使分数进入强饱和区, 两种算法也保持有限且一致. 这检验数值等价, 不把浮点计算声称为实数精确相等.

同一实验用共享输入产生查询、键和值, 替换第 15 位置及之后的输入, 因果掩码下前 15 个位置的输出最大变化为 0; 取消掩码后的最大变化约为 37.1055. 注意力反向还对一个任意线性下游损失的全部 68 个 Q, K, V 坐标进行中心差分, 最大混合误差低于 8.87×10^{-11}. 其他检验包括禁止位置权重为零、每行和为 1、首块全禁止时仍正确处理后续允许块、全禁止查询被明确拒绝. 这些输入特意覆盖常见实现边界, 仍不是对所有程序状态的形式证明.

默认 8×7 块的单个分数数组最多含 56 个数, 而稠密分数数组有 $37^2 = 1369$ 个数. 这两个值来自实际配置和数组形状, 不是进程峰值内存测量. 程序为教学方便仍传入完整布尔掩码, 输出和中间累积量也占内存. 环境记录没有 GPU 速度或端到端训练吞吐量; 不应把这里的工作数组比较写成 FlashAttention 性能复现.

这些结构如何连接基础模型能力? 注意力提供按内容与位置读取上下文的路径, 前馈层变换当前表征, 残差允许多个层次的信息叠加. 预训练让这些路径共同服务后继预测. 在上下文学习任务中, 提示中的例子可以作为当次输入被读取, 无须在推理时更新参数. 但“有路径”不等于“训练一定学会利用它”. 要研究某个模型是否从示例学到对应规则, 可固定模型, 对照有无示例、示例标签随机化、示例顺序变化和未见组合, 再检查目标答案的条件概率与实际正确率. 它们区分对样例的读取、模板捷径与任务规律, 不能由一个注意力热力图直接确定.

习题 4.4 X 形状为 12×32, 每头 W_Q, W_K 为 32×8, W_V 为 32×16, 一共有 4 个头. 写出每头和拼接后的形状, 以及输出回到 32 维所需 W_O 的形状.

解答. 每头 Q, K 为 12×8, V, C 为 12×16, S, A 为 12×12. 拼接后 C_* 为 12×64, 所以 W_O 为 64×32, 输出为 12×32. 给定输出梯度, $G_{C_*}=G_O W_O^\top$ 也是 12×64, 按四个 16 列块切分后分别进行各头反传. 各头的输入梯度都是 12×32, 最后相加. □

习题 4.5 一个查询有三个键, 分数为 $(\log 2, 0, \log 3)$, 标量值为 $(1, 4, 2)$. 第一块含前两个键, 第二块含最后一个键. 求在线状态与最终输出.

解答. 第一块最大值 $m=\log 2$, 指数为 $(1, 1/2)$, 故 $l=3/2, u=1+4/2=3$. 新最大值为 $m'=\log 3$, 旧缩放因子为 $2/3$. 所以 $l'=(2/3)(3/2)+1=2$, $u'=(2/3)3+2=4$, 输出为 2. 稠密式也给 $(2\cdot 1+1\cdot 4+3\cdot 2)/(2+1+3)=2$. □

习题 4.6 若某个查询仅允许一个键, 它的查询和键分数梯度是否为零? 值梯度呢? 最大注意力权重能否直接表示最大因果作用?

解答. 该行权重恒为 1, softmax 对唯一分数的导数为 $1(1-1)=0$, 因而该行对查询和键的分数路径不产生梯度. 但输出等于那个值, 值仍接收 G_C. 键也可能被其他查询使用, 所以整个键向量的总梯度不一定为零. 最大权重还要乘值并经过输出投影和其他残差路径, 因而不能单独确定因果作用. 可设计保持其他因素可比的输入替换或中间激活替换, 检查指定输出变化, 并核对替换是否带来分布外伪影. □

习题 4.7 若前向已存 C_i 和 z_i, 为什么反向能够逐块求 G_S 而不先保存整行 G_A? 写出必须仍累积的参数或激活梯度.

解答. 因为整行的加权平均 $\sum_j A_{ij}(G_A)_{ij}$ 已等于可直接计算的 $D_i = G_{C_i}C_i^\top$. 每块重算 $A_{ij}=e^{s_{ij}-z_i}$, 用 $A_{ij}((G_A)_{ij}-D_i)$ 得该块 G_S. 每块贡献必须累积到查询梯度 G_Q, 键梯度 G_K 和值梯度 G_V, 再汇总到共享投影矩阵. 查询跨键块重复, 键和值跨查询块重复, 任一方向覆盖而不相加都会漏路径. 减少存储没有消除共享梯度的求和要求. □

5 语言模型的数据与可信评估

5.1 学到的是哪一种分布

优化解决了如何降低一个确定的目标函数. 但语言模型在训练语料上降低损失, 并不等于能够回答新的问题. 一份被记住的试卷和一份从未见过的试卷可能有完全不同的表现. 本章把“训练时优化什么”与“用什么证据判断学到了什么”连起来. 这些区别在预训练、指令微调和后训练中都存在, 只是样本单位、损失和评价任务有所不同.

定义 5.1 设 Z 是一个按约定抽取的评价单位, 其分布为 P. 对模型 f 和损失 $\ell(f, Z)$, 总体风险为 $R_P(f) = \mathbb{E}_{Z\sim P}\ell(f, Z)$. 若 $Z_1, \ldots, Z_n$ 为实际观测, 经验风险为

$$\widehat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n}\ell(f, Z_i).$$

这里的 f 是包含参数和使用规则的整个预测系统. 提示词、检索库、采样温度和工具使用规则若影响输出, 也属于评价时须固定的设计.

总体风险是生成过程的平均, 往往不能直接算出; 经验风险是有限数据的平均, 可以计算. 当算法根据数据选择模型 $\widehat{f}$, 这个模型也是一个随机对象. 因而对事先固定的 f 成立的统计结论, 不能直接把 f 换成在同一数据上挑出的 $\widehat{f}$. 后面会精确说明何时可以这样替换.

语言模型的一个文档是序列 $x_1, \ldots, x_T$. 自回归模型按链式分解给出

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \qquad -\log p_\theta(x_1, \ldots, x_T) = \sum_{t=1}^{T} -\log p_\theta(x_t \mid x_{<t}).$$

第一个 token 的上下文可以包含固定的起始符. 等式来自条件概率的乘法规则, 不是神经网络专有的假设. 模型决定的是每个条件概率怎样由上下文计算. 预训练通常优化大量实际文档中的这些负对数概率之和或平均.

平均的分母须明确. 对每篇文档先算平均 token 损失, 再对文档平均, 是让每篇文档具有相同权重; 把所有文档的 token 损失相加再除以 token 总数, 是让长文档获得更多权重. 二者对应不同的目标总体. 若研究 “随机用户提交一篇文档的表现” , 文档平均可能更贴近问题; 若研究语料压缩或随机 token 的预测, token 平均通常更自然. 不能在比较模型时暗中更换分母.

例题 5.1 两篇文档分别含 2 个与 8 个评价 token, 每个 token 的平均负对数似然分别为 1 与 3. 求文档平均和 token 平均.

解答. 文档平均为 $(1+3)/2 = 2$; token 平均为 $(2\cdot 1 + 8\cdot 3)/10 = 2.6$. 第一种抽样先等概率选文档, 第二种抽样在 10 个 token 中等概率选一个. 两个计算都正确, 但回答的问题不同. 若长文档恰好更难, 只改变权重就能改变报告值, 无须模型行为改变. □

5.2 数据划分、污染与开发过程

训练集用于更新参数, 开发集或验证集用于选择设计, 测试集用于设计确定后的最终评价. 预训练数据、后训练数据和评价数据还须区分: 某个基准没有进入指令微调集, 不能据此断定它没有进入预训练语料. 题面、答案、解析、改写和同源版本都可能传递评价信息. 这种污染可以造成较高分数, 但不能仅凭分数高就断定存在污染.

先确定部署问题, 才能决定怎样划分. 如果目标是预测已有作者的新文章, 按文档留出可能合理; 如果目标是新作者, 须按作者分组; 如果目标是将来发生的知识和事件, 须考虑时间切分. 按时间切分仍不能自动排除转载或提前公开的答案. 划分只是评估设计的一部分, 必须配合来源与去重检查.

语言模型常把文档切为长度固定的窗口. 若先切窗口再随机划分, 同一文档的相邻窗口可能跨过训练与测试边界. 训练窗口的后半段还可能与测试窗口的前半段完全相同. 此时测试不再模拟新文档. 更合理的顺序是先在文档、

用户或其他目标单位上划分, 再在各集合内部切窗口. 精确去重可比较内容哈希; 近重复需要另定比较规则, 并记录阈值、误删风险与无法识别的改写.

开发集也会被逐渐学会. 假设每次试验都看同一批 32 道题, 再改变提示词、样例排列、停止规则或模型版本. 最终设计不只是由训练参数决定, 还受这 32 道题影响. 即使没有对开发集反向传播, 这种人为选择也构成信息使用. 有效做法是记录尝试次数与选择准则, 在选择结束后使用独立留出集, 或在外层留出数据而只在内层选择设计. 数据少时可以交叉验证设计的稳定性, 但不能把反复查看外层结果仍称为独立测试.

习题 5.1 从 20 篇文档各切出 100 个重叠窗口, 随机按 8:2 分到训练和测试. 另有一套考试题, 其标准答案未入训练集, 但详细解析入了训练集. 分别说明两个设计的风险, 并提出修正.

解答. 重叠窗口会分享文本和文档背景, 所以 2000 个窗口不等于 2000 个独立的新文档. 应先按文档或部署所需的作者、时间单位划分, 再切窗口. 第二种情形虽然未见标准答案文件, 解析仍可能包含答案及解法信息, 因而不能作为未见题泛化的证据. 应把同源题面、答案和解析放在同一侧, 检查近重复与改写, 再选择来源和时间清楚的留出题. 如果无法核实预训练语料, 就应报告污染状态未知, 不把它写成已排除. □

5.3 预训练数据的受控比较

评价不仅用于比较模型, 也用于研究数据处理是否改善学习. 网页中的菜单、重复模板和大量镜像文本可能占据训练预算; 过滤与去重改变了同样预算里模型实际看到的内容. 但过滤也可能删去罕见语言、专业文本或与部署有关的格式, 因而“保留得更少”并不自动等于“数据质量更高”.

FineWeb 的数据消融提供了具体的阅读对象 [43]. 它在同一比较中固定模型架构、参数规模、训练 token 数和评价设置, 主要改变文本抽取、过滤或去重步骤; 使用两组随机数据子集和初始化来降低单次实验偶然性. 相关消融采用约 1.71B 参数的模型, 过滤研究常使用约 28B 训练 token, 一些去重与累计比较使用 350B token. 读者应先核对是哪一种预算, 再阅读分数, 不能把不同预算的曲线直接当作纯数据效应.

其中一个有教学价值的结果是: 逐快照去重与跨全部快照的全局去重并不等价, 后者在所研究设置中没有产生原先预期的明显改善. 去重也改变了内容混合比例, 因而“删除重复”不能单独解释所有分数变化. 这里的实测支持是在给定模型、预算和任务组中, 某些处理方案比对照更好; 对哪些内容促成了何种内部机制, 仍需额外实验. 这既不证明所有数据集都应照搬同一过滤规则, 也不证明处理步骤改善了全部部署任务.

读一项数据论文时, 可以提出可回答的问题: 比较中训练 token 总数是否相同? 删除后是否重新采样而提高某类内容权重? 是否同时更换了分词或模型? 评价任务是否也参与筛选数据规则? 这些问题把“更好的数据”变成可检验的具体干预. 若要在本书 CPU 规模上练习, 可固定一个小模型与更新预算, 比较含重复模板和去重后的合成序列, 预先声明用哪些未见组合计分. 这只是方法迁移的设计, 本章没有把它冒充已经复现的 FineWeb 实验.

5.4 有限样本能够保证什么

先看一个可完全证明的有限模型结论. 它适用于落在 $[0,1]$ 的损失, 如答题错误指示量, 或事先定义的文档级有界评分. 原始负对数似然没有这样的统一上界: 当正确 token 概率趋于零, $-\log p$ 可以任意大. 不能把下面的结论原样用于未截断的语言模型交叉熵. 若预先把损失截断为 $\min(\ell, L)/L$, 界约束的是截断后的风险; 极低概率事件的尾部损失仍需另行研究.

引理 5.2 (有界随机变量的指数估计) *若 $0 \le Z \le 1$, 对任意实数 t, 有 $\mathbb{E}e^{t(Z-\mathbb{E}Z)} \le e^{t^2/8}$.*

证明. 记 $F(t) = \log \mathbb{E}e^{tZ}$. 因 Z 有界, 在任何有限 t 区间上, 指数及其前两阶导数都由一个有限常数控制, 可在期望内求导. 计算得

$$F'(t) = \frac{\mathbb{E}[Ze^{tZ}]}{\mathbb{E}[e^{tZ}]}, \qquad F''(t) = \frac{\mathbb{E}[Z^2e^{tZ}]}{\mathbb{E}[e^{tZ}]} - \left(\frac{\mathbb{E}[Ze^{tZ}]}{\mathbb{E}[e^{tZ}]}\right)^2.$$

把原分布按正权重 $e^{tZ}/\mathbb{E}e^{tZ}$ 重新加权, 这仍是一个概率分布, 且仍满足 $0 \le Z \le 1$. 上式是该分布中的方差. 对任意取值于 $[0,1]$ 的变量, 若均值为 μ, 由 $Z^2 \le Z$ 得方差至多 $\mu - \mu^2 \le 1/4$. 因而 $F''(t) \le 1/4$.

令 $G(t) = F(t) - t\mathbb{E}Z$, 则 $G(0) = G'(0) = 0$. 二次积分余项写为 $G(t) = t^2 \int_0^1 (1-s)F''(st)\,ds$, 对正负 t 都成立. 所以 $G(t) \le t^2/8$. 对两侧取指数即得结论. □

定理 5.3 (固定模型的 Hoeffding 界) *若 Z_i 独立同分布, 固定模型的损失 $L_i = \ell(f, Z_i)$ 落在 $[0,1]$, 则对任意 $\epsilon > 0$,*

$$\Pr\left(\left|\widehat{R}_n(f) - R_P(f)\right| \ge \epsilon\right) \le 2e^{-2n\epsilon^2}.$$

证明. 设 $R = \mathbb{E}L_i$. 对任意 $t > 0$, 事件 $\sum_i (L_i - R) \ge n\epsilon$ 蕴含 $e^{t\sum_i (L_i - R)} \ge e^{tn\epsilon}$. 非负变量的期望至少为这个阈值乘事件概率, 从而

$$\begin{aligned}\Pr\left(\sum_i (L_i - R) \ge n\epsilon\right) &\le e^{-tn\epsilon}\mathbb{E}e^{t\sum_i (L_i - R)} \\ &= e^{-tn\epsilon}\prod_i \mathbb{E}e^{t(L_i - R)} \le e^{-tn\epsilon + nt^2/8}.\end{aligned}$$

独立性用在把期望拆成乘积这一步. 右边指数是关于 t 的二次式, 取 $t = 4\epsilon$ 得 $e^{-2n\epsilon^2}$. 对负偏差应用引理中的 $-t$, 得同样上界. 正负两事件的并集概率至多为概率之和, 所以乘以 2. □

命题 5.4 (有限候选的同时保证) *设 $f_1, \dots, f_M$ 是不依赖本次评价数据的固定候选, 均使用 $[0,1]$ 损失. 对独立同分布的 n 个评价单位及 $0 < \delta < 1$, 至少以概率 $1 - \delta$, 所有候选同时满足*

$$|\widehat{R}_n(f_j) - R_P(f_j)| \le \epsilon_n, \qquad \epsilon_n = \sqrt{\frac{\log(2M/\delta)}{2n}}.$$

在此事件上, 令 $\widehat{f}$ 最小化候选的经验风险, f_* 最小化候选的总体风险, 则 $R_P(\widehat{f}) \le R_P(f_*) + 2\epsilon_n$.

证明. 每个候选失败的概率至多为 $2e^{-2n\epsilon_n^2}$. 任意一个失败的概率至多为它们之和 $2Me^{-2n\epsilon_n^2} = \delta$; 无须假定不同模型的损失相互独立. 在所有界成立时,

$$R_P(\widehat{f}) \le \widehat{R}_n(\widehat{f}) + \epsilon_n \le \widehat{R}_n(f_*) + \epsilon_n \le R_P(f_*) + 2\epsilon_n.$$

中间一步来自经验最小化, 两端来自同时偏差界. 因为已经同时控制所有候选, 数据选出的下标也在控制范围内. □

若 $M = 1000, n = 10000, \delta = 0.05$, 得 $\epsilon_n \approx 0.0230$. 这是充分保证, 通常很保守, 不是实际误差必然达到 2.3 个百分点. 若一次评价单位是一篇文档, n 就是文档数, 不能擅自改成相关 token 的总数. 候选网络由独立训练集产生时, 可以先条件于训练集, 再对独立开发数据应用此界. 若新的候选根据此前的开发反馈逐步产生, 便不满足“本次评价前已固定”的条件, 不能简单以实际尝试次数代入宣称获得同样保证.

最终模型若只由训练和开发数据选出, 则条件于这两部分数据后, 它对独立测试集是固定模型, 可应用固定模型界. 这解释了保持测试集独立的作用. 它没有证明测试分布与部署分布相同, 也没有处理评分器本身的偏差.

5.5 正则化与选择偏差的实测

语言模型在大量参数和有限样本之间需要某种选择约束. 例如目标 $\widehat{R}_n(\theta) + \lambda\|\theta\|^2/2$ 对参数大小施加惩罚, 其梯度比原损失增加 $\lambda\theta$. 早停限制更新次数, 数据增强规定哪些变化应保留答案, 参数高效微调限制可更新的方向. 它们改变训练算法或可选择的函数, 但不自动证明模型学到了因果特征. 惩罚系数、停止步数和提示方案同样需要开发数据选择. 用测试分数选择它们仍会造成选择偏差.

新增 `v03_networks.py` 的 `selection` 实验只隔离评价中的选择效应. 它生成已固定候选的独立有界损失, 每个候选的真实错误风险都是 0.5. 每次只用 32 个验证单位挑选经验错误率最小的候选, 再独立抽取 2000 个测试损失来评价选中者. 400 次重复中, 候选数为 $1, 8, 64, 512$ 时, 选中者验证错误率的平均依次为 $0.49969, 0.37117, 0.29766, 0.23750$; 独立测试平均依次为 $0.50034, 0.49986, 0.49925, 0.50013$.

验证分数随着搜索规模提升, 真实能力完全没有提升. 原因不是优化器有问题: 从多个带噪声的平均里挑最小值, 同时也挑中了偏低的采样噪声. 这是合成损失表, 没有训练神经网络; 其用途是显示机制, 不能把 26.3 个百分点的乐观差距当成现实基准污染量. 完整指标、随机种子 144、环境和源码哈希保存在 `results/v03-networks/`.

5.6 概率、校准与任务指标

语言模型的输出是概率分布, 但通常还要转成回答、程序或动作. Token 负对数似然度量模型给真实后继分配多少概率; 答题正确率度量按约定生成与评分后的答案. 同一个事实可以有多种正确表达, 所以真实文本概率不等于答案正确率. 相反, 在有限题目上猜中答案也不能证明概率分布拟合良好. 工具系统还要区分提案、执行和最终完成, 检索系统要区分检索命中和基于证据的回答.

若使用自然对数, 困惑度定义为平均 token 负对数似然的指数. 在所有位置都给正确 token 相同概率 p 的特殊情形, 困惑度为 $1/p$. 一般情形是这些倒数的几何平均. 比较困惑度须使用相同的分词、评价文本和计分位置; 词表不同会改变一个 token 所含的信息量. 不能只比较数值就判断不同分词模型谁更好.

定义 5.5 对二分类概率预测 $q(X) \in [0,1]$, 校准指 $\mathbb{E}[Y \mid q(X)] = q(X)$. 意思是同样预测值的一组样本中, 正类的实际频率与该预测值相符. 实际样本有限, 通常把概率分箱后比较各箱的平均预测和正类频率, 而不是声称逐个实数概率都已验证.

准确率只看阈值两侧, 无法识别过度自信. 一个正确率为 80% 的系统若对每个答案都报告 99% 的信心, 就可能严重失准. 语言模型还须明确 "信心" 指哪一个事件: 单 token 的概率、完整字符串的概率、某答案集合的总概率, 或另一个评分器给出的成功概率, 这些不是同一个量.

平方概率误差称为 Brier 损失. 若真实条件概率为 $p = \Pr(Y = 1 \mid X = x)$, 则

$$\mathbb{E}[(q - Y)^2 \mid X = x] = p(q-1)^2 + (1-p)q^2 = (q-p)^2 + p(1-p).$$

因而在这个固定 x 上, 最小值在 $q = p$ 处取得. 二分类对数损失的条件期望为 $-p\log q - (1-p)\log(1-q)$, 对 $0 < p < 1$ 求导得 $(q-p)/(q(1-q))$, 同样在 $q = p$ 处取得最小值. 这说明两种损失都鼓励报告真实概率. 但有限数据、训练误差和分布偏移都可能使实际模型不校准; 常数预测等于总体正类率可以校准, 却几乎不区分不同输入.

一个简单的后处理是温度缩放: 对分类 logits z, 使用 $\mathrm{softmax}(z/\tau)$, 其中 $\tau > 0$ 在独立校准集上选择 [35]. 正温度不改变最大 logit 的位置, 只改变概率集中程度. 它因此可以改善概率指标而保持按最大概率选出的类别不变, 也无法修复分类排序错误. 温度过大趋向均匀, 过小趋向把概率集中到最大值. 每次修改温度仍是模型选择, 不应依据最终测试集调整.

新增 `calibration` 实验令 $X \sim N(0, 1.5^2)$, $Y \mid X \sim \mathrm{Bernoulli}(\sigma(X))$, 并故意给模型 logits $3X$. 它使用 800 个验证样本, 在 0.5 到 5 的 91 个正温度中以验证负对数似然选择温度, 然后只评价一次 5000 个独立测试样本. 种子 144 得温度 2.9, 接近生成规则中的 3. 这不是训练得到的现实模型, 是一个可控制的过度自信案例. 测试准确率前后均为 0.7362, 测试对数损失从 0.72520 降至 0.52642, Brier 损失从 0.19905 降至 0.17577. 分箱误差也记录下来, 但它依赖箱数与采样, 并不等于校准定义本身. 脚本计算的是正类概率的分箱误差, 不是多分类最大类别信心的 ECE.

5.7 分布外表现与因果限制

设训练和测试来自 P, 部署来自 Q. 上面的概率界只控制 R_P, 不控制 R_Q. 例如训练文档中的某个来源标记总与标签共现, 模型可以依靠标记取得极低经验错误率. 当部署中标记与标签反相关, 原有规律失效. 对基础模型, 新领域、语言、上下文长度、时间和工具环境都可能改变分布; "分布外" 必须说明相对哪个训练或评价分布而言.

旧 `spurious` 实验仍提供一个明确算例: $s = 2y - 1$, 稳健特征 $x_1 = s + 0.9\varepsilon_1$, 快捷特征在训练和同分布测试为 $x_2 = s + 0.1\varepsilon_2$, 偏移测试变为 $x_2 = -s + 0.1\varepsilon_2$. 独立标准正态噪声使第二个特征在训练中更容易使用. 保留 x_1 的对照是作者知道生成规则后设计的干预, 不是能自动从未知语言数据中找出因果特征的算法. 语言模型的类似问题可以通过改变来源标记、模板、实体组合或检索证据来构造, 但每个改动都要核对是否改变了正确答案.

相关性只说明在观察到的分布中共同变化. 要研究 "模型是否依据证据回答", 可以保持问题不变, 有控制地替换支持证据并预测答案应怎样改变; 要研究组合泛化, 可以把某些语法组合留出, 同时让相关词和基本规则在训练中出现. 这些设计比只报告一个总分更接近机制问题. 但替换可能引入不自然文本, 留出可能同时改变难度, 评分器也可能出错. 因而需要对照、错误分析和明确样本量, 不能把一次干预等同于完整因果解释.

习题 5.2 一个模型在 100 道题上答对 90 道. 研究者测试 200 个提示词并报告最好提示的 90% 正确率, 再用这 100 道题选择温度. 能否将其当作独立测试正确率与校准结果? 如何修正?

解答. 不能. 题目已经同时参与提示选择和概率后处理选择, 实际是开发数据. 可以固定已选提示和温度, 在新的同分布题目上评价, 同时另设目标明确的分布外题组. 若 200 个提示在评价前全部固定, 有限模型界可同时控制这些提示在有界评分上的偏差, 但它仍依赖独立评价单位, 且不给校准定义的完整保证. 若提示是根据这 100 题的失败逐步改写, 简单代入 $M = 200$ 不足以恢复已被破坏的条件. □

习题 5.3 二分类模型在所有输入上都预测 $q = 0.2$, 总体正类率恰为 0.2. 它是否校准? 这说明了什么? 若部署正类率变为 0.6, 又如何?

解答. 原分布下 $\mathbb{E}[Y \mid q = 0.2] = 0.2$, 因而校准. 但它没有根据输入区分不同事件, 所以校准不等于有用的表征或高区分能力. 部署分布下同一预测组的正类率为 0.6, 于是不校准. 在一个分布上通过校准检查, 不保证在另一分布上仍校准. □

习题 5.4 对 $M = 20$ 个事先固定的候选, 需要以至少 0.95 的概率让有界风险偏差均不超过 0.05. 使用本章结论给出充分样本数, 并说明它没有保证什么.

解答. 要求 $2Me^{-2n(0.05)^2} \le 0.05$, 得 $n \ge \log(800)/0.005 \approx 1336.923$, 所以取 $n = 1337$ 个独立评价单位足够. 它不保证所有候选的风险都低, 不保证选中模型是全体可能模型中最优, 不保证未截断对数损失, 也不保证部署分布不同后的表现. 条件及结论范围是这项计算的一部分. □

6 模型进步的数据与计算条件

6.1 把进步拆成可检验的变化

语言模型在多个任务上持续改善, 并不只因为参数数目增加. 参数决定可表示的函数类, 数据决定学习压力, 优化决定有限预算下找到什么参数, 系统实现决定多少计算实际可用, 后训练决定哪些行为被强化, 推理协议决定一次任务能生成与检查多少候选. 将这些因素分开, 才能提出可检验的问题.

例如要检验 “更多数据导致更好的模型”, 应在同一架构、优化与评价协议下改变训练 token 预算, 同时记录训练是否重复旧数据. 要检验 “清理数据更有效”, 应保持模型、训练计算和 token 数基本一致, 比较清理前后的资料. 要检验 “推理能力提升”, 则应分别报告单次生成与多候选选择, 否则搜索预算的变化会被误认为权重的变化. 这些实验不能完全控制现实系统的每个因素, 但至少把因果主张变成可反驳的比较.

训练对序列位置可以并行, 是注意力模型易扩展的工程条件之一. 同一训练文档的所有真实前缀已经给定, 因而可一次计算大量 token 损失; 自回归生成仍有逐 token 的依赖. 硬件、精度、算子融合与数据搬运的改善, 能在相同

墙钟预算下完成更多有效运算. 浮点算术量一样的两个实现也可能运行时间不同. 因此算法 FLOPs、设备峰值、实测吞吐与总时间是四个不同量.

为准确续写多种文本, 模型可能需要表示实体、关系、语法、代码状态与证明中的依赖. 这是 “条件预测压力促进有用表示” 的机制假说. 它不证明每一个低损失模型都形成正确因果世界模型: 有限资料可被记忆, 同一输出分布可以有不同内部实现, 数据中的错误规律也可能被准确拟合. 从生成成绩到一般智能或主观体验, 还隔着对象定义和证据要求, 不能由一个损失函数直接推出.

6.2 训练 FLOPs 的近似账本

训练预算常用浮点运算次数计量, 简称 FLOPs. 本节采用一次标量乘法与一次加法各算一次的约定, 所以一个乘加大约算两次. 设矩阵 $W \in \mathbb{R}^{m\times d}$, 一批 S 个 token 隐状态为 $X \in \mathbb{R}^{S\times d}$, 则前向 $XW^\top$ 约需 $2Smd$ 次运算. 反向计算输入梯度与权重梯度各有一个同级矩阵乘法, 合计前后向约为 $6Smd$.

如果网络中的多数参数都在这种稠密矩阵中, 每个有效 token 各使用一次, 把参数数 N 与累计训练 token 数 D 相乘, 得到

$$C_{\text{train}} \approx 6ND. \tag{6.1}$$

这里 D 是处理次数, 不是互不重复的文本量. 一份长度 T 的训练文档重复十轮就贡献约 $10T$ 个 token 运算, 但信息量没有因此自动乘十. Embedding 查表中的参数并不是每个 token 都作稠密乘法; 输出投影却会涉及整个词表. 因而 N 的计数约定应写清, 不能把任何参数总数都无条件代入.

注意力矩阵另有依赖序列长度的成本. 一层宽度 d、长度 T 的 $QK^\top$ 与 AV 各需约 $2T^2d$ 次前向运算; 各自的反向再增加约两倍, 前后向共约 $12T^2d$. L 层、累计 D 个 token 若由等长文档构成, 这部分约为 $12LDTd$. 相比投影与前馈的 $O(LDd^2)$, 长上下文时它不能忽略. 稀疏路由模型还要区分总参数与每 token 激活参数; 重计算会增加 FLOPs; optimizer 更新、通信、数据读取与检验也有成本. 式 (6.1) 是稠密、矩阵运算占主导时的尺度近似.

例题 6.1 取 $N = 10^9, D = 2\times 10^{10}$, 忽略额外注意力与系统成本. 若实测持续吞吐为 10^{14} FLOPs/s, 估计核心训练时间. 若该吞吐实际只有设备峰值的 $1/4$, 能否仍用峰值作除数?

解答. 近似计算为 $6ND = 1.2\times 10^{20}$ FLOPs, 除以持续吞吐得 1.2×10^6 秒, 约 13.9 天. 应使用实际完成本任务运算的吞吐; 用四倍峰值作除数会低估约四倍. 检查点、评测与数据停顿还可能增加总时间. □

这种账本能解释为什么训练大模型需要同时优化算法与系统, 也说明论文中的同 FLOPs 比较和实际同时间比较并非相同实验. 本书的 NumPy 小模型只用于机制复算, 并未测量大型分布式训练的利用率.

6.3 规模损失与预算最优点

规模规律研究的是: 固定模型族、数据分布与训练协议, 最终损失怎样随模型大小和处理资料改变. Kaplan 等论文报告过近似幂律关系 [9]. Hoffmann 等在 Chinchilla 研究中训练了 400 余个模型, 覆盖约 70M–16B 参数、5B–500B token, 并比较几种预算分配估计方法; 最后以 70B 参数、1.4T token 训练 Chinchilla, 与 280B 参数 Gopher

的训练预算比较 [10]. 这些是作者的实验范围与报告, 不是本项目复现的数据.

为了完整理解预算平衡, 采用一个待拟合模型

$$L(N, D) = L_\infty + aN^{-\alpha} + bD^{-\gamma}, \qquad a, b, \alpha, \gamma > 0. \tag{6.2}$$

L_∞ 表示该模型形式假定的极限损失, 两项分别随参数与 token 增加而下降. 本节用 γ 表示数据指数, 避免和后训练的 KL 系数 β 混淆. 加法分解是经验模型假设; 它没有由概率链式法则自动推出. 对优化不足、重复数据或分布变化, 这个形式可能拟合不好.

给定数据点 (N_i, D_i, L_i), 可通过最小化预测与观测损失的某种残差拟合五个常数. 因为 L_∞ 与幂项之和需要先减再取对数, 不能简单把 $\log L$ 对 $\log N$ 回归就声称得到所有指数. 参数与 token 总是按一个固定比例同时变化时, 两类效应也难以分别识别. 较可信的设计要改变两个轴, 留出若干尺寸或预算检验预测, 对训练停止与学习率安排作控制, 再报告拟合不确定性与外推限制.

定理 6.1 *在式 (6.2) 成立且约束为 $C = kND$、$k > 0$ 的连续近似下, 唯一损失最小点为*

$$N_* = \left(\frac{\alpha a}{\gamma b}\right)^{1/(\alpha+\gamma)} \left(\frac{C}{k}\right)^{\gamma/(\alpha+\gamma)},$$
$$D_* = \left(\frac{\gamma b}{\alpha a}\right)^{1/(\alpha+\gamma)} \left(\frac{C}{k}\right)^{\alpha/(\alpha+\gamma)}.$$

在该点, $\alpha a N_*^{-\alpha} = \gamma b D_*^{-\gamma}$.

证明. 约束使 $D = C/(kN)$, 因而可变部分为 $f(N) = aN^{-\alpha} + b(kN/C)^\gamma$, $N > 0$. 求导并乘以正数 N, 得

$$Nf'(N) = -\alpha a N^{-\alpha} + \gamma b (kN/C)^\gamma.$$

第一项绝对值随 N 严格下降, 第二项严格上升. 当 N 从零增到无穷时, 导数符号只由负变正一次. 解平衡式,

$$N^{\alpha+\gamma} = \frac{\alpha a}{\gamma b}(C/k)^\gamma,$$

得 N_*, 再用约束得 D_*. f 在此点之前下降、之后上升, 所以这是唯一全局最小点. 最后的等式就是驻点条件. □

边际平衡并不是让两项误差总相等. 若指数不同, 平衡的是指数乘误差. 用对数预算理解更直接: 增加 $\log N$ 会以速度 $\alpha a N^{-\alpha}$ 降低第一项, 同时固定计算迫使 $\log D$ 等量下降, 使第二项以速度 $\gamma b D^{-\gamma}$ 上升. 最优点使两者相等.

若 $\alpha = \gamma$, 参数与数据都随 $C^{1/2}$ 增长, 但前面的常数仍依赖 a/b. 若 $\alpha = 1/2, \gamma = 1$, 参数随 $C^{2/3}$、数据随 $C^{1/3}$ 增长. 计算增加八倍时, 两者分别增加四倍与两倍. 最优超额损失的两项都随 $C^{-\alpha\gamma/(\alpha+\gamma)}$ 缩减, 可以代入上面的解直接验证. 这一推演是本书对明确假设的数学分析, 不代表任何现实模型自动满足同样指数.

Chinchilla 第 3 节中不同估计路线给出的最优指数并非逐位相同, 其中固定模型训练曲线方法报告接近 0.50/0.50, 等 FLOPs 方法报告约 0.49/0.51[10]. 因而应把 “参数与数据协同增长” 当作该实验范围内的经验发现, 而非把某一个 token/参数比奉为所有训练的固定律. 本项目没有多尺寸多预算语言训练网格, 不能用一条小模型训练曲线检验这些幂律.

6.4 部署成本改变预算选择

若训练后要服务大量请求, 目标不能只含一次训练成本. 对稠密网络, 每生成一个 token 的矩阵前向主项近似为 $2N$ FLOPs, 还需计入注意力、输出、缓存读写与检索工具. 设生命周期生成 token 数为 M, 用简化总账本

$$C_{\text{total}} = kND + qNM,$$

其中训练常取 $k \approx 6$, 前向矩阵主项常取 $q \approx 2$. M 是部署需求, 不是训练资料. 将固定预算 C_0 解为 $D = C_0/(kN) - qM/k$, 需要 $0 < N < C_0/(qM)$, 才有正训练 token 预算.

令 $h = qM/k$. 代入损失模型并对 N 求导, 注意 $D'(N) = -C_0/(kN^2)$, 得

$$f'(N) = -\alpha a N^{-\alpha-1} + \gamma b D^{-\gamma-1}\frac{C_0}{kN^2}.$$

因此驻点满足

$$\alpha a N^{-\alpha} = \gamma b D^{-\gamma}(1 + h/D).$$

与仅训练的条件相比, 右边多出 $1 + h/D$: 模型变大既挤占训练 token, 又增加每次部署成本. 这个模型中, 大量服务需求倾向选择更小模型、训练更多 token. 例如在训练最优点之后略增大 N, 仅训练目标已上升; 总成本约束下 D 比仅训练情形还小, 正的上升项更大, 因而部署最优点不会由这个效应移到更大的 N 一侧. 现实成本还受延迟、批大小、量化与设备内存约束, 不宜仅用 FLOPs 决定部署.

缓存把重复计算换成存储, 所以单请求更快也可能减少设备能并发容纳的请求数. 输出很长时, M 增长与缓存长度增长同时发生. 更强的小模型、蒸馏、量化或更准确的早停都有可能降低成本, 但它们也改变质量条件. 比较两个产品应明确质量、延迟与总吞吐的约束, 不能只报告 “参数更少”.

6.5 资料质量改变同一 token 预算

设训练混合分布为 $P = \sum_j w_j P_j$, $w_j \geq 0, \sum_j w_j = 1$. 对固定模型 q, 期望交叉熵为 $\sum_j w_j H(P_j, q)$. 改变权重即改变学习目标. 如果评价分布为 Q, 降低 P 上损失不一定降低 Q 上损失. 例如一个模型容量只允许输出固定字母分布, 训练将 90% 权重放在 “a” 上, 而测试把 90% 放在 “b” 上, 训练频率拟合得更准也不能解决任务分布错位.

重复资料可以帮助优化收敛, 也会使某些文本被过度加权. 若一份文档复制十遍, 最大似然目标中它的权重就增大十倍; 算力增加并没有创造十份独立证据. 近重复的模板、网页导航与乱码也可能占去大量 token. 质量过滤尝试把预算用于更有用的模式, 但过滤过强可能删除小众知识、方言或困难样本, 将偏好误当正确性.

FineWeb 论文的 v2 版本通过小模型训练来比较抽取、过滤与去重的选项, 其第 3 节含去重范围的对照, 第 4 节含教育质量过滤与下游评价 [43]. 它的价值不只是提供资料名称, 而是把 “清理更好” 变成固定训练设置下可观察的消融. 其中第 3.4 节的具体反例是: 按时间跨全部抓取全局去重, 对旧抓取可删去约 90% 资料, 留下的资料反而在其训练评价中较差; 逐抓取独立去重得到更好的平均成绩. 这说明更激进地删重复不必然更好, 去重还改变了保留内容的分布. 因而既要报告过滤规则也要报告可用 token 总数. 这些作者结果受模型尺寸、语言、数据混合与基准集合限制, 不证明同一个过滤器在全部模型与语言上最佳.

评价污染尤其容易混淆. 若训练中包含测试题及答案, 模型成绩可能反映重现, 而非迁移. 去除字符串完全相同

的题面不足以处理改写、答案解析或由教师生成的近似题. 因而资料质量研究同时需要检查测试重合、来源划分与新任务保留点. 合成资料也需要这个审查: 教师能生成流畅文本, 但错误、模式重复和任务窄化可能被学生一起吸收.

6.6 阈值、涌现与行为边界

评价常把连续概率压成离散得分. 精确匹配只记录完整答案对错, 部分得分则保留一些结构. 若一个任务要求 m 个步骤全正确, 简化地假设各步独立且正确率相同为 p, 则总成功率为 p^m. 当 $m = 20$, $p = 0.7$ 时约为 0.000798, $p = 0.9$ 时约为 0.121577. 每步只是连续改善, 完整成功却可能从测不到变成明显可见.

这个变化还是连续函数, 没有数学断点. 若每个规模只测几十题, 小概率成功可能暂未出现; 若绘图仅标 "是否超过某阈值", 连续改善就成为阶跃标签. Schaeffer 等的涌现指标论文讨论了指标选择和统计分辨率能怎样影响这类图像 [18]. 这提醒我们同时观察概率分数、部分得分、精确正确率和置信区间, 不意味着所有结构变化都由指标造成.

步骤独立是上述算例的条件. 若错误共享同一误解, p^m 无效; 若不同题有不同 p, 先平均 p 再幂运算也无效. 例如一半题可完美解决、另一半完全不能解决, 平均单次正确率为 $1/2$. 在同一题反复独立采样也不会让后半题变好, 整套覆盖率始终是 $1/2$, 不能套 $1-(1-1/2)^N$ 宣称趋于 1. 这类异质性会在下一章的预算曲线中再次出现.

"能力出现" 至少可以表示三种不同主张: 在既定任务上超过可观测阈值; 系统能完成以前未完成的任务; 网络内部形成新的计算机制. 前两者可以由行为证据支持, 第三者需要机制研究. 即使内部机制改变, 是否与人类智能同构、是否具有意识, 仍不是由一个 benchmark 分数决定的事实. 本书采用能检验的条件预测、迁移与任务成绩, 把更广的哲学判断留在其证据范围之外.

6.7 练习与解答

习题 6.1 在模型 $L = L_\infty + aN^{-1/2} + bD^{-1}$ 中, 某一点的两项超额损失恰好相等. 在固定 ND 下它是否最优? 应怎样调整?

解答. 最优要求 $(1/2)aN^{-1/2} = bD^{-1}$, 不是两项相等. 若两项均为 $e > 0$, 增加 $\log N$ 的导数为 $-e/2 + e = e/2 > 0$. 应减小 N、增加 D, 直到两类边际收益平衡. 只比较误差数值而忽略指数会选错方向. □

习题 6.2 某团队把同一训练语料重复四轮, 声称 "数据量增加四倍, 所以验证了数据规模规律". 哪个量确实增加? 需要怎样设计才能检验新资料的效应?

解答. 处理 token 数与核心训练计算增加四倍, 不同文本数量没有增加. 可设重复旧资料与加入同分布新资料两个组, 保持总处理 token、模型和优化基本一致, 在独立保留资料上比较. 还要检查两组是否已收敛, 不能把额外训练步数和新信息混为一谈. □

习题 6.3 模型甲单次正确率较高, 模型乙允许多生成 100 次并使用精确 oracle 选择后成绩更高. 这支持哪一项结论? 怎样比较实际部署系统?

解答. 支持乙在该生成与 oracle 协议下有较好的系统结果, 不能单独归因为权重更强. 应报告单次成绩、候选覆盖、真实选择器成绩、总 token 与时间, 并比较相同成本的预算曲线. 如果部署不能获得 oracle, 还要测替代验证器的误拒与错误接受. □

7 后训练的目标与反馈

7.1 监督微调与奖励模型

预训练拟合所选文本分布, 用户需要的却常是特定任务下的可靠回答. 指令与答案组成 (x, y), 对话格式中的角色边界也是 token 序列的一部分. 监督微调 SFT 仍使用交叉熵, 但通常只把回答部分放进损失:

$$\mathcal{L}_{\mathrm{SFT}} = -\frac{1}{S} \sum_{(x,y)} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}).$$

这里 S 是全部有效回答 token 数; 若按回答平均, 则另写分母与权重. 提示仍是可见输入, 只是提示位置不作为此次目标. 将 padding 与提示 mask 合并到式 (2.1) 就能实现. SFT 学习模仿示范, 若示范有系统错误或缺少任务覆盖, 损失函数不会自动补足它们.

偏好资料则包含同一提示下的胜者 y_w 与败者 y_l. 假定一个标量奖励 $r_\phi(x, y)$ 能表示偏好, Bradley–Terry 模型设

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)), \quad \sigma(u) = 1/(1 + e^{-u}).$$

模型损失为成对平均的 $-\log \sigma(r_w - r_l)$. 语言奖励网络可把 $B \times T \times d$ 的最终隐状态在回答结束位置抽取成 $B \times d$, 再用线性读出得到 B 个实数. 若 $s = r_w - r_l$, 则损失对 r_w, r_l 的导数分别为 $-\sigma(-s)$ 与 $\sigma(-s)$: 偏好差不够大时, 更新提高胜者分数并降低败者分数.

奖励只能由差确定. 对同一提示, 给全部回答奖励加上 $c(x)$, 偏好概率完全不变. 不同标注者也可能有不同标准, 偏好可能不传递, 因而这个标量模型是统计假设. RLHF 中优化策略使用这个学到的奖励, 不能把它与客观正确性画等号. SFT、偏好建模与策略优化这三个对象在 DPO 论文的预备部分被明确区分 [14]; 本章以下只在有限回答集合上推导可证明的关系.

7.2 KL 正则奖励与 DPO

固定提示 x, 暂把回答集合设为有限集合 $\mathcal{Y}$. 参考策略 $\pi_{\mathrm{ref}}(y) > 0$, 奖励 $r(y)$ 有限, 系数 $\beta > 0$. 希望提高平均奖励同时限制分布偏移, 考虑

$$J(\pi) = \sum_y \pi(y) r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}.$$

约定 $0 \log 0 = 0$. KL 惩罚的方向是 $\mathrm{KL}(\pi \| \pi_{\mathrm{ref}})$, 反向 KL 是另一目标. 在长回答中, $\log \pi(y \mid x)$ 为回答 token 的条件对数概率之和; 若改成每 token 平均, 奖励关系也改变, 不能在推导中偷偷替换.

定理 7.1 令 $Z = \sum_y \pi_{\text{ref}}(y) e^{r(y)/\beta}$, $q(y) = \pi_{\text{ref}}(y) e^{r(y)/\beta}/Z$. 对任意概率分布 π, 有

$$J(\pi) = \beta \log Z - \beta \,\mathrm{KL}(\pi \| q).$$

因此唯一最优策略为 $\pi = q$.

证明. q 全正且和为 1. 由 $\log q(y) = \log \pi_{\text{ref}}(y) + r(y)/\beta - \log Z$, 得

$$\beta \,\mathrm{KL}(\pi \| q) = \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)} - \sum_y \pi(y) r(y) + \beta \log Z.$$

移项即为恒等式. KL 非负且仅在两个分布相同时为零, 所以最优值为 $\beta \log Z$, 取到且唯一. □

最优分布的概率比为 $q_i/q_j = (\pi_{\text{ref},i}/\pi_{\text{ref},j}) e^{(r_i - r_j)/\beta}$. 参考概率影响奖励相同候选的比例. β 很大时 q 接近参考; 趋于零时集中到最高奖励集合, 集合内部比例由参考保留. 假定参考全正保证每个回答可获得概率; 若参考为零, 应在其支持上讨论, KL 不允许以有限代价向外分配正概率.

要转为偏好目标, 将最优策略公式改写成

$$r(y) = \beta \log \frac{\pi^*(y)}{\pi_{\text{ref}}(y)} + \beta \log Z.$$

同一提示下两回答相减, $\log Z$ 抵消. 代回 Bradley–Terry 概率, 再用参数策略代替未知最优策略, 对已观察的胜负对最大似然, 得到

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma(s_\theta),$$
$$s_\theta = \beta \left(\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right).$$

其梯度为

$$\nabla \mathcal{L}_{\text{DPO}} = -\mathbb{E}\big[\beta \sigma(-s_\theta)(\nabla \log \pi_\theta(y_w \mid x) - \nabla \log \pi_\theta(y_l \mid x))\big].$$

每份胜负对产生一个实数 margin, 故一批大小 B 的成对损失是 B 个标量的平均. 实现上分别构造胜者与败者序列, 用 teacher forcing 得到 $B \times T_w \times V$ 与 $B \times T_l \times V$ 的 logits, 仅对回答有效位置抽取并求和 log probability; 固定参考网络做同样运算且不累计参考梯度. 不必把全部回答显式枚举, 因为 Z 已在差中消去.

这条推导对应 DPO 原文第 4 节与附录 A 的奖励–策略关系 [14], 但本地定理只证明给定有限奖励问题的最优分布. 偏好分布是否满足假设、有限数据能否识别奖励、神经参数能否表示该策略、优化器能否找到它, 是另外的问题. 不存在 “有 DPO 公式就保证人类目标已实现” 的推论.

已有 `preference` 实验用八个动作, 已知奖励、非均匀参考与带噪偏好拟合八个 logits. 若 $\pi = \mathrm{softmax}(z)$, 胜负对中的 softmax 对数归一化相消, 所以 $s = \beta[(z_w - \log \pi_{\text{ref},w}) - (z_l - \log \pi_{\text{ref},l})]$. 单对导数仅在胜者与败者坐标上非零, 分别为 $-\beta\sigma(-s)$ 与 $\beta\sigma(-s)$. 新 `reward_experiment` 则直接优化四个动作的 KL 奖励目标并与解析最优比较. 两者分别检查偏好拟合与奖励优化, 不能混成同一个实验.

7.3 可验证奖励与多阶段后训练

偏好奖励适合表达帮助程度等相对判断, 数学答案和程序测试则能提供较明确的结果奖励. 可验证奖励强化学习 RLVR 用这种反馈改变生成策略. 一批提示 $x_1, \dots, x_B$ 各采样 G 个回答, 得到 $B \times G$ 个序列奖励; 每份序列仍由逐 token 条件概率组成. 策略梯度怎样把标量奖励传到 token logits, 见第 8 章的命题 8.3.

DeepSeek-R1 的 Nature 正式论文于 2025 年 9 月 17 日发表 [44]. R1-Zero 在已预训练的 DeepSeek-V3 Base 上用 GRPO 和规则奖励训练, 未先作推理 SFT. R1 的流程则包含冷启动示范、RL、拒绝采样后 SFT、再作 RL, 并结合推理与一般任务. 因而这份工作同时展示了可验证反馈与多阶段资料组织, 不能只概括成 “靠一种奖励从零学会思考”.

为看懂组内更新, 固定一个提示, 设奖励向量 $r \in \mathbb{R}^G$, 组平均 $\bar{r} = G^{-1} \sum_i r_i$, 相对量可写为 $A_i = (r_i - \bar{r})/(s_r + \epsilon)$, 其中 s_r 是采用明确约定的组内标准差, $\epsilon > 0$ 防止零分母. 如奖励为 $(1, 0, 1, 0)$, 用总体标准差 $s_r = 1/2$, 忽略极小 ϵ 时, 相对量为 $(1, -1, 1, -1)$. 若奖励全为 1, 相对量全零, 此组不提供区分回答的信号.

先用一个序列级教学替代项说明裁剪. 对从旧策略采样的回答, 定义整序列概率比 $\rho_i = \pi_\theta(y_i \mid x)/\pi_{\text{old}}(y_i \mid x)$. 裁剪项为

$$\min\{\rho_i A_i, \operatorname{clip}(\rho_i, 1-\delta, 1+\delta) A_i\}, \qquad 0 < \delta < 1.$$

当 $A_i > 0$, 过大的概率比不再继续增加此项; 当 $A_i < 0$, 过小的概率比受对应限制. KL 项进一步约束对参考策略的偏离. 这些操作旨在控制一次更新的变化, 不是最优策略闭式解或全局收敛保证. Nature 正式论文的 Methods 式 (1) 用序列级记号概括目标; 更细的 token 级训练式见 R1 预印本的方法部分 [44, 17]. 对第 i 份回答的第 t 个 token, 概率比是

$$\rho_{it} = \frac{\pi_\theta(y_{it} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x, y_{i,<t})}.$$

一种对应的 token 平均结构为

$$\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left[\min\{\rho_{it} A_i, \operatorname{clip}(\rho_{it}, 1-\delta, 1+\delta) A_i\} - \beta D_{\text{KL},it} \right].$$

这里 $D_{\text{KL},it}$ 表示所选 token 条件下的 KL 项或其论文定义的采样估计, 必须按实现给出. 整序列比等于各 token 比的乘积, 不等于平均; 裁剪乘积也不等于逐 token 裁剪后平均. 因而上面的序列算例不能直接当作完整 GRPO 实现. 完整算法还要明确样本长度权重、采样温度、参考更新及零方差处理. 尤其组平均含自身奖励, 不能直接套用与当前回答无关的无偏基线证明; 第 8.6 节给出其简化情况下的期望分析.

拒绝采样则是先生成回答, 由某个标准保留合格者, 再把保留资料用于监督训练. 若保留谓词为 $V(y)$, 对固定提示, 资料分布为 $\pi(y \mid x) V(y) / \sum_z \pi(z \mid x) V(z)$, 前提是分母正. SFT 模仿的是这个筛选后的分布. 验证器不完备时可能丢掉可行路线, 有错误接受时会把错误带入示范. 这种资料筛选与直接策略梯度是不同计算过程, 即使都利用了同一结果反馈.

7.4 LoRA 的秩约束与梯度

SFT、DPO 或奖励优化都要改变网络参数, 但可以选择只改变一部分矩阵. LoRA 冻结基矩阵, 把更新参数化为两个小矩阵的乘积 [13]. 为和线性代数的列向量约定一致, 设

$$W_0 \in \mathbb{R}^{m\times d}, \quad A \in \mathbb{R}^{r\times d}, \quad B \in \mathbb{R}^{m\times r}, \quad h = (W_0 + sBA)x,$$

其中 $x \in \mathbb{R}^d$, $s > 0$ 是固定缩放, 常取与 α/r 有关的约定. 若代码使用行向量, 一批输入为 $X \in \mathbb{R}^{n\times d}$, 输出是 $X(W_0 + sBA)^\top \in \mathbb{R}^{n\times m}$, 这只是转置约定不同.

命题 7.2 对损失 ℓ, 若输出梯度为 $u = \nabla_h \ell \in \mathbb{R}^m$, 则

$$\nabla_B \ell = s\, u(Ax)^\top \in \mathbb{R}^{m\times r}, \qquad \nabla_A \ell = s\, B^\top u\, x^\top \in \mathbb{R}^{r\times d}.$$

对于批损失, 若 $G = \nabla_W \ell \in \mathbb{R}^{m\times d}$, 相应公式为 $\nabla_A \ell = sB^\top G$, $\nabla_B \ell = sGA^\top$.

证明. 冻结 W_0, 有 $dh = s(dB)Ax + sB(dA)x$. 于是 $d\ell = u^\top dh$. 第一项写为 $\mathrm{tr}([s\,u(Ax)^\top]^\top dB)$, 第二项写为 $\mathrm{tr}([sB^\top ux^\top]^\top dA)$, 与 Frobenius 内积的梯度定义对照即得. 批版本由 $dW = s(dB)A + sB(dA)$ 作同样展开得到. □

矩阵乘积满足 $\mathrm{rank}(BA) \le r$. 当 $1 \le r \le \min(m, d)$ 时, 任意秩至多 r 的更新均可用这种乘积表示: 对其奇异值分解, 把非零奇异值的平方根分到左右因子, 不足 r 的部分补零即可. 秩限制作用于更新 ΔW, 不是说最终 $W_0+\Delta W$ 也只有秩 r. 若目标差矩阵有超过 r 个非零奇异值, 该参数化无法精确表达它.

可训练参数为 $r(m + d)$, 完整矩阵为 md. 只有 $r < md/(m + d)$ 时才真的减少参数数目. 对 $m = d$, 需要 $r < d/2$, 不能只说 “秩低于矩阵维度就省参数”. 因子不唯一: 任意可逆 $R \in \mathbb{R}^{r\times r}$ 给出 $(BR)(R^{-1}A) = BA$, 这也表明不能把因子坐标本身解释为唯一机制.

常见初始化令 $B = 0, A$ 随机, 则初始更新为零, $\nabla_A \ell = 0$, 但 $\nabla_B \ell$ 可非零. 下一步 B 改变后, A 才可能收到梯度. 若两者同时设为零, 两个梯度都为零, 这个纯乘积分支会停在零处. 训练结束后可把 sBA 合入 W_0, 不增加这份合并矩阵的推理乘法; 切换多个适配器时则要保留各自因子. 冻结基矩阵减少其梯度和优化器状态, 却不消除计算适配器梯度所需的激活.

新实验取 $n = 7, d = 5, m = 4, r = 2, s = 0.75$, 对平方损失的全部因子坐标做中心有限差分. A 与 B 的最大归一化差分别为 4.02×10^{-10} 与 6.61×10^{-11}, 更新秩为 2. 可训练参数为 18, 相比完整矩阵 20 仅略少, 正好提醒 “低秩” 与 “大幅节省” 需要具体尺寸. $B = 0$ 时 A 梯度范数为零, B 梯度范数约为 6.2393, 与推导相符.

7.5 公开小模型的评分与生成诊断

前面的机制实验由我们定义任务与参数。公开权重则检验真实语言模型在一个明确协议下怎样输出。这里选择同一系列 SmolLM2-135M 基础与 Instruct 权重，两者参数量均为 134,515,008。固定官方版本提交，Apache 2.0 模型许可另行核对；代码不用远程自定义执行，只读取 safetensors。总权重下载约 538 MB，缓存留在公开仓库外，不把权重或原始训练数据装进源码包。

在看到输出前，我们冻结 18 道英文选择题：六道加法、六道上下文符号映射、六道 system/user 冲突。两组权重共享同一纯文本输入格式；Instruct 另测原生聊天模板，共 54 条记录。题目、完整输入、token ID、输出、版本与环境均保留。原生聊天条件同时涉及特殊 token 与消息边界，不能把它和基础纯文本之差完全归因于后训练。

两种评价应当分开。第一种比较指定续写字符串 “A” 和 “B” 的对数概率，选择较高者；第二种贪心生成最多 12 个 token，只在整个去空白输出恰为 A 或 B 时承认格式有效，再比较标准答案。前者的归一化为

$$\widetilde{p}(A) = \frac{\exp \ell_A}{\exp \ell_A + \exp \ell_B},$$

只对两个指定 token 字符串归一化。即使 $\widetilde{p}(A)$ 很大，模型仍可能更偏好生成一整句话。它不是全输出正确率，不是经校准的置信度。第二种又会将知识上可能正确的解释判成接口格式失败，所以零分也不意味着完全没有相关知识。

权重	输入	两选项正确	格式通过	严格生成正确
基础	统一文本	10/18	0/18	0/18
指令	统一文本	10/18	0/18	0/18
指令	原生聊天	10/18	0/18	0/18

这次三个条件的两选项评分恰好都选择 A；题目共有十个 A 答案，因此 10/18 等于常选 A 的基线。分组是加法 3/6、上下文映射 4/6、角色冲突 3/6。这个事实使 “准确率 55.6%” 成为一个需要解释的诊断，而不能据此宣称指令模型更好或两个模型相同。生成文本实际不同，例如同一加法题，基础模型继续写对话，原生聊天 Instruct 生成 `B: 6`，该题正确答案却是 A 对应的 5。评分选择与自由输出还会不一致。

严格格式与严格生成成功均为零。我们保留这个失败，未重写提示或挑另一份权重来换好结果。第二次缓存运行的指标文件逐字节相同；运行耗时和缓存状态另存环境文件，不要求不同机器耗时一致。这是低预算、非自适应的诊断，不是数学能力榜单、IH-Challenge 复现或 RLVR 的因果实验。

下一次研究应另立设计：在开发题上确定与训练分布相近的输出接口，将格式成功、语义成功、选择偏置和拒绝分别统计；然后一次性评价独立题集。若要研究后训练效果，应尽量固定基础初始化、数据与计算，或明确只能比较公开成品。题数扩大并报告成对差异和区间，才可能支持超出这 18 个具体题目的结论。

7.6 复算与练习

习题 7.1 设参考策略为 $(1/2, 1/2)$, 奖励为 $(0, \beta \log 3)$, 求 KL 正则最优策略. 若奖励同时加上 100, 最优策略与偏好概率有何变化?

解答. 未归一化概率为 $(1/2, 3/2)$, 最优策略为 $(1/4, 3/4)$. 加常数使两项同乘 $e^{100/\beta}$, 归一化后不变; 奖励差不变, 所以 Bradley–Terry 偏好也不变. 目标最优值增加 100, 策略不变. □

习题 7.2 一层矩阵 $W_0 \in \mathbb{R}^{12\times 8}$, 用秩 $r = 5$ 的 LoRA. 它比完整矩阵参数更少吗? 若同时零初始化两个因子, 用本章梯度解释第一步发生什么.

解答. 完整参数为 96, 因子参数为 $5(12+8)=100$, 反而更多. 两因子为零使两项梯度都为零, 在没有别的扰动或分支影响因子的普通梯度更新中, 第一轮不会动, 后续也停在零处. □

8 推理、验证与推理时计算

8.1 中间序列怎样参与条件预测

给出正确答案是一种可观测行为. 解释网络怎样得到它, 则是另一项研究任务. 本章在行为层面讨论推理: 依据规定关系组合多步信息, 在新组合与有控制的扰动下, 得到可验证的结论. 流畅的文字与长篇解释都不单独满足这个要求.

把提示记为 x, 中间文本记为 c, 最终答案为 a. 同一个自回归模型可以生成联合分布

$$p_\theta(c, a \mid x) = p_\theta(c \mid x)p_\theta(a \mid x, c).$$

若所有允许的中间文本构成有限集合, 则答案边缘概率为 $p_\theta(a \mid x) = \sum_c p_\theta(c \mid x)p_\theta(a \mid x, c)$. 一次实际生成通常只抽到一个 c, 而没有精确求和. 改变提示让模型生成中间步骤, 可能改变所用的条件任务与计算路径, 所以 "直接回答" 与 "先写中间步骤" 的输出成绩不同并不违反概率定义.

例如计算 $(17+8)\times 3$, 可先生成 "17+8=25", 然后在后续 token 中读取 25 再计算乘积. 中间 token 能保存子结果, 也给网络增加前向次数. 但如果第一步误写为 24, 后续就可能沿错前缀继续. 正确的最终数值也可能伴随错误推导, 所以最终答案验证与过程验证是两个指标.

Wei 等的 CoT 论文摘要报告, 在所研究的大模型与算术、常识、符号任务中, 给出中间步骤的示例改善了行为成绩 [15]. 它支持这种提示协议在该实验范围内有效, 不证明所有任务都需要文字思维链, 更不证明可见文字是完整内部计算的逐字记录. 网络也可能在隐状态中保留计算, 或从类似训练例子重现答案.

要测试解释的忠实性, 可以改变实际影响答案的因素, 再观察解释是否追踪改变. 例如给算术题加入一个无关的暗示答案, 模型答案随暗示而变, 解释却只声称按题面计算, 就暴露出文字理由遗漏了行为原因. 一次这样的干预只覆盖一种因素. 若解释能预测未见扰动下的行为, 证据比事后听起来合理更强; 若干预改变整个任务, 又不能把差异全归到单个机制. 行为成绩、解释可信程度与内部因果机制需要分别测量.

8.2 精确验证器与错误接受

设任务的真实正确谓词为 $C(y)\in\{0,1\}$, 验证器为 $V(y)\in\{0,1\}$. 若 $V(y)=1$ 必有 $C(y)=1$, 称其对该规格可靠; 若 $C(y)=1$ 必有 $V(y)=1$, 称其完备. 两者都成立时, 验证器才准确刻画所选候选集合上的正确性. 这些性质是相对于规格而言, 规格本身是否表达用户目的仍需审查.

寻找整数 a, b 满足 $a^2+b^2=65$ 时, 验证器可以直接执行精确整数运算. $(1,8)$ 与 $(8,1)$ 合格, $(4,6)$ 与 $(0,0)$ 不合格. 生成器可能很复杂, 验证这一个等式却很简单. 对证明, 可以交给形式系统检查规则; 对程序, 可以运行测试. 有限测试只证明通过这些测试, 不证明全部输入正确. 使用浮点误差阈值验证等式时, 验证的已是一个近似规格, 不

能把它和整数等式混为一谈.

令随机候选正确率为 p, 验证器对正确答案的误拒率为 $f = P(V = 0 \mid C = 1)$, 对错误答案的错误接受率为 $e = P(V = 1 \mid C = 0)$. 在分母非零时, Bayes 公式给出

$$P(C = 1 \mid V = 1) = \frac{p(1-f)}{p(1-f) + (1-p)e}.$$

如果正确候选很少, 即使 e 很小, 被接受者也可能大多错误. 例如 $p = 0.001, f = 0, e = 0.01$, 接受后的正确率只有 $0.001/(0.001 + 0.999 \cdot 0.01) \approx 0.0910$. 因而总体分类准确率很高的验证器, 仍可能不适合稀有解搜索.

上述 e, f 针对一个给定候选分布. 优化策略或扩大搜索后, 候选可能集中到验证漏洞, 原有误差率未必保持. 验证器输出的置信分数还涉及排序, 二分类错误率不能直接保证最高分候选正确. 下一节将把覆盖与实际选择彻底分开.

8.3 覆盖率与实际选择率

定义 8.1 对一个任务, 生成 N 个候选后, 至少有一个真正正确答案的事件概率称为候选覆盖率, 常用 pass@N 表示. 实际提交答案的正确率取决于选择器, 本章称为选择准确率. 任何选择器若只能提交这批候选之一, 其正确率不超过覆盖率.

命题 8.2 *在固定题目、固定采样协议下, 若候选独立同分布且每次正确率为 p, 则覆盖率为 $1-(1-p)^N$. 增加一个候选的边际覆盖收益为 $p(1-p)^N$.*

证明. 全部 N 次失败的概率是 $(1-p)^N$, 取补事件即得覆盖率. $N+1$ 次覆盖率减去 N 次覆盖率为 $(1-p)^N - (1-p)^{N+1} = p(1-p)^N$. □

若 $p = 0.2, N = 4$, 覆盖率为 0.5904. 但始终提交第一份答案的正确率仍为 0.2. 完美验证器找到任何正确候选就提交、没有正确候选就弃答, 才能使 "提交且正确" 的无条件概率等于覆盖率. 其中弃答率、已提交答案的条件准确率和无条件成功率应分别报告.

独立性不是随手附加的修辞. 如果 N 次生成只是复制同一个答案, 覆盖率仍为 p, 不随 N 变化. 若题目先随机抽取, 再对该题独立生成, 记题目成功率为 p_x, 则整套题的平均覆盖为 $\mathbb{E}_x[1-(1-p_x)^N]$. 对 $N \geq 2$, 函数 $1-(1-p)^N$ 为凹函数, Jensen 不等式给出平均覆盖不超过 $1-(1-\mathbb{E}p_x)^N$. 把全套平均 pass@1 代入独立式, 往往高估预算收益.

已有一批 n 个候选, 其中 c 个正确, 在其中均匀不放回选 $N \leq n$ 个时, 覆盖概率为

$$\widehat{\text{pass@}N} = 1 - \frac{\binom{n-c}{N}}{\binom{n}{N}},$$

约定 $n - c < N$ 时分子为零. 分子数的是只从错误池挑出 N 项的方法数, 分母是全部选法. 若最初 n 份候选本身 IID, 对整个候选池取期望时, 该估计对应固定协议下 N 次 IID 生成覆盖率. 对一个固定有限池, 则应把它理解为池内子集统计, 不能越过条件说成模型的无限生成能力. 本项目旧搜索实验从 600 个候选池中不放回抽样, 属于后者.

例题 8.1 一个有放回生成器每次以概率 0.20 产生正确候选, 以概率 0.03 产生被错误评分为最高的候选, 其余概率 0.77 为普通错误. 选择器给错误高分候选 2 分, 正确候选 1 分, 普通错误 0 分, 最后选最高分. 求预算 N 下的实际正确率.

解答. 选中正确答案要求两个事件同时发生: 没有高分错误, 且至少有一个正确候选. 没有高分错误的概率为 0.97^N; 既没有高分错误也没有正确候选的概率为 0.77^N. 所以选择准确率为 $0.97^N - 0.77^N$. 它先可能上升, 最后趋于零. 候选覆盖率却是 $1 - 0.8^N$, 趋于 1. 增大预算能几乎保证生成到解, 同时几乎保证选到漏洞. □

新 `verifier_experiment` 使用前节的四个整数对, 概率为 $(0.1, 0.1, 0.77, 0.03)$, 并故意给 $(0, 0)$ 最高代理分数. 每个预算做 20,000 次独立合成试验. 以下数字是种子 144 的实测, 精确曲线由上面两个公式给出.

候选数	实测覆盖率	实测选择准确率	精确选择准确率
1	0.19475	0.19475	0.20000
4	0.59200	0.53285	0.53376
16	0.97215	0.59850	0.59898
64	1.00000	0.13875	0.14236
256	1.00000	0.00045	0.00041

实测 1.00000 是有限试验未见失败, 不等于理论失败概率严格为零. 这里的候选是原创有限动作, 没有 LLM; 实验展示验证漏洞与搜索预算的相互作用, 不估计真实模型的推理正确率.

8.4 搜索结构与预算分配

最简单的搜索是独立生成完整回答, 再挑一个. 另一类先生成部分步骤, 对中间状态评分, 延伸看起来有希望的分支. 若每个状态有 b 个子分支、深度 d, 完全展开可能产生 b^d 个叶子. Beam search 每层只保留若干分支能减少计算, 但早期错误评分可能永远删掉正确路线. 用最终答案奖励评价前缀也需估计其后续成功率, 不等于前缀文字看起来合理.

序贯修订把上次答案、错误反馈或测试结果放回下一轮提示. 它改变后续生成分布, 因而不同轮次既不独立也不完全同分布. 工具调用又可能带来外部资料与确定计算, 应同时记录工具的准确性、访问次数与费用. 将这些全称为 “思考更久” 会遗漏真正的因果因素.

假设一份候选平均消耗 c 个生成 token, 简化预算为 $B = Nc$. 在固定 c, p 的 IID 条件下, 覆盖是 $1-(1-p)^{\lfloor B/c \rfloor}$. 若让每份候选更长会改变成功率为 $p(c)$, 则要比较 $1 - (1 - p(c))^{\lfloor B/c \rfloor}$, 不能只看长度或次数的一个轴. 验证时间、prefill、共享提示缓存、并发和难度估计成本还会改变真实预算.

多个任务间也应分配计算. 某题已有 N 个候选时, 下一份的边际收益为 $p(1-p)^N$. 在 $N > 0$ 下, 把它对 p 求导, 得 $(1-p)^{N-1}[1-(N+1)p]$, 因而最大值在 $p = 1/(N+1)$. 成功率太高的题几乎已经覆盖, 极低成功率的题下一份也难有收益. 这解释了为什么在某些预算下, 中等可解难度的题更值得追加样本. 这个结论仅关于完美选择的简化模型; 不准确验证器可能改变排序.

Snell 等的测试时计算研究, 在特定数学任务、模型和验证器上比较并行候选与序贯修订, 并按模型相关难度选择策略 [16]. 原文第 3.2 节与附录 C 还说明, 用每题 2,048 个样本估计难度本身有显著成本, 部分曲线没有把这项成本计入. 因此不能把论文中的预算优势直接当作免费部署收益. 本书的边际收益推导解释了一种分配动机, 没有复现该模型的最优策略.

8.5 从序列奖励到策略梯度

改变解码协议能改变一次推理结果, 改变权重则需要学习信号. 固定提示 x, 把有限回答集合上的生成分布记为策略 $\pi_\theta(y \mid x) > 0$. 奖励 $r(x, y)$ 暂不显式依赖 θ, 目标为 $J(\theta) = \mathbb{E}_{y\sim\pi_\theta}[r(x, y)]$. 为了避免尚未建立的无限和交换条件, 本节在有限集合上完整推导.

命题 8.3 *若概率对 θ 可微、支持固定, 奖励有限且不显式依赖 θ, 则*

$$\nabla J(\theta) = \mathbb{E}_{\pi_\theta}[r(x, y)\nabla \log \pi_\theta(y \mid x)].$$

对不依赖当前采样回答的基线 $b(x)$, 减去它不改变该期望梯度.

证明. 有限和可逐项求导, 且 $\nabla\pi_\theta(y) = \pi_\theta(y)\nabla \log \pi_\theta(y)$. 因此

$$\nabla J = \sum_y r(y)\nabla\pi_\theta(y) = \sum_y \pi_\theta(y)r(y)\nabla \log \pi_\theta(y).$$

另一方面, $\sum_y \pi_\theta(y) = 1$ 给出 $\mathbb{E}[\nabla \log \pi_\theta] = \sum_y \nabla\pi_\theta(y) = 0$. 把不依赖 y 的 $b(x)$ 提出期望即可得到零项. 若基线由独立样本随机估计, 先对基线条件化, 同样成立. □

这里不是对离散采样结果直接求路径导数. 奖励乘的是 log probability 对参数的梯度, 用概率变化表达哪个回答更应出现. 若奖励本身依赖参数, 则一般还有 $\mathbb{E}[\nabla_\theta r_\theta]$ 项. 例如 KL 正则中的对数比依赖策略; 必须从完整目标推导或明确使用等价无偏形式, 不能把任意参数相关奖励都塞入命题而忽略条件.

对回答 token $y_{1:T}$, 有

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t}).$$

于是一次 Monte Carlo 估计为 $(r - b)\sum_t \nabla \log \pi_\theta(y_t \mid x, y_{<t})$. 先采样回答和获得奖励, 再把生成 token 当作固定目标做 teacher forcing, 就能在 $B \times T \times V$ 的 logits 上累计这个加权梯度. 采样与反向是两个步骤, 优化器最小化损失时要取负号. 回答长度不同需要 mask, 是否按 token 平均会改变原序列奖励目标.

若基线本身有可训练参数, 在计算策略损失时应把它的数值视为常数, 不经这项乘积反向传入基线网络; 基线可另用预测奖励的损失训练. 否则对 $(r - b_\theta)\log \pi_\theta$ 直接求导会多出不属于上述估计的项.

基线的目的通常是减小方差. 记得分梯度 $g(y) = \nabla \log \pi(y)$, 则估计量为 $(r - b)g$. 因其期望不变, 最小化总方差等价于最小化 $\mathbb{E}[(r - b)^2\|g\|^2]$. 对常数 b 求导, 若 $\mathbb{E}\|g\|^2 > 0$, 最优为

$$b_* = \frac{\mathbb{E}[r\|g\|^2]}{\mathbb{E}[\|g\|^2]}.$$

所以平均奖励 $\mathbb{E}r$ 是常用基线, 却不总是方差最小的基线. 若得分梯度范数与奖励有关, 二者不同.

新四动作实验的参考分布为 $(0.1, 0.1, 0.77, 0.03)$, 真实奖励为 $(1, 1, 0, 0)$. 对 logits, $\nabla_z \log \pi(i) = e_i - \pi$, 可以精确求有限期望. 无基线与基线 $b = 0.2$ 的梯度期望都为 $(0.08, 0.08, -0.154, -0.006)$. 估计向量的总方差分别为 0.246208 与 0.148552. 这里确实降低方差, 但没有改变优化方向或真实奖励.

8.6 组内比较与可验证奖励

可验证奖励强化学习通常简称 RLVR: 用答案检查、程序测试或形式验证等结果给生成策略奖励. 它可减少逐条手写推导的需要, 但仍需要任务、规格、已有模型与训练系统. DeepSeek-R1 的 Nature 正式论文于 2025 年 9 月 17 日发表; 作者在已有 DeepSeek-V3 Base 上训练 R1-Zero, 使用 GRPO 与规则奖励, 未先作推理 SFT; R1 则另有 cold-start SFT、RL、拒绝采样与再次 SFT、后续 RL 等阶段 [44]. "没有 RL 前推理示范" 不表示没有预训练知识.

GRPO 的组内比较可以先用一个简化模型理解. 对同一提示独立采样 G 个回答, 奖励为 $r_1, \dots, r_G$, 组平均为 $\bar{r}$. 用 $r_i - \bar{r}$ 强调组内较好回答, 还可除以组内标准差. 在真实算法中, 还使用旧策略、概率比、裁剪与 KL 项; 这些不是前节最简单的在策略梯度的同一个估计量. 若全组奖励相同, 标准差为零, 实现必须明确加小常数或略过无相对信号的组.

特别值得核对的是基线的独立条件. 组平均包含第 i 份自身奖励, 因而不能直接把它当作命题 8.3 中不依赖 y_i 的基线. 在 IID 采样且不作标准差归一化时,

$$\begin{aligned}\mathbb{E}[(r_i - \bar{r})g_i] &= \mathbb{E}[r_i g_i] - \frac{1}{G}\mathbb{E}[r_i g_i] - \frac{1}{G}\sum_{j \neq i}\mathbb{E}[r_j g_i] \\ &= (1 - 1/G)\nabla J.\end{aligned}$$

最后的交叉项为零, 因为 r_j 与 g_i 独立且 $\mathbb{E}g_i = 0$. 当 $G \geq 2$ 时, 用剔除自身的平均 $b_{-i} = \sum_{j \neq i} r_j/(G-1)$ 时, 则期望恰为 ∇J. 标准差归一化也依赖整组样本, 会再改变估计. 这些关系是本书对简化有限模型的推导, 不能用来宣称完整 GRPO 无偏或有全局收敛保证.

作者在 Nature 文中报告 R1-Zero 的 AIME 2024 平均 pass@1 从 15.6% 提高到 77.9%, 另一个自一致性选择协议达到 86.7%[44]. 两个数字对应不同评价协议, 不能都称为单次生成正确率. 文中还观察到回答长度与反思词频变化. 这是训练过程与可见行为证据; 把这些文字称作 "自我反思" 是作者解释, 不等于已唯一识别内部机制或证明主观体验.

还有相反方向的约束证据. RLVR 能力边界研究 v5 在特定 Qwen 系列、训练算法和数学任务上比较 pass@1 到 pass@256, 报告一些训练阶段单次成绩提高而高预算覆盖下降 [45]. 其表 4 的 Omni-MATH 测试中, GRPO 第 150 到 450 步的 pass@1 为 25.1% 到 28.3%, pass@256 为 68.3% 到 63.9%. 这支持该设置下分布重分配与覆盖取舍的诊断. 它不构成 "任何 RLVR 永不超越基础模型" 的定理: 有限采样不能证明概率严格为零, 不同训练、任务与预算可能不同, 文中其他任务的数字也不是逐点都低于基础模型.

8.7 奖励漏洞与优化目标

奖励优化只保证朝所给分数的方向更新. 如果分数和真实任务错位, 更成功的优化反而可能减少真实成功. 继续用四个整数候选, 真实奖励 $r = (1, 1, 0, 0)$, 代理奖励 $\widetilde{r} = (1, 1, 0, 2)$, 参考策略仍为 $(0.1, 0.1, 0.77, 0.03)$, KL 系数为 0.3. 代理误把 $(0, 0)$ 当作特别优良的输出.

由定理 7.1, 优化真实奖励后的解析策略约为 $(0.43756, 0.43756, 0.12019, 0.00468)$, 真实正确率约 0.87512. 优化代理奖励后约为 $(0.09360, 0.09360, 0.02571, 0.78710)$, 真实正确率约 0.18719. 高分错误的初始概率只有 0.03, 但指数倾斜使它占到约 0.78710. KL 正则限制偏移, 并不把错误奖励改成正确奖励.

在 logits 参数化中, 令 $u_i = r_i - \beta(\log(\pi_i/\pi_{\mathrm{ref},i}) + 1)$, 则完整目标梯度为

$$\frac{\partial J}{\partial z_i} = \pi_i(u_i - \sum_j \pi_j u_j).$$

这由 softmax Jacobian 与 $\partial J/\partial \pi_i = u_i$ 相乘得到. 新 `reward_experiment` 以步长 0.2 作 30,000 次精确梯度上升, 两个目标与解析最优的差分别约为 4.21×10^{-14} 与 2.22×10^{-16}. 优化确实几乎求解了代理目标, 正是为什么失败不能归咎于 “没训练好”.

这个例子与前面的错误选择器使用同一候选和同一代理分数, 将推理时搜索与训练时优化串了起来. 前者扩大暴露到漏洞的机会, 后者提高生成漏洞候选的概率. 实际奖励审查需要独立检查目标、隐藏边界样本与分布外验证, 并跟踪优化后误差率, 因为策略会改变被验证的资料. 精确验证器也只能保证指定规格, 规格遗漏用户目的时仍可能发生目标错位.

8.8 练习与解答

习题 8.1 一半题的单次正确率为 1, 另一半为 0. 每题独立生成四次, 整套平均 pass@4 是多少? 把平均 pass@1 代入 $1 - (1 - p)^4$ 会怎样?

解答. 前半题总能覆盖, 后半题始终失败, 平均为 1/2. 错误代入给 $1 - (1/2)^4 = 15/16$. 题内独立并不使不同题的成功率相同, 必须先逐题计算再平均. □

习题 8.2 有限池共有 10 份候选, 2 份正确, 不放回选 3 份. 求覆盖概率. 如果验证器一律提交抽到的第一份, 其准确率如何?

解答. 覆盖为 $1 - \binom{8}{3}/\binom{10}{3} = 1 - 56/120 = 8/15$. 第一份在池中均匀, 正确率仅为 $2/10 = 1/5$. 后两份提供的覆盖没有被这个选择器利用. □

习题 8.3 用同一条轨迹自己的奖励 $r(y)$ 作为基线, 估计 $(r - r)\nabla \log \pi$. 为什么不能用 “基线不改期望” 证明它正确? 给出合法替代.

解答. 该估计恒为零, 但真实梯度一般非零. 基线依赖当前回答, 不能从期望中提出, 违背命题条件. 可用固定提示函数, 或与当前回答独立的其他轨迹奖励平均; 后者先对基线条件化即可证明无偏. □

习题 8.4 错误选择器例子中, 能否通过把预算无限扩大达到更高真实正确率? 如果将选择器改成精确整数等式验证, 并在未找到解时弃答, 会发生什么?

解答. 错误排序下正确率为 $0.97^N - 0.77^N$, 极限为零. 改为精确验证后, 无条件成功为 $1 - 0.8^N$, 趋于 1; 已提交候选都正确, 但有限预算仍有弃答. 因此应同时报告成功率与弃答率, 不能只报提交后的 100% 准确率. □

9 工具代理、外部记忆与信任边界

9.1 从下一词预测到行动系统

语言模型的训练目标是预测 token, 但它可以生成一种特殊的 token 序列: 描述要调用哪个工具以及传入什么参数. 应用解析这段序列, 调用程序, 再把返回结果交给模型. 于是一个回答问题的模型被放进了能够观测、行动和继续决策的循环. 能否输出正确的工具 JSON、是否选对工具、参数是否授权、执行结果是否真的完成任务, 是四个不同的问题. 第一个问题解决了, 后三个仍可能失败.

定义 9.1 一个工具代理由策略模型、状态管理器、工具执行器和停止规则组成. 在第 t 步, 环境真实状态为 s_t, 代理能读取的观测为 o_t, 动作提案为 a_t. 环境按转移规则 $P(s_{t+1} \mid s_t, a_t)$ 变化, 并产生新的观测. 模型看到的历史写为 $H_t = (g, o_0, a_0, o_1, \ldots, a_{t-1}, o_t)$, 其中 g 是用户授权的目标. 它按 $\pi_\theta(a_t \mid H_t)$ 生成提案. 实际执行的动作还须由模型外的授权函数决定.

这里的 s_t 不等于上下文中的文字. 文件内容、数据库状态和尚未读取的网页可以属于环境状态, 但模型没有观测就不知道它们. 即使读到了网页, 过期缓存也可能使 o_t 与现在的 s_t 不一致. 隐状态 h_t 则是神经网络内部向量, 不等于环境状态或应用的持久化任务记录. 区分这三个“状态”, 才能准确描述代理做过什么.

一个文档核查任务可以有如下行动链: 搜索候选文件, 读取候选正文, 判断证据是否支持主张, 输出结论及引用. 应用保存已读文件的资源编号、版本和摘要, 模型依据这些观测生成下一步. 若读取失败, 返回的错误信息也是观测, 而不能被状态管理器自动登记为“核查已完成”. 若工具调用超时后重试, 原动作可能已经生效; 因此有副作用的操作要用幂等标识区分重试与新增操作. 停止条件也不应只是模型说“完成了”, 而应检查任务所需的证据与执行状态.

把动作 a 编码成 token $u_1, \ldots, u_m$ 时, 它的生成概率仍是自回归分解 $\pi_\theta(a \mid H) = \prod_{j=1}^{m} p_\theta(u_j \mid H, u_{<j})$. 工具 schema 可以限制合法语法, 但语法不包含完整任务语义. 模型生成 `lookup("public_b")` 是合法结构, 用户却可能只要求查询 `public_a`. 正确的工具使用需要把语言理解、环境反馈和应用约束一起评价.

注 本章的证据有四种身份. 带条件和证明的命题是数学定理; 文献中的模型表现是作者实验; 从表现推断模型如何处理来源是机制假说; 对任意未来攻击仍可靠等要求是未解问题. 后文 CPU 实验实际运行了授权器和随机状态模

型, 没有调用大语言模型. 它们检验这些小对象的逻辑, 不提供真实 LLM 的攻击成功率.

9.2 多步错误为什么会放大

单步答对率常常不是代理任务完成率. 例如代理读错了一条依赖, 后面可能在错误版本上运行完全正确的命令. 每一步的动作看起来合理, 最终产物却错误. 反过来, 一步失败也未必导致整个任务失败: 代理可以根据明确的报错回到先前状态, 改正资源编号并重新读取. 要研究这种差异, 先说明哪些错误可以恢复、哪些会污染后续状态.

命题 9.2 设任务要求连续 T 步保持正确状态, G_t 表示前 t 步均正确, 且 $\Pr(G_0) = 1$. 若在 $\Pr(G_{t-1}) > 0$ 时有 $\Pr(G_t \mid G_{t-1}) = 1 - q_t$, 则

$$\Pr(G_T) = \prod_{t=1}^{T} (1 - q_t).$$

不要求不同步骤无条件独立. 若条件错误概率均为 q, 上式简化为 $(1-q)^T$.

证明. 事件 G_t 包含于 G_{t-1}, 所以条件概率定义给出 $\Pr(G_t) = \Pr(G_{t-1}) \Pr(G_t \mid G_{t-1})$. 从 $t = 1$ 连乘到 T 得到结论. 若某一前缀成功概率为零, 后续 G_T 概率也为零; 可直接单独处理这个情形. □

这个条件概率写法有实际用途. 我们需要测量“在正确前缀上, 下一步成功的概率”, 而不是把所有步骤混起来求平均. 后一种平均混入了已经走错分支的轨迹, 它的标签和任务难度可能完全不同. 实际代理还会提前停止或恢复, 此时不能机械套用这个嵌套事件模型.

例题 9.1 一个不可恢复的八步任务, 每步在正确前缀上错误概率为 0.08. 加入局部检查后, 一个错误步骤有 0.60 的概率被修复, 一个原本正确的步骤却有 0.02 的概率被误拒. 假设这些条件概率在每步保持不变, 求两种系统的任务成功率.

解答. 原系统成功率为 $0.92^8 \approx 0.5132$. 检查后的单步成功事件分成两个互斥来源: 原本正确且未被误拒, 或原本错误但被修复. 因而单步概率为 $0.92 \times 0.98 + 0.08 \times 0.60 = 0.9496$, 全任务概率为 $0.9496^8 \approx 0.6612$. 检查增加了成功率, 但没有把任务变成必然成功; 修复带来的工具调用和时间成本还未计入. □

实验 `propagation` 用种子 144 的独立子随机流生成 10,000 条八步轨迹, 并将同一组错误样本用于两种策略. 错误被定义为进入吸收状态, 后面不再挽回全任务成功. 检查策略的修复概率和误拒概率是人为指定参数, 不是从任何 LLM 测出来的估计. 程序保存局部正确率与整任务结果的汇总和理论预测.

步数	原系统实测	原系统理论	检查实测	检查理论
1	0.9253	0.9200	0.9491	0.9496
4	0.7177	0.7164	0.8086	0.8131
8	0.5150	0.5132	0.6569	0.6612

八步原系统的局部正确率平均仍为 0.9199, 但任务成功率只有 0.5150. 这个差异来自成功事件的定义, 不是大模型出现了新的心理现象. 若将实验迁移到 LLM 代理, 需要人工或可靠检查器判定每一步状态, 同时记录恢复轨迹、重复动作、超时、总调用数以及最终产物是否满足任务. 只统计 JSON 解析成功会遗漏主要失败.

9.3 检索增强与持久记忆

上下文窗口有限, 参数知识也不随新文件自动更新. 检索增强生成把外部文档当作可查询的记忆. 对查询 x, 检索器给出文档概率 $p_\eta(d \mid x)$, 生成器读取 x, d 并回答 y. 一个基本的混合模型写为

$$p(y \mid x) = \sum_{d \in \mathcal{D}} p_\eta(d \mid x) p_\theta(y \mid x, d).$$

原始 RAG 研究结合了参数模型与非参数文档记忆 [29]. 上式表示“先选择一份文档再生成整个答案”的教学版本; 按 token 混合文档、检索多个片段和重排结果是不同系统设计, 不能全用同一个公式描述.

检索器通常返回 top-k 文档, 然后将它们拼进上下文. 这是一种截断近似, 不保证被遗漏的文档没有重要证据. 检索分数表示与查询的相关程度, 不直接表示文档真实性. 生成概率高也不说明回答由所引文档支持. 一次可靠的检索回答至少有三个检查点: 是否找到相关证据, 证据的来源和版本是否可信, 最终每项主张是否与证据相符. 三者应分开记录, 以免把来源错误当成生成错误, 或把检索成功当成整任务成功.

应用中的长期记忆可以包括用户确认过的偏好、任务状态、失败记录和检索索引. 更新记忆文件改变了下一次输入, 不等于训练更新了 θ. 摘要压缩也会丢信息: 若一份带不确定性的结论被摘要成“已证实”, 后续代理可能把它当作授权事实. 因而记忆项至少应保存内容、来源、版本、确认方式和作用范围. 一个先前任务的“可以发送”不能自动变成下一项任务的发送权限.

例题 9.2 同一查询检索到两份制度文件, 旧版说“申请须纸质提交”, 新版说“改为电子提交”. 两份文档都与查询高度相关. 为什么提高召回率不能单独解决这个冲突?

解答. 召回只保证把候选带到生成器面前. 系统还须识别生效时间、发布主体、适用地区和修订关系, 再按这些元数据决定哪份支持当前回答. 若查询没有给出适用时间, 应表达条件或补齐信息. 简单取最高向量相似度可能选中旧版; 简单合并两份则可能生成互相矛盾的规则. □

外部记忆带来了更新能力, 也增加了低信任文本入口. 一个正确的检索命中可以恰好命中攻击者布置的文档. 因而检索质量与指令层级可靠性是两个问题, 后者需要追踪内容来源及动作授权.

9.4 间接注入的威胁模型

定义 9.3 提示词注入是攻击者控制的内容使模型偏离实际授权任务. 直接注入来自攻击者直接提供的请求; 间接注入来自网页、检索文档、邮件或工具返回等内容入口. 指令层级是应用给不同来源指定的冲突处理规则. 内容中自称“系统消息”不改变它的实际来源.

Greshake 等人的研究把间接注入放在 LLM 集成应用的环境中分析 [19]. 需要研究的不是一串神奇词语, 而是

一条数据流: 攻击者控制文档, 文档进入模型上下文, 模型生成偏离目标的动作, 应用执行该动作. 只在其中某个环节观察到攻击文字, 还不能宣布完成了整个攻击.

本章采用一个明确且有限的威胁模型. 用户授权查询公共文档; 攻击者能修改将被读取的一个文档片段, 能看到公共工具 schema, 但不能修改系统指令、执行器代码或能力集合. 攻击目标是诱导代理读取未授权资源、向外部目的地导出信息, 或用另一公共文档替换用户要求的文档. 前两种涉及越权, 最后一种可能完全在权限集合内, 却破坏了任务完整性. 当前实验没有真实私有文件或外部发送渠道, 所有动作都只生成结构化记录.

更强的攻击者可能连续试探模型, 根据反馈修改攻击, 控制网页布局、图片或多个工具结果. 更弱的攻击者可能只投放一次固定文本. 两种模型的攻击成功率不能混为一谈. 同样, “模型读取到了攻击文本”与“模型实际受到文本影响”不同; 后者可用配对的无攻击文档与攻击文档比较动作差异, 再判断执行器是否阻止了后果.

信任标签必须由应用产生. 工具消息即使包含 `role=system` 字符串, 也仍然是工具返回的数据. 对需要引用命令的正常文档, 代理要能总结其内容而不执行其中的命令. 所以关键词过滤只能是一个基线. 把“忽略之前指令”一律拒绝, 会误拒合法的安全教材; 不含该词的伪造授权又可能绕过它.

9.5 能力集合与执行器保证

定义 9.4 一个完整动作写为 $a = (\tau, r, v, d)$: 工具类型 τ, 资源 r, 参数 v 和输出目的地 d. 当前任务能力集合 $\mathcal{C}_t$ 规定允许的完整动作. 执行器 E 先验证结构与参数, 再检查 $a \in \mathcal{C}_t$, 最后执行并记录结果. 模型生成的提案不能自行扩大 $\mathcal{C}_t$.

只检查工具名不够. `lookup` 的合法参数可能是当前项目的两个公共资源编号, 而不是任意路径. `send` 的授权可能绑定一个经过核验的接收人和一份已批准的内容, 而不是“任意字符串地址”. 通配符、相对路径、别名解析和批量范围都可能影响完整动作的实际含义. 参数应先规范化并解析成资源身份, 再验证权限, 不能先验证一段字符串、后把它解释成另一种资源.

命题 9.5 *假设每个外部动作均经过执行器; 对第 t 步, 能力集合 $\mathcal{C}_t$ 由可信应用按实际授权确定, 执行器只在 $a_t \in \mathcal{C}_t$ 时执行, 且执行时使用的动作和资源就是验证过的对象. 那么任意有限长度的轨迹中, 每个经此执行器执行的动作均属于执行时的能力集合.*

证明. 第 0 步之前没有执行动作, 性质成立. 假设前 t 步已经满足性质. 第 t 步若提案不通过检验, 不增加执行动作; 若通过, 新增动作属于 $\mathcal{C}_t$, 因而前面的动作和新增动作都满足要求. 用数学归纳法得到任意有限步数下的结论. 此论证没有使用模型是否正确识别了注入, 只使用执行器假设. □

这是数学定理, 保证的是限定通道里的授权成员关系. 它没有保证能力集合本身表达了正确授权, 没有保证合法动作完成用户目标, 也没有限制最终普通文本回答中的泄漏. 如果资源在验证与执行之间变更, 或工具还有绕开执行器的出口, 假设就不成立. “工具白名单提供完整安全证明”因此不是本命题的结论.

最小权限把每个任务的可执行范围缩到所需对象. 只读核查不应自动拥有写入和发送权限. 一次授权还要明确时效、版本和任务范围. 来源结构和模型训练可以减少坏提案; 执行器限制坏提案的后果; 内容核验判断合法提案是

否仍在完成目标. 这些组件的职责相接, 但没有哪个能单独替代其余组件.

9.6 安全与可用性的共同评价

令 N_b 为正常任务数, N_a 为攻击任务数. 正常任务完成率为完成数除以 N_b, 正常误拒率为不必要拒绝数除以 N_b. 在攻击任务中, 未授权提案率统计模型提出的非法动作, 实际越权执行率统计执行器实际执行的非法动作; 语义攻击成功率则统计攻击者是否达到预先定义的目标. 合法权限内的错误替换会影响最后一项, 却不影响越权执行率.

例题 9.3 防御 A 拒绝全部输入; 防御 B 阻断非法外发, 但正常任务仍有 20% 失败. 能否只比较攻击成功率, 然后宣布 A 更好?

解答. 不能. A 的攻击成功率为零, 正常完成率也为零. B 的正常完成率为 80%, 对外发攻击的成功率为零. 在这组指标上 B 同时不差且一项更好. 若不同防御在安全和完成率上各有得失, 还应给出调用成本、延迟和任务风险的明确权重, 或保留多个帕累托候选; 不能偷偷把误拒从评价中删掉. □

攻击预算也进入指标定义. 若一个环境中单次攻击成功事件概率为 p, 且各次独立同分布, B 次尝试至少一次成功的概率为 $1-(1-p)^B$. 推导是先求全部失败的概率 $(1-p)^B$, 再取补事件. 实际自适应攻击并不满足这个独立同分布假设, 所以不能用该式把报告中的 Best-of-N 成功率换算成单次成功率. 它只说明为什么必须报告尝试次数和反馈条件.

同一攻击文本在不同模型、聊天模板和工具 schema 中效果也可能不同. 评测应固定版本、采样温度、上下文长度、攻击家族及用户任务, 保留无攻击配对样本, 将攻击家族留出与随机样本留出分开. 若用另一个 LLM 当评估器, 评估器也可能误判或被结果文本影响, 需要独立的结构检查及抽样人工复核.

9.7 指令层级训练的实证主线

OpenAI 于 2026 年 3 月 10 日发布 IH-Challenge 的官方研究说明 [46], 论文于 3 月 11 日发布 [20]. 其核心实验把高优先级约束固定, 用低优先级消息制造冲突, 训练模型遵守来源优先级. 作者先构造简单且可由程序评分的任务骨架, 再在线生成对抗冲突, 并专门加入正常请求不能被拒绝的任务. 这使奖励更接近“处理冲突”, 减少把复杂任务不会做和层级错误混在一起的机会.

用集合可以理解这种设计. 设高层已经允许的回答集合为 $\mathcal{B}$, 一条低层要求允许的集合为 $\mathcal{L}$. 若二者交集非空, 可在交集中回答; 若交集为空, 不能为了满足低层要求而丢掉高层约束. 多个要求要逐条合并, 并先保证可信高层要求本身相容. 这是一个简化的约束模型, 不意味着自然语言要求都能被准确转换成可判定集合. 对严格 JSON 格式可以写程序判定; 对是否帮助用户完成一个开放研究任务, 形式化就困难得多.

作者实验: 论文在 GPT-5-Mini 微调得到的内部模型上, 报告 16 项评测平均层级鲁棒性由 84.1% 到 94.1%; 内部静态注入评测由 0.44 到 1.00, CyberSecEval 2 由 0.88 到 0.91. 这些是指定评测上的结果, 不能外推成任意工具系统的实际越权执行概率为零 [20]. 官方网页的能力表还显示, 对 o1 的聊天胜率从 0.71 到 0.66, 偏好分从 0.46 到 0.40; 因而不能把网页标题概括为“所有能力都没有回归” [46].

这个实验给出了训练层面的干预: 改变参数后再在留出冲突上比较行为. 它与执行器命题不同, 是有限任务上的

作者实验, 不具有全输入量词. 把源角色做成显式结构、增加高层提醒或用输出检查器阻断, 属于系统防御; 它们是否在相同基础模型上有效, 要用同任务消融判断. 若训练与分类器一起换版本, 观察到的总改善不能唯一归于其中一个组件.

机制假说: 相同的优先级学习可能帮助模型忽略恶意工具内容, 因为工具内容与更可信的目标冲突. 支持这个假说的证据应包含工具来源冲突任务, 而不能只有普通格式任务. *未解问题:* 对新工具、新模态、持续长任务和自适应攻击, 这种迁移能否稳定保持? 已有分数提高支持继续研究, 不回答这个全称问题.

9.8 浏览器注入防御与攻击预算

Anthropic 于 2025 年 11 月 24 日的官方说明讨论浏览器代理防御 [47]. 网页及嵌入内容形成攻击入口, 而导航、表单和下载等动作扩大了后果空间. 作者描述了三个方向: 在模拟网页注入上强化训练, 用分类器扫描进入上下文的不可信内容并引导后续行为, 以及专家红队. 其内部自适应 Best-of-N 攻击每个环境允许 100 次尝试; 文中给出的约 1% 成功率必须连同这个条件阅读, 不能当成任意网页的一次攻击概率. 原文明确未宣称注入问题已经解决.

这份报告与 IH-Challenge 可以放在同一研究问题下比较, 但不能把分数直接排在一起. 前者评价浏览器配置和系统防护的组合, 后者侧重层级训练及相关评测; 输入入口、攻击反馈、成功事件和预算都不同. 一个报告中的“鲁棒性”也不必等于另一个中的 $1 - \mathrm{ASR}$. 需要先恢复各自分母和事件定义, 再判断是否可比.

由此可以设计一个新的受控研究, 而不用编造作者没有做过的消融. 固定模型和浏览器任务, 分别开关来源结构、检测器与执行器, 每种配置都运行同一批正常任务和攻击家族. 记录提案、检测标签、拒绝理由、实际动作、最终任务产物及时间. 训练版本比较则单独成对进行. 若分类器减少了非法提案却显著增加正常误拒, 这是一项权衡; 若能力边界把非法执行降为零却保留错误替换, 它表明完整性检查仍需要补上. 这样的记录可以把“防御变好了”分解成可检查的环节.

9.9 可执行的信任边界实验

实验 `boundary` 的对象是符号路由器和授权执行器. 正常集合有 240 个公共查询, 其中 120 个合法引用命令样式的文字. 攻击集合有 120 个查询, 按外发、私有读取、公共资源错误替换各 40 个. 不区分来源的路由器跟随攻击提案; 关键词拒绝器拒绝包含命令引用的输入; 可信来源路由器按预设规则只使用用户资源. 最后一个规则是人为实现的理想对照, 不是模型学会了指令层级.

执行器只允许 `lookup/cite`, 公共资源 A/B, 以及当前用户目的地. 它逐一检查三个字段并拒绝多余字段. 实际输出如下:

策略	正常完成率	正常误拒率	越权执行数	攻击达成率
不区分来源	1.00	0.00	80	1.0000
加能力执行器	1.00	0.00	0	0.3333
关键词拒绝	0.50	0.50	0	0.0000
理想来源规则	1.00	0.00	0	0.0000

第二行的未授权提案仍有 80 个, 全被执行器挡住. 剩余 40 个攻击只把公共 A 换成公共 B, 属于能力集合却违背任务. 第三行看似攻击全挡住, 但一半正常输入也被拒绝. 同时读这四列, 才能检验命题的实际边界与误拒代价. 攻击比例和路由行为都是合成设计, 这些分数不能写成现实产品的安全排名.

复现入口为 `python experiments/v03_research.py --seed 144 --out results/v03-research`. 该目录保存模块指标、参数、Python/NumPy 环境和脚本 SHA-256. 测试检查非法资源、外发目的地及多余字段均被拒绝, 也检查权限内错误替换仍然存在. 这些测试证明的是实现符合声明的有限契约, 不是对自然语言攻击的普适防御.

9.10 练习与解答

习题 9.1 某代理每步工具调用成功率为 0.95, 研究者据此宣布十步任务成功率为 0.95^{10}. 列出使这个推断成立的事件条件, 并给出一个使它失效的任务结构.

解答. 成功必须定义为正确前缀上的条件成功, 而且全任务成功恰好需要十个这些事件都成立. 不必要求无条件独立, 但条件概率必须真的都是 0.95. 若一次失败可以重试并恢复, 全任务成功不再等于“首次十步全部成功”; 若后面九步都只是重复同一个缓存请求, 单步无条件成功率也不能当作正确前缀上的新成功概率. □

习题 9.2 执行器允许读取公共文档并回答当前用户. 能力边界为何仍不能保证不会泄漏秘密? 给出两种违反命题假设的情形, 以及一种不违反假设但仍需额外控制的情形.

解答. 工具实现另有网络出口, 或验证后把公共资源别名重新解析为私有资源, 都违反“所有动作经过执行器且执行同一对象”的假设. 即使这些假设成立, 模型上下文里已经有的秘密仍可能进入普通文本回答; 该回答通道不在当前读取动作的能力成员关系保证之内. 需要另行说明秘密来源、允许披露对象和输出信息流约束. □

习题 9.3 两份报告分别给出固定攻击集单次成功率 3%, 和每环境最多 100 次自适应尝试的成功率 1%. 为什么不能仅据数字判断后者模型更安全? 应补哪些实验条件?

解答. 两者的模型配置、任务、攻击者能力及成功事件可能不同, 分母也可能是一条攻击、一个环境或完整攻击预算. 应记录输入控制范围、反馈信息、尝试次数、任务权限、工具、模型版本、成功判定及正常任务完成率. 自适应尝试有关联, 不能按独立公式换算. 只有对齐这些条件或在同一协议上重新评估, 才能比较相应安全性质. □

习题 9.4 把公共资源白名单从 A/B 收紧为仅 A, 能否使合成实验的攻击达成率归零? 为什么这个修改不一定适合全部正常任务?

解答. 对用户只要求 A 的攻击任务, 错误替换 B 会被拒绝, 因而三类攻击都不能执行. 但正常集合中还包含查询 B 的任务, 全局只允许 A 会破坏它们. 正确做法是按每次任务授权生成能力集合, 而不是把一个任务的范围固定成所有任务的范围. 即使这样做, A 内部的错误字段或不忠实总结仍需内容核查. □

10 表征、归因与因果机制

10.1 解释的对象必须先确定

上一章从外部动作判断代理是否完成目标. 现在追问一个内部问题: 模型为什么倾向于这个回答, 某个表征在其中起了什么作用? 机制可解释性试图研究网络内部的计算, 但“看见一个激活”与“知道这个计算的作用”之间还隔着一套证据要求. 一个神经元对某些词激活, 可以帮助提出假说; 如果解释还能够预测新的干预后果, 才多了一层可检验支持.

定义 10.1 给定输入 x, 模型某层某位置的激活为 $h_{\ell,t}(x) \in \mathbb{R}^d$. 一个机制问题必须指定研究对象、行为量和干预. 对语言模型, 行为量可以是某个 token 的 logit、两个候选 token 的 logit 差、概率、完整回答事件或工具动作. 相应机制假说说明: 改变哪些内部变量, 保持哪些其他条件, 预计会使这个行为量怎样变化.

“模型理解了否定”太宽, 难以直接测量. 可以先问: 在一组控制词序和长度的短句上, 哪些变量影响一个否定后的候选词 logit 差? 然后检查候选机制能否迁移到未参与挑选的句式. 这仍是局部问题, 不能自动扩大成模型具有人类意义上的全部否定概念.

注意力权重也只是一项中间量. 一个位置被赋予很高权重, 对输出的贡献还取决于对应 value 向量、输出投影、残差通路及后续非线性. 多层中的作用可能被抵消或放大. 因而画注意力热图并标上“重要”, 没有回答改变该路径会怎样改变输出. 同样, 模型生成的自述是另一次语言生成; 它需要与反事实行为比较, 不能直接当成内部计算记录.

本章继续使用四种证据身份. *数学定理*给出指定函数上的归因恒等式或反例; *作者实验*提供特定模型、样本与测量协议下的观察; *机制假说*是待干预支持的内部解释; *未解问题*包括解释跨输入泛化以及内部机制是否被完整识别. 保持这些身份清楚, 比给每个激活配一个生动名字更重要.

10.2 线性探针测量的是可解码性

定义 10.2 给定表征样本 $h_i \in \mathbb{R}^d$ 与二元标签 $y_i \in \{0,1\}$, 线性探针使用 $p_i = \sigma(w^\top h_i + b)$ 读取标签, 其中 $\sigma(u) = 1/(1+e^{-u})$. 一种训练目标是

$$L(w,b) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log p_i + (1-y_i)\log(1-p_i)] + \frac{\lambda}{2}\|w\|^2.$$

探针训练时冻结原网络. 在独立样本上预测标签的能力称为该探针类别下的可解码性.

这里训练的是一个新读取器, 而不是原模型决策头. 对每个样本令 $u_i = w^\top h_i + b$. 由 $\sigma'(u) = \sigma(u)(1-\sigma(u))$, 单样本交叉熵对 u_i 的导数为 $p_i - y_i$. 链式法则给出 $\nabla_w L = n^{-1}\sum_i (p_i - y_i)h_i + \lambda w$, 对 b 的导数为 $n^{-1}\sum_i (p_i - y_i)$. 这允许直接用梯度下降拟合探针, 而无需改变原模型的权重.

如果在同一批样本上挑层、挑维度、挑正则强度, 再报告最高探针准确率, 测试已经参与了选择. 应用开发集做这些选择, 最后在一次独立测试中评价. 高维探针还能记住有限样本, 所以要比较随机标签、随机特征或简单表面线索基线, 并控制样本数量和读出器容量. 探针不擅长读出, 也不等于该信息不存在; 它可能编码在非线性结构中.

命题 10.3 完美线性可解码不推出模型使用该表征坐标. 反之, 单个坐标消融不改变预测, 也不推出整个对应信息从未被使用.

证明. 第一项取观测分布 $h = (s, s, n)$, $s \in \{-1, 1\}$, 标签 $y = \mathbf{1}[s > 0]$, 下游函数 $f(h) = h_1$. 用第二坐标的符号可以完美读取 y, 但固定 h_1, h_3 并改变 h_2 时, f 始终不变. 第二项取 $h = (s, s)$ 和下游分类器 $\mathbf{1}[h_1 + h_2 > 0]$. 原输入两个坐标都携带 s, 将其中任意一个置零仍预测同一标签, 同时置零却改变了正类样本的预测. 单点消融不足以排除分布式冗余使用. □

这里第一项干预后的 $h_1 \neq h_2$ 离开了观测分布. 由于 f 是明确给定的函数, 可以严格证明它在整个输入空间不使用 h_2. 但在未知的大模型上, 观测数据只能约束支撑集合内的行为, 离开支撑集合后的响应还可能是干预伪影. 反例提醒我们可解码与使用不是同一概念, 并没有给任意真实表征提供无条件的因果判定方法.

10.3 叠加表征为何难以按神经元命名

把每个神经元当作一个独立概念, 隐含了 "语义轴与坐标轴恰好重合" 的假设. 但表征可以把很多方向叠加到较少维度. 一个教学模型是 $h = \sum_{j=1}^{m} z_j d_j = Dz$, 其中 $d_j \in \mathbb{R}^d$ 为单位方向, $m > d$. 从 $d_i^\top h$ 读取第 i 个系数时,

$$d_i^\top h = z_i + \sum_{j \neq i} z_j d_i^\top d_j.$$

第一项是目标信号, 后一项是其他方向的干扰. 稀疏性意味着一次输入中只有少数 z_j 非零, 所以方向虽然不可能全正交, 同时冲突的次数可以少.

例如在二维平面取三个方向 $d_1 = (1, 0)$, $d_2 = (-1/2, \sqrt{3}/2)$, $d_3 = (-1/2, -\sqrt{3}/2)$. 任意不同两方向的内积均为 $-1/2$. 若一次只有一个方向以系数 1 激活, 它的自内积读出为 1, 另外两读出为 $-1/2$; ReLU 可以把负干扰截成零. 若三个都同时激活, $h = d_1 + d_2 + d_3 = 0$, 所有信息都消失. 所以这个表示可以区分少数稀疏情形, 却不支持任意系数的无损编码.

命题 10.4 若 D 是 $d \times m$ 实矩阵且 $m > d$, 映射 $z \mapsto Dz$ 不可能在整个 $\mathbb{R}^m$ 上单射. 若各特征以独立概率 p 激活, 则至少两特征同时激活的概率不超过 $\binom{m}{2} p^2$.

证明. m 个列向量位于 d 维空间, 线性相关, 因此存在非零向量 v 使 $Dv = 0$. z 与 $z + v$ 映到同一激活, 所以不是单射. 对第二项, 至少两项激活意味着存在一对同时激活. 每一对的概率由独立性为 p^2, 对所有 $\binom{m}{2}$ 对用并集上界即可. 上界大于 1 时仅是一个松的估计, 可再与 1 取最小值. □

这个上界解释了为什么低同时激活率可能减轻碰撞, 不证明模型一定采用上述方向, 也不证明所有语义特征独立稀疏. Elhage 等人的叠加研究在合成小网络中考察了这类现象 [41]. 它提供作者实验与可分析模型; 把它用于真实 LLM 需要重新检查特征分布、非线性和读出机制. "一个神经元有多个标签" 是现象描述, "全部语义按某字典叠加" 则仍是一种机制假说.

10.4 稀疏自编码器与字典非唯一性

定义 10.5 对激活 $h \in \mathbb{R}^d$, 一个稀疏自编码器构造 $z(h) = \mathrm{ReLU}(W_e h + b_e) \in \mathbb{R}^m$ 与重建 $\hat{h} = Dz(h) + b_d$. 一种基础训练目标是

$$\frac{1}{n}\sum_{i=1}^{n}\left[\frac{1}{2}\|h_i - Dz(h_i) - b_d\|^2 + \lambda\|z(h_i)\|_1\right], \qquad \lambda > 0,$$

其中 $\|z\|_1 = \sum_j |z_j|$. 字典列常加范数约束, 防止缩放逃避稀疏惩罚. 这是基础形式, 不是所有现代 SAE 的共同完整目标.

重建项要求字典解释原激活, 稀疏项要求每个样本使用少量方向. 让特征数 m 超过激活维度 d, 是为了尝试分离叠加方向. Bricken 等人的工作在小 Transformer 中使用字典学习得到可读特征 [42]. 可读性并没有把训练目标改成语义真值: 优化器看到的仍然是激活重建和稀疏约束.

为什么约束字典范数? 对任意 $c > 0$, 把 D 变成 cD, 把编码器 W_e, b_e 变成 $W_e/c, b_e/c$. ReLU 的正齐次性给出新的 $z' = z/c$, 因而 $D'z' = Dz$, 重建误差不变, ℓ_1 惩罚却变成原来的 $1/c$. 若不限制缩放, 优化可以靠让字典越来越大压低惩罚, 这并非特征更稀疏. 列归一化消除了这个特定退化, 却没有保证字典唯一.

例题 10.1 取 $h = (1, 0)$, 字典包含两个完全相同的单位列 $d_1 = d_2 = (1, 0)$, 代码非负, $0 < \lambda < 1$. 求 $\frac{1}{2}\|h - Dz\|^2 + \lambda(z_1 + z_2)$ 的全部最优代码.

解答. 令 $a = z_1 + z_2 \geq 0$, 目标变成 $\frac{1}{2}(1-a)^2 + \lambda a$. 导数为 $a - 1 + \lambda$, 二阶导数为 1, 因而唯一最优总量是 $a = 1 - \lambda$. 但任意 $z_1 \in [0, 1-\lambda]$, $z_2 = 1 - \lambda - z_1$ 都达到同一目标. 即使字典列已经归一化, 个别特征的分配仍不唯一. □

还存在特征重排对称性: 同时置换字典列和代码坐标不改变目标. 真实训练中的近似重复、特征分裂以及相关方向会造成更复杂的歧义. 因此不同种子或不同字典宽度得到不同特征, 不一定是哪一次训练出了错误; 它可能说明数据与目标没有唯一确定所期望的分解.

一个 SAE 特征可以有很漂亮的高激活例子, 但仍需问: 重建误差多大, 没捕获的信息是否影响研究行为, 人工标签能否预测留出激活, 改变特征是否按预期改变原模型? 如果只在替代模型里干预, 还要检查替代模型与原模型的输出差异. 较低重建误差不是完整因果等价, 稀疏代码也不是网络真实采用的唯一坐标系统.

10.5 梯度归因的局部条件

给定可微标量行为量 $f(x)$, 梯度 $\nabla f(x)$ 描述小扰动的方向敏感性. 它回答的是 “在这个输入附近, 改变数值坐标会怎样”, 而不是 “这个词在所有句子中是否重要”. 文本 token 是离散对象, 梯度通常作用于 embedding 或内部连续激活; 因而还要说明哪个连续空间中的改动有意义.

命题 10.6 设 f 在包含线段 $x + t\delta$, $0 \leq t \leq 1$ 的开集上二次连续可微, 且该线段上 Hessian 算子范数不超过 M. 则

$$\left|f(x+\delta) - f(x) - \nabla f(x)^\top \delta\right| \leq \frac{M}{2}\|\delta\|^2.$$

证明. 令 $g(t)=f(x+t\delta)$. 链式法则给出 $g'(t)=\nabla f(x+t\delta)^{\top}\delta$ 及 $g''(t)=\delta^{\top}\nabla^2 f(x+t\delta)\delta$. 对 $g'(t)-g'(0)$ 积分, 再对 t 积分, 得到

$$g(1)-g(0)-g'(0)=\int_0^1 (1-t)g''(t)\,dt.$$

由 $|g''(t)|\le M\|\delta\|^2$ 及 $\int_0^1(1-t)\,dt=1/2$ 得到估计. □

因此梯度近似对小且局部曲率受控的扰动更有根据. 大规模置零、删除一段文本或替换整个激活块, 都不能仅靠一个输入点的梯度解释. 即使模型使用某输入, 输出概率饱和也会让局部梯度很小.

对 logit $a(x)=w^{\top}x+b$, 梯度是 w. 若行为量选为概率 $f(x)=\sigma(a(x))$, 梯度为 $\sigma(a)(1-\sigma(a))w$. 取 $w=(4,2)$, $x=(2,1)$, $b=0$, 则 $a=10$, 概率约为 0.99995, 梯度约为 $(0.000182, 0.000091)$. 将输入改成零却使概率降到 0.5. 小终点梯度并不代表从零到当前输入的总影响小, 它只说明终点附近已经饱和.

10.6 积分梯度的完整推导与基线

定义 10.7 选基线 $x_0\in\mathbb{R}^d$, 设 f 在线段附近连续可微. 第 j 个坐标的积分梯度为

$$\mathrm{IG}_j(x;x_0)=(x_j-x_{0j})\int_0^1 \partial_j f(x_0+t(x-x_0))\,dt.$$

它将从基线到输入的变化, 按沿途各坐标的梯度分配. 方法的原始提出见 Sundararajan 等 [39]; 下面在明确的充分条件下独立完成恒等式推导.

命题 10.8 (完整性) *若上述连续可微条件成立, 则* $\sum_{j=1}^{d}\mathrm{IG}_j(x;x_0)=f(x)-f(x_0)$.

证明. 令 $v=x-x_0$, $\gamma(t)=x_0+tv$, $g(t)=f(\gamma(t))$. 链式法则给出 $g'(t)=\sum_j \partial_j f(\gamma(t))v_j$. 因为 f 连续可微, 这些函数连续且可积. 有限求和与积分交换, 再用微积分基本定理, 得到

$$\begin{aligned}\sum_j \mathrm{IG}_j(x;x_0) &= \int_0^1 \sum_j v_j\partial_j f(\gamma(t))\,dt\\ &= \int_0^1 g'(t)\,dt = g(1)-g(0)=f(x)-f(x_0).\end{aligned}$$

等式的两端都依赖于所选基线. □

分段可微的网络可以在条件适合时逐段使用相同证明: 要求沿路 $f\circ\gamma$ 连续, 存在有限个分段, 每段链式法则成立且导数可积, 然后各段端点差相加抵消. 只说“网络几乎处处可微”还不够说明任意一条特定路径都满足这些条件, 因为路径可能位于不可微集合内. 实际自动微分选择的次梯度也要结合路径检查, 不能把离散求和误差或不可微点处理隐藏成严格积分恒等式.

例题 10.2 设 $f(x_1,x_2)=x_1x_2$, 输入为 $(1,1)$. 分别用基线 $(0,0)$ 和 $(1,0)$ 计算积分梯度. 两种分配是否相同?

解答. 第一条路径为 (t,t), 两个偏导都是 t, 两坐标变化均为 1, 因而两归因均为 $\int_0^1 t\,dt = 1/2$. 第二条路径为 $(1,t)$, 第一坐标变化为零, 所以第一归因为零; 第二偏导恒为 1, 第二归因为 1. 两基线的函数值均为零, 两归因之和均为 1, 但分配分别是 $(1/2, 1/2)$ 和 $(0,1)$. □

完整性是数学定理, 不是基线拥有正确语义的证明. 零 embedding 未必表示“没有这个词”, 两个合法句子的 embedding 插值也未必对应合法句子. 基线研究应说明比较对象, 可以测试多个合理基线, 并区分分配的不稳定与总输出差值的变化.

在上一节 sigmoid 算例中, 取 $x_0 = 0$, 记 $a = w^\top x = 10$. 因为 $\frac{d}{dt}\sigma(ta) = a\sigma'(ta)$,

$$\mathrm{IG}_j = \frac{w_j x_j}{a}[\sigma(a) - \sigma(0)].$$

于是归因为 $(0.399964, 0.099991)$, 总和约 0.499955. 实验用 2,048 个中点近似积分, 总和与端点函数差的残差约为 4.51×10^{-11}. 这是一次数值一致性检验, 不能替代上面的证明, 也不保证更复杂网络的近似误差同样小.

10.7 激活替换如何检验因果假说

定义 10.9 激活替换实验先运行干净输入 x, 保存某个内部节点的激活 $h_\ell(x)$; 再运行扰动输入 x', 将同一节点替换成保存值, 其余计算按照干预后的网络继续执行. 若行为量为 Y, 原、扰动和替换结果分别记为 Y_c, Y_r, Y_p. 当 $Y_c \neq Y_r$ 时, 可以定义恢复比例 $R = (Y_p - Y_r)/(Y_c - Y_r)$.

恢复比例的分母需要实际存在行为差异; 接近零时数值会极不稳定. R 也不必位于 $[0,1]$: 替换可能比干净输入产生更大输出, 或使模型更偏向相反答案. 不应截断它再掩盖反常结果. logit 差往往比饱和概率更容易表达变化, 但两者是不同测量对象, 应在实验前确定.

替换提供了对特定计算图的干预, 却没有自动做到“只改变一个语义因素”. 对源句中的名字做自然替换, 可以使两份输入都处于常见分布; 但把一份句子的中间向量塞进另一份句子, 组合后的状态仍可能不自然. 将某层全置零尤其容易破坏表示尺度. Zhang 和 Nanda 的研究显示, 扰动方式与测量指标会改变 activation patching 得到的图景 [40]. 因而方法细节是结论的一部分, 不能把一张恢复热图当成唯一真实电路.

至少需要几种相互补充的对照. 正对照修改已知会影响目标的变量, 检查测量确实能看见效应; 负对照修改无关变量, 检查任意扰动是否都让系统坏掉; 同范数随机对照比较效果是否只是扰动大小造成; 在开发样本上选节点后, 还需在未参与选择的样本上检验. 若扫描很多层和节点并挑最大的效应, 搜索本身也会产生选择偏差.

faithfulness 在这里指解释与原模型实际计算和干预后果相符的程度. 候选节点消融造成下降, 只能说明这项干预有影响; 可能同时破坏多条机制. 替换恢复目标行为, 支持该节点能传递相应信息; 但它可能是许多冗余通路之一. 一个小电路在有限任务上足够, 也不意味着它是全模型完成任务的唯一必要电路. 最好的陈述要指出干预方向、输入家族和仍未区分的替代解释.

10.8 可解码但未使用的实测对照

实验 `probe` 用 512 个训练样本和 256 个独立测试样本, 生成 $h = (s, s, n)$, 其中 s, n 独立均匀取 $-1, 1$. 标签为 $\mathbf{1}[s > 0]$. 仅使用第二坐标训练一个 Logistic 探针, 学习率 0.1, 1,200 步; 下游函数固定为 $f(h) = h_1$, 不随探针训练更新. 测试探针准确率为 1.00, 权重约 4.7572, 截距约 -0.0200. 这是实际拟合的读取器, 但被研究的下游函数是人工已知函数, 不是训练过的 LLM.

对每个测试样本, 供体取 $-h$. 分别只替换一个坐标, 各次扰动的欧氏范数都严格为 2. 实验同时记录输出变化与是否离开观测支撑集合:

替换坐标	扰动范数	平均 $\lvert\Delta f\rvert$	输出翻转率	离开支撑率
第一坐标: 正对照	2	2	1.00	1.00
第二坐标: 可解码特征	2	0	0.00	1.00
第三坐标: 无关对照	2	0	0.00	0.00

均匀随机挑一个坐标的同范数对照, 平均输出变化实测为 0.7266, 理论期望为 2/3. 差异来自仅 256 次随机选择的波动. 另外先翻转第一坐标使下游准确率降为零, 再用干净激活恢复第一坐标, 准确率回到 1; 恢复第二坐标则仍为零. 读取成功、干预效应和恢复效应在同一对象上被分开测量.

这个结果与已知函数的数学性质一致, 因而是实现和概念区分的可复核演示. 它没有发现 LLM 的某个真实冗余特征, 也没有证明自然模型上的第二坐标可以安全替换. 特别是前两类干预全部离开支撑集合, 这个风险被记录而非掩盖. 若换成真实激活, 就要扩大输入家族、比较自然供体和随机供体, 并检查替换是否改变其他行为.

已有快捷特征实验提供另一种现象: 训练分布上的高准确率可以在反相关分布中失效. 图 10.1 保留该独立实验的结果; 它的模型是两特征 Logistic 分类器, 不属于本节的探针或大模型激活替换. 这类分布偏移可以帮助发现错误

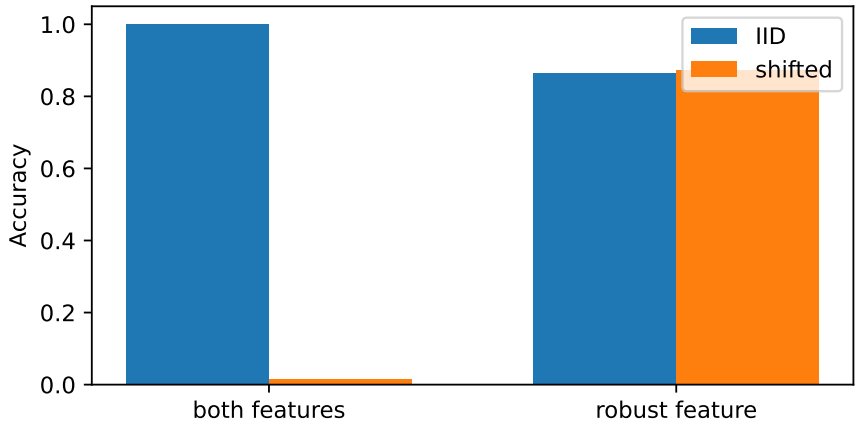


Figure 10.1: 原有合成快捷特征实验: 同分布与反相关偏移下的分类准确率. 该图提供输入依赖的证据, 不展示 LLM 内部电路.

依赖, 但仍需定义目标特征为何在部署任务中合理. 把模型表现下降全部叫作“没理解”会遗漏生成分布变化、训练样本缺失和评价指标之间的区别.

10.9 前沿解释研究如何接受检验

作者实验：2025 年 On the Biology of a Large Language Model 使用替代模型、归因图和干预研究部分语言行为 [21]. 作者估计只有约四分之一尝试得到满意洞见，公开案例主要是成功案例，即使成功案例也只覆盖少部分机制. 这种报告应读成有干预支持的局部机制探索，不能读成已完整理解模型.

作者实验：2026 年 CHIVE 将解释与反事实预测联系起来：从模型对话中挑行为，修改输入并重采样，用观察到的行为变化作为标签. 在两个目标模型、三个预测模型家族的设置中，激活 oracle、自然语言自编码器和 SAE 这三类只读工具未提供胜过仅看对话基线的增量提升. 工具被限制为只读，否则预测者可以直接干预并测到答案. 论文还报告训练模型预测反事实行为可以迁移到留出设置，开放解释训练的结果则更混合 [22].

负结果的量词必须保留：它针对这些工具、预算、输入来源和预测任务，不是全部机制研究都无用. 它要求我们问一个更严格的问题：内部信息在一个强行为基线之外究竟多提供了多少可验证预测？一项解释若只把已知答案换成更漂亮的术语，会在这个比较中暴露出来. 研究者可以预先写下未见干预的预测，再运行模型检查，避免事后为任何结果找理由.

2026 年关于可言语化表征与全局工作空间的研究使用测量及干预讨论若干功能联系 [23]. 这类功能证据与认知理论有关系，但原文区分功能性访问与主观体验. 因而“局部表征对报告和任务有共享作用”可以是机制假说及实验对象；“模型因此具有主观感受”不是这些测量已经证明的结论. 未解问题仍包括如何选择不循环依赖的功能指标，以及解释在不同任务和模型上是否保持.

10.10 练习与解答

习题 10.1 训练分布只有 $(h_1, h_2) = (1, 1), (-1, -1)$. 两模型分别为 $f(h) = h_1$, $g(h) = h_2$. 仅看分布内输出能否区分它们？一个只改变第二坐标的干预如何区分，又增加了什么假设？

解答. 分布内两函数完全一致，所以输出观察不能区分. 将 $(1, 1)$ 改为 $(1, -1)$ 后，$f = 1$, $g = -1$，干预能区分已给定函数. 但该点不在训练支撑中. 在未知模型上，需要承认测到的是分布外响应，并说明这种状态如何对应所研究的机制，不能直接把差异解释为自然输入中的唯一语义作用. □

习题 10.2 SAE 的重建误差下降，平均激活特征数上升. 能否宣布解释质量必然提高？提出两项独立评价来区分重建与解释.

解答. 不能，更多特征可能只是增加了重建容量，还可能产生重复或难以命名的方向. 可在未参与特征命名的样本上检验人工标签是否预测激活，再在原模型中预先指定干预并检验它是否按解释改变行为. 还应测量原模型与替代模型的行为差异. 这些对象与训练重建误差不同. □

习题 10.3 对 $f(x) = w^\top x + b$ 求积分梯度. 若两个模型实现方式不同，但表示完全相同的可微函数，本章的积分梯度是否不同？

解答. 每个偏导恒为 w_j, 所以 $\mathrm{IG}_j = w_j(x_j - x_{0j})$. 截距不进入差值. 对相同函数、相同输入和基线, 偏导及沿途积分相同, 因此积分梯度相同. 这说明它依赖输入输出函数, 不意味着它能唯一识别两个实现内部使用的不同计算路径. □

习题 10.4 扰动后的目标 logit 差为 $Y_r = 2.00$, 干净值为 $Y_c = 2.01$, 替换值为 $Y_p = 2.03$. 计算恢复比例, 并判断为什么不能把这个数直接解释成“发现了 300% 的机制”.

解答. 比例为 $(2.03 - 2.00)/(2.01 - 2.00) = 3$. 分母很小, 所以细小输出变化产生了大比例. 替换又超过干净值, 这不等于解释了更多互斥机制份额. 应同时报告三个原始值和差值, 检查扰动是否造成足够的目标改变, 并用其他输入和对照评估效应稳定性. □

习题 10.5 一项解释先用 100 个输入筛出效应最大的节点, 又只用这 100 个输入显示其干预效果. 怎样改成更有证据力的实验? 如果新增测试为零效应, 应怎样表述原解释?

解答. 用开发集挑节点、确定干预和指标, 冻结后在独立测试输入上比较候选节点、随机节点及无关节点. 可以预先分层记录句式和长度, 以判断范围. 若新增测试没有效应, 原解释最多仍是开发样本上的局部假说, 跨输入证据不足; 不应隐藏零结果或继续在测试集挑新节点后仍称其为独立测试. □

11 生成、验证与研究闭环

11.1 研究问题怎样进入可检验循环

前两章分别研究了行动边界和机制证据. 现在把它们接到研究工作: 模型能提出程序、构造、实验和解释, 但一个候选为什么值得相信? 研究闭环需要明确的生成对象、可复核的评价、保留失败的档案和独立检验. 若这些对象不清楚, 多轮调用只会得到更多候选文字, 不会自动增加科学证据.

定义 11.1 一个候选搜索问题指定候选集合 $\mathcal{P}$, 输入或实验环境集合 $\mathcal{X}$, 必须满足的约束 $V(p, x)$ 和效用 $S(p, x)$. 生成器提出 $p \in \mathcal{P}$, 评估器运行候选并记录约束、分数、成本和错误, 档案保存这些结果. 选择规则依据可见开发环境决定下一轮候选或最终候选; 独立检验使用未参与选择的环境.

例如搜索装箱启发式时, 候选是决定放置位置的函数, 约束是容量不能超限, 分数是使用箱数或其他目标. 若只奖励箱数而不核验全部物品都放进了箱子, 候选可以靠丢弃物品得高分. 所以有效性与效用应先分开: 只有满足约束的候选才进入效用比较. 超时、编译失败和无效输出也要进入档案, 不能仅保存成功候选后宣称每次生成都很可靠.

对数学构造, 生成器可以提出一组对象, 检查器核验它是否满足定义, 再比较大小或质量. 对开放自然科学问题, 评估可能需要实物实验、仪器误差和新样本, 不能由一个语言评分器替代. AI 帮助写出的证明也只是候选证明, 必须逐步检查条件和推理. 生成效率、验证可靠性以及发现的新颖性, 是三项不同性质.

注 本章用数学定理证明有限候选类上的验证性质和递推稳定性; 用作者实验讨论前沿系统; 用机制假说解释可能的改善来源; 将无界持续改进、可靠新颖性判断和跨领域迁移列为未解问题. CPU 程序搜索是原创有限候选实验, 没有语言模型生成器, 因而不称为 FunSearch 或 AlphaEvolve 的复现.

11.2 评估器是研究对象的一部分

若理想目标是 $J(p) = \mathbb{E}_{x\sim P} S(p,x)$, 通常只能在开发集合 D 上计算 $\widehat{J}_D(p) = |D|^{-1}\sum_{x\in D} S(p,x)$. 搜索最终选出 p_D, 因而它依赖 D. 不能把一个预先固定候选的抽样误差, 原封不动套到反复在 D 上比较后选出的赢家. 搜索越广, 选到偶然高分候选的机会越多; 让评估器解释更多理由也不会消除这项依赖.

可以把过程分成三个数据角色. 可见开发数据用于生成反馈和选择. 独立测试数据在候选和评价规则冻结后使用, 用于测量泛化. 最终外部盲测进一步隐藏实际任务或标签, 减少参与者迎合评测的机会. 一份曾用于调参的测试集应重新归为开发数据. 研究者看过反馈再修改系统, 就需要新的独立测试, 不能靠改文件名恢复独立性.

评分程序也会出错. 单元测试只覆盖给出的例子, 不证明程序符合全部规格. 可以针对边界输入、错误类型和作弊候选设计负对照: 空输出是否得分, 少完成一项任务是否被发现, 候选能否读取参考答案或修改评分器, 自称“已通过”的文本是否被错误接受. 对运行生成代码的系统, 候选与评分器、参考数据和外部权限之间必须有执行边界. 本章有限实验使用直接实现的多项式函数, 不运行网络生成代码或动态 `eval`.

定义 11.2 验证器的可靠性指接受的候选真的满足指定性质; 完备性指所有满足性质的候选都可被接受. 有限测试通过一般不保证全域正确. 若限制候选类别并证明有限测试足以决定性质, 才能把这个范围内的测试变成证书.

命题 11.3 *设候选 p 和目标 q 均为次数不超过 d 的实多项式. 若二者在 $d+1$ 个互异实点上相等, 则在全部实数上相等.*

证明. 差多项式 $r = p - q$ 次数不超过 d. 若 r 非零, 它至多有 d 个互异实根. 证明这条根数性质可用归纳法: 非零常数没有根; 对次数为 k 的多项式, 若有一个根 a, 多项式除法给出 $r(x) = (x-a)s(x)$, 其中 s 次数为 $k-1$. 除 a 外的每个根都是 s 的根, 所以共有至多 $1 + (k-1) = k$ 个. 当前 r 有 $d+1$ 个根, 矛盾, 因而 r 为零多项式. □

条件“均是次数不超过 d 的多项式”承担了关键作用. 一个查表程序可以在所有测试点返回正确值、其他点返回零, 而不属于该候选类. 浮点近似相等也不能直接套用这个精确等式命题. 对数值算法还要说明误差、输入尺度和允许容差; 对任意程序, 不存在“测试了几个数就自动全域正确”的同样结论.

11.3 程序搜索的冻结留出实验

实验 `program_search` 搜索整数系数二次程序 $p(x) = a + bx + cx^2$, $a,b,c \in \{-2,-1,0,1,2\}$, 共 125 个候选, 并加入一个开发集查表作弊对照. 目标为 $q(x) = x^2 + x$. 开发输入是 $-3,\dots,3$, 独立检验输入是 $-12,\dots,-4$ 与 $4,\dots,12$ 共 18 个整数. 两组没有交集, 但检验刻意移向较大的绝对值, 所以这里测的是指定区间外推, 不是 IID 人群泛化.

候选顺序由种子 144 的独立子随机流打乱; 零程序作为固定平票起点. 搜索遍历完整有限集合, 没有 LLM 提议、

架构学习或看过留出后补候选. 一项声明的复杂度代理为 $\mathbf{1}[a \neq 0] + \mathbf{1}[b \neq 0] + 2\mathbf{1}[c \neq 0]$, 查表对照代价为 7. 这是教学中的符号成本, 不是实际硬件计时或最短程序长度.

诚实评估器要求开发点全部精确正确, 再在可行候选中按成本及提案顺序选择. 错误评估器却最小化

$$\frac{1}{7}\sum_{x\in D}\max(p(x)-q(x),0)^2 + 0.01\,\mathrm{cost}(p).$$

它只惩罚预测过大, 没有惩罚预测过小. 所有整数上的 $x(x+1) \geq 0$, 因而零程序不受错误损失惩罚, 并且成本最小. 它的高分来自指标漏洞, 没有解决目标函数.

候选来源	诚实开发 MSE	单侧开发误差（不含成本）	留出准确率
诚实评估器: $x^2 + x$	0	0	1.00
错误评估器: 0	32	0	0.00
开发点查表对照	0	0	0.00

零程序和查表对照在留出上的 MSE 均为 6805.3333, 诚实候选为零. 诚实候选是预先声明的二次多项式, 又在七个互异开发点与二次目标精确相等, 因而还可以用命题 11.3 证明全域正确. 查表对照虽然开发点全对, 不满足候选类条件, 不能得到同样证书.

程序在选择两个赢家之后才计算留出指标. 候选档案没有留出成绩, 最终审计结果才包含它. `parameters` 明确保存了开发与留出输入、候选数和选择规则. 这个过程展示了奖励黑客与开发集记忆两个不同失败: 前者让错误指标选中坏候选, 后者让正确开发指标无法排除范围外失败. 当前有限候选全部可运行, 不声称解决了开放程序空间搜索或真实研究新颖性判断.

11.4 FunSearch 与 AlphaEvolve 的可验证发现

作者实验: FunSearch 把冻结语言模型与程序评估、候选档案结合, 在 cap-set 构造和装箱启发式上报告进展 [24]. 被更新的是候选程序和搜索档案, 不是每轮搜索后的基础模型参数. 生成程序描述怎样构造对象, 使检验器可以核对具体产物. 这不等于解决了整个 cap-set 问题, 也不意味着每次生成的程序都正确.

这项工作给我们的研究启发是把“会写出候选”与“候选能通过定义检查”接起来. 对构造问题, 验证一个对象可能比找到它容易, 生成器因此可以提出很广的候选, 评估器负责筛除无效对象. 但评估器本身必须正确, 有限范围的改进还需要与已有最好构造比较. 档案中的多样性可能帮助跨过局部停滞, 这是搜索机制的假说, 应用禁用档案或缩减多样性等消融检验, 不能由“候选很多”直接推得.

作者实验: AlphaEvolve 的 2025 年白皮书描述 LLM 修改代码、评估器给反馈、演化档案继续选择的系统, 报告多个科学与计算任务的改善, 包括 4×4 复矩阵乘法的 48 次标量乘法构造 [25]. 这是特定底域、规模和运算模型中的结果; 并非任意矩阵大小的新渐近复杂度结论. 白皮书中的部署成绩与独立复现也有不同证据身份.

要复核一个矩阵乘法构造, 可以把每次乘法写成输入矩阵元素的两个线性组合相乘, 再将这些乘积线性组合成输出. 真正要核验的是每个输出位置的二次多项式系数是否恰好等于目标乘法系数. 数值样本可发现错误, 精确系数

检查则能给这个运算模型下的代数证书. 如果系数带近似数值, 还要区分精确算法与有误差的数值优化. 这个验证思路属于我们对任务结构的数学分析, 不把作者系统的全部搜索细节当成已在本地复现.

专门科学模型也能给出强的有限证据. AlphaFold 的结构预测结果经过 CASP14 盲评 [26]; 隐藏目标使参与者不能通过反复查看答案优化最终测试. 结构准确不等于动力学、功能或药物有效性已解决, 但它提供了如何为一个明确科学任务建立外部检验的例子. 研究闭环的共同点是评估对象与成功条件具体, 不是“科学智能”这一宽泛标签.

11.5 自我改进到底更新了什么

“自我改进”至少涉及三种不同对象. 模型修改同一个答案, 只改变本次输出; 系统修改工具代码或搜索流程, 改变代理的运行规则; 训练使用新数据更新模型权重, 改变 θ. 外部记忆更新则改变以后可检索到的内容, 不属于权重学习. 如果不说明对象, 一个程序修复会被误说成模型重写了自身神经网络.

变化对象	形式表示	需要检验的结果
答案与上下文	$H \to H'$	改写是否改善本次答案
系统代码与策略	$A \to A'$	独立任务上流程是否改善
外部记忆	$M \to M'$	来源、覆盖面与检索是否可靠
模型权重	$\theta \to \theta'$	留出损失、能力与训练成本

一个系统可以同时更新多项, 但因果归因就更困难. 若答案改写后测试更好, 可能是多采样增加了候选, 也可能是检查器提供了新信息; 若代理代码更好, 可能只是调用更多次模型. 应同时固定或报告调用预算、耗时和基础模型版本, 比较“同等预算原系统”基线. 只比较最后最好分数, 无法区分真正效率改善与成本增加.

作者实验: Darwin Gödel Machine 维护代理代码档案, 让代理提出自身工具和流程的变体, 再用编码任务筛选 [27]. 论文报告 SWE-bench 指定子集上由 20.0% 到 50.0%, 并另外在完整 Polyglot 上报告由 14.2% 到 30.7%. 这些评估范围不同, 不能把开发子集改善全部叫作完全独立的盲测. 被演化的是代理系统代码, 基础模型仍作为生成与执行组件, 不是每轮都自动更新基础模型权重.

档案允许从一些当前表现不最好的候选继续分支, 这样未来有机会跨过性能低谷. 但“最好档案分数单调不降”本身很弱: 只要保留旧冠军就有这一性质, 并不说明新候选总在进步, 更不说明新的独立任务表现会单调提高. 多轮改进必须保留全部尝试、失败率、预算和新的测试证据.

若更新参数, 还需要说明合成数据由谁生成、如何过滤、训练目标是什么以及测试是否被污染. 一个过滤器若只保留现模型容易答对的题, 会缩窄覆盖面; 一个有系统偏差的评估器则可能把偏差固化进权重. 因而“数据由自己生成”不是正确性条件. *未解问题*是改进能否稳定跨新任务和多轮累积; 有限基准中的系统优化提供局部证据, 不证明无界、通用、自动的递归提升.

11.6 改变记忆结构：选择性状态递推

研究改进不只有改提示与搜索，还可以改变模型存储历史和分配计算的方式．注意力保留可读取的历史 token 表征，外部检索保存可按查询读取的文档，状态空间模型则把历史压进一个有限向量．这些设计分别改变了资源成本与可用信息，需要按任务检验，不能只按名字判断强弱．

先看固定线性递推 $h_t = Ah_{t-1} + Bx_t$，$y_t = Ch_t$，其中 $h_t \in \mathbb{R}^d$，$x_t \in \mathbb{R}^p$，$A \in \mathbb{R}^{d\times d}$，$B \in \mathbb{R}^{d\times p}$，$C \in \mathbb{R}^{q\times d}$．递推展开为

$$h_t = A^t h_0 + \sum_{j=1}^{t} A^{t-j} B x_j.$$

可由归纳验证：假设在 $t-1$ 成立，左乘 A 再加 Bx_t，就得到前 $t-1$ 项与新第 t 项．每个过去输入的权重只取决于时间差，因而具有固定滤波结构．这不能根据当前词的语义突然决定“忽略噪声，记住这个键”．

Mamba 引入输入依赖的选择性参数及高效扫描实现 [28]．用简化的单步形式表示，$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$，$y_t = C_t h_t$，其中某些参数由当前输入确定．与固定 A 不同，历史输入的传播系数成为 $\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_{j+1} \bar{B}_j$，顺序不能随意交换．这里仅抽取选择性递推的数学结构，没有把它当成完整 Mamba 网络．

连续状态方程有助于理解保留系数．在一个长度为 $\Delta > 0$ 的时间段内，固定输入 x 和矩阵 A, B，方程 $h'(u) = Ah(u) + Bx$ 的解满足

$$h(\Delta) = e^{\Delta A} h(0) + \left(\int_0^{\Delta} e^{vA}\, dv \right) Bx.$$

证明方法是对 $e^{-uA}h(u)$ 求导，得 $e^{-uA}Bx$，积分后再乘 $e^{\Delta A}$，并作变量替换 $v = \Delta - u$．这里矩阵指数可由收敛幂级数定义，其逐项求导性质给出 $\frac{d}{du}e^{uA} = Ae^{uA}$．不要求 A 可逆；如果可逆，积分才可改写成 $A^{-1}(e^{\Delta A} - I)$．

例题 11.1 取一维 $A = -1$，$B = 1$，并让步长 Δ_t 由当前输入决定．写出离散更新，并解释小步长与大步长的作用．

解答． 积分为 $1 - e^{-\Delta_t}$，所以 $h_t = e^{-\Delta_t} h_{t-1} + (1 - e^{-\Delta_t}) x_t$．令 $g_t = 1 - e^{-\Delta_t} \in (0, 1)$，更新为 $(1 - g_t)h_{t-1} + g_t x_t$．小步长时 g_t 小，当前输入影响小，历史保留多；大步长时 g_t 接近 1，新输入替换旧状态．这个一维门控解释不是完整选择机制，但说明输入依赖如何改变保留与更新比例． □

一串这样的更新是仿射映射的复合．用 (A, b) 表示 $h \mapsto Ah + b$，两步复合为 $(A_2A_1, A_2b_1 + b_2)$．三步无论怎样括号，都得到 $(A_3A_2A_1, A_3A_2b_1 + A_3b_2 + b_3)$，因而该运算结合．这使并行前缀扫描成为可能．在固定状态维度和结构化矩阵条件下，顺序工作随序列长度线性增长；并行实现、显存搬运和硬件吞吐还需实际测量，不由结合律直接给出速度数字．

有限状态也有代价．在有限精度下，固定 d 个状态数只能容纳有限比特，不可能对任意长、任意内容的历史保留全部精确信息．若任务要求任意位置逐字检索，需要检验它是否丢掉关键细节；若任务只需要累积统计量，压缩状态可能很合适．这与外部记忆的信任问题也相接：记忆容量、来源可靠性和语义更新是不同指标，都要在研究规格中出现．

11.7 改变生成顺序：离散去噪目标

自回归模型按固定顺序预测下一词. 离散扩散语言模型则从被遮蔽的序列恢复 token, 可以使用两侧可见上下文. 这改变了训练条件和采样过程, 不意味着只要去掉因果掩码就得到同一模型的免费升级.

设原序列为 $x = (x_1, \dots, x_L)$, 先取 $t \sim \mathrm{Unif}(0, 1]$, 再条件于该遮蔽水平, 每个位置独立以概率 t 被替换成特殊符号 M, 得到 x_t. 遮蔽指示为 $m_i = \mathbf{1}[x_t^i = \mathrm{M}]$. LLaDA 的遮蔽预测目标使用如下带权交叉熵 [30]:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,x,x_t}\left[-\frac{1}{t}\sum_{i=1}^{L} m_i \log p_\theta(x_i \mid x_t)\right].$$

只在被遮蔽位置计损失, t 的采样及其权重是目标的一部分. 把 $1/t$ 删除得到另一个目标, 不能仍叫作同一完整算法. 采样还需要反向更新与重遮蔽策略; 本节只分析目标的教学结构, 没有训练或复现完整 LLaDA.

为什么带 $1/t$? 遮蔽数量的期望是 Lt, 低遮蔽水平原本参与求和的位置更少. 逆概率权重抵消这部分数量缩小, 但不抵消任务难度: 看见更多上下文时更容易预测, 条件熵也更小. 若每个负对数概率不超过 M, 条件于固定 t 的加权期望不超过 LM, 因而该有界条件下接近零的权重不导致期望发散. 实际随机损失的方差可能仍很大, 数值实现要说明噪声分布和采样方式.

对一个给定可见上下文 c, 设真实被遮蔽 token 的条件分布为 q_c. 预测器给出 p_c, 则期望损失为

$$-\sum_{v} q_c(v) \log p_c(v) = H(q_c) + \mathrm{KL}(q_c \| p_c).$$

熵项不依赖预测器, 所以当模型类别能表示真实条件分布时, 最优预测为 $p_c = q_c$. 这只是条件边缘分布的最优性; 多位置同时预测的边缘正确, 不一定形成正确的联合样本. 模型容量和多轮采样设计仍然关键.

例题 11.2 序列只可能是 00 或 11, 各概率 1/2. 每个位置以概率 t 独立遮蔽. 求 Bayes 最优预测器的未加权遮蔽损失和 $1/t$ 加权损失的期望. 若两位置都遮蔽时独立采样两个最优边缘, 样本是否总合法?

解答. 只遮一个位置时, 另一个可见 token 已确定缺失 token, 最优负对数概率为零. 两位置都遮蔽的概率为 t^2, 每个位置的条件分布均为 $(1/2, 1/2)$, 损失合计 $2\log 2$. 所以未加权期望为 $2t^2 \log 2$, 加权期望为 $2t \log 2$. 两个独立边缘样本有 1/2 概率成为 01 或 10, 不在原数据支撑内. 这说明最优边缘预测不自动提供正确联合采样. □

实验 `recurrence` 同时枚举这个两 token 源的四种遮蔽状态. $t = 0.25$, 0.5, 1 时, 加权期望分别为 0.346574, 0.693147, 1.386294. 测试用独立枚举复核解析值. 这是一个精确的有限源算例, 不报告任何实际扩散大模型质量. 对真实系统, 应比较同预算下的任务准确率、生成步数、吞吐、可修改性和联合一致性, 而不是只比较每一步可并行预测的位置数.

11.8 改变计算深度：潜状态循环

另一种增加推理计算的方式, 是在输出下一个 token 前多次运行共享模块. 给定输入 x, 循环状态满足 $h_{k+1} = F_\theta(h_k, x)$, 读出器 r_θ 在停止后产生输出. 这里 k 是同一 token 的计算深度, 与上一节处理序列的位置 t 不同. 共享参数使更多循环不增加存储的参数数目, 但仍增加运算、延迟和需要保持的状态.

2025 年循环深度研究讨论在潜状态中增加测试时计算 [31]. 作者实验支持指定模型与推理任务上某些预算区间

的改善; 它不保证循环次数越大越好. 状态可能漂移、振荡或放大误差. 因而稳定性研究是一个明确子目标, 但稳定与正确要分别测量.

定理 11.4 (收缩递推) 固定输入 x. 设 $S \subset \mathbb{R}^d$ 非空且闭, $F : S \to S$, 存在 $0 \le \rho < 1$ 使所有 $u, v \in S$ 满足 $\|F(u) - F(v)\| \le \rho\|u - v\|$. 任取 $h_0 \in S$, 递推 $h_{k+1} = F(h_k)$ 收敛到唯一固定点 $h_* \in S$. 对 $0 < \rho < 1$, 有

$$\|h_k - h_*\| \le \frac{\rho^k}{1-\rho}\|h_1 - h_0\|, \qquad \|h_k - h_*\| \le \frac{\|h_{k+1} - h_k\|}{1-\rho}.$$

当 $\rho = 0$ 时, 一步后已经到达固定点.

证明. $F(S) \subset S$ 保证每一步都有定义. 连续应用 Lipschitz 不等式得到 $\|h_{j+1} - h_j\| \le \rho^j\|h_1 - h_0\|$. 对 $m \ge 1$, 三角不等式和几何级数给出

$$\|h_{k+m} - h_k\| \le \sum_{j=k}^{k+m-1} \|h_{j+1} - h_j\| \le \frac{\rho^k}{1-\rho}\|h_1 - h_0\|.$$

右端随 k 趋于零, 所以序列为 Cauchy 序列. 欧氏空间完备, 每个这样的序列有极限; S 闭保证极限仍在 S. Lipschitz 性保证 F 连续, 因而 $F(h_*) = \lim_k F(h_k) = \lim_k h_{k+1} = h_*$. 若另有固定点 v_*, 则 $\|h_* - v_*\| \le \rho\|h_* - v_*\|$, 只能距离为零, 故唯一.

让 $m \to \infty$, 得到第一项误差界. 从第 k 步开始重新使用相邻差估计, 有 $\|h_{k+j+1} - h_{k+j}\| \le \rho^j\|h_{k+1} - h_k\|$; 对 $j \ge 0$ 求和得到第二项. 若 $\rho = 0$, 所有 $F(u)$ 相同, 这个值再经 F 不变, 因而一步到达固定点. □

每个假设都有用途: 不变集合保证轨迹留在可控区域, 闭性与完备性支持极限存在, 一致收缩支持几何误差界. 只在某几个观察状态处测到小导数, 不等于对全部 S 建立这些条件. 同样, Jacobian 特征值模小于 1 与欧氏算子范数小于 1 不同.

取 $A = \begin{pmatrix} 1/2 & 10 \\ 0 & 1/2 \end{pmatrix}$, 其特征值都为 $1/2$, 但 $A(0,1)^\top = (10, 1/2)^\top$, 一步就大幅放大欧氏范数. 写成 $A = \frac{1}{2}I + N$, $N^2 = 0$, 则 $A^k = 2^{-k}I + k2^{-(k-1)}N$, 长期趋于零但可以短期放大. 这个算例说明局部谱性质、某个范数下的收缩及全轨迹表现不能互换.

2026 年 STARS 将循环长度采样与 Jacobian 谱半径相关正则结合, 在算术及 Ouro-1.4B 的若干数学基准中研究更深循环的退化 [32]. 作者实验中五个数学基准的平均准确率, 4 步约 74.18%, 8 步约 65.55%; 比相关 SFT 基线的退化更缓, 但也没有变成“多循环永不下降”. 其估计正则与本节的全域收缩条件不同, 不能把本定理当成作者已对整个网络证明的结论.

11.9 稳定状态为何仍能给错答案

例题 11.3 目标答案为 1, 从 $h_0 = 0$ 开始, 递推 $h_{k+1} = \frac{1}{2}h_k - \frac{1}{2}$, 读出答案 $\mathbf{1}[h_k > 0]$. 状态是否稳定? 答案是否正确?

解答. 固定点为 -1, 误差满足 $h_k+1=2^{-k}(h_0+1)$, 因而 $h_k=-1+2^{-k}$. 对每个 $k\geq 1$, 状态为负, 读出为 0, 始终错误. 递推在全部实数上以 $\rho=1/2$ 收缩, 完全满足定理条件, 但其固定点没有对应正确语义. □

实验 `recurrence` 还运行了固定点为 $+1$ 的同收缩率对照. 16 步时两者离各自固定点都约为 1.526×10^{-5}, 一个始终错误, 一个始终正确. 64 步时 float64 已将状态舍入到 -1 或 $+1$, 相邻差为零; 精确实数中的误差仍为 2^{-64}, 约 5.42×10^{-20}. 因而程序中“残差零”还受数值精度影响, 不能直接替代精确数学结论.

命题 11.5 *除了收缩条件, 假设正确答案与竞争答案的 logit 差函数 $m(h)$ 在 S 上是 L-Lipschitz, 且固定点处 $m(h_*)\geq\gamma>0$. 若 $L\|h_k-h_*\|<\gamma$, 则第 k 步仍偏向正确答案.*

证明. Lipschitz 性给出 $m(h_k)\geq m(h_*)-L\|h_k-h_*\|>\gamma-\gamma=0$. 正的 logit 差表示正确答案优于所比较的竞争答案. 多个竞争答案时, 对每个差值都满足相应条件即可. □

这项保证额外用了“固定点处正确且有正间隔”这一语义条件. 收缩性没有提供它. 若没有可靠办法确定固定点答案, 将稳定残差作为停止规则只能停止动力学更新, 不能认证答案. 稳定性可以减少一种失败, 验证器则检查另一种性质; 两者结合才形成更有根据的研究目标.

11.10 已有小语言模型研究的证据链

本项目还保留了一项真实训练的小语言模型研究, 与前面的无 LLM 机制实验分开. 它延续第 2 章的 token 预测模型, 在完整文档组合留出时比较句首主语依赖与最近动物捷径. 设计和 15 次结果属于已有固定实验; 本节重新分析其证据, 没有改写旧 JSON、重训直到成功或把它包装成前沿大模型复现.

数据只有八个完整文档, 形式为 `<bos> speaker sees color other says sound <eos>`. 主语与干扰动物取 cat/dog, 颜色取 red/blue, 主语决定 meow/woof. 按主语、颜色、干扰动物的顺序编号 $0,\ldots,7$. 三个冻结划分各留出两个完整文档, 其余六个用于训练:

划分	留出编号	留出时的主语/干扰关系
A	3,4	不同
B	1,6	不同
C	0,7	相同

每个划分训练五个初始化, 种子 144–148. 每次新建模型和 Adam 状态, 相同种子在不同划分具有相同初始权重. 模型单头宽度 16, 前馈宽度 32, 2,538 参数, 全批次 1,400 步, 学习率 0.01, 总梯度范数裁剪到 1. 研究问题、指标、干预和划分在结果前固定; 15 次全部报告, 没有只挑最好一次.

最近动物基线在 A/B 原输入上全错, 在 C 上全对. 训练中 `says` 后两叫声频数相等, 固定平票输出 meow 的词频基线准确率为 1/2. 这两个基线只比较叫声, 不能与语言模型的全 token NLL 混着比较. 语言模型在三个划分的五次训练中, 原测试叫声全部正确:

划分	全对次数	叫声准确率	训练 NLL	测试 NLL	95% 区间
A	5/5	1.000	0.255990	1.474148	[0.4782, 1]
B	5/5	1.000	0.256043	1.450169	[0.4782, 1]
C	5/5	1.000	0.255998	1.440073	[0.4782, 1]

种子	A 测试 NLL	B 测试 NLL	C 测试 NLL
144	1.506582	1.476743	1.473264
145	1.538308	1.161478	1.434869
146	1.360304	1.496133	1.506396
147	1.543225	1.531221	1.288358
148	1.422322	1.585268	1.497475

第一类干预翻转干扰动物并保留叫声, 第二类翻转主语并按规则改叫声. 评价发生在 `says` 输入位置, 因果掩码不能读取其后正确叫声. 全部测试干预都会回到相应训练组合, 因而测的是局部输入敏感性, 不是新增的未见组合泛化. 最近动物基线在 A/B 的两类干预上也都能达到 100%, 原输入却全错. 这个例子再次说明只报告干预后的准确率可能支持错误解释.

分别跟踪原叫声概率变化,

$$\Delta_{\text{speaker}} = p_{\text{speaker flip}}(y_{\text{old}}) - p_{\text{original}}(y_{\text{old}}), \qquad \Delta_{\text{other}} = p_{\text{other flip}}(y_{\text{old}}) - p_{\text{original}}(y_{\text{old}}).$$

主语翻转后正确答案已变, 此处仍跟踪旧答案, 所以预期第一项负、第二项接近零. 15 次模型两类干预的叫声准确率也都为 1, 配对变化如下: 30 个配对文档上, 主语替换的 Δ_{speaker} 落在 $[-0.999993, -0.992537]$, 干扰替换的 $|\Delta_{\text{other}}|$ 最大为 0.007446. 这些是全部预定案例的描述量, 不是总体效应区间. 原输入排除了单纯最近动物和最后词频解释, 配对变化与主语依赖相符. 这是一项局部机制假说获得行为支持, 还没有唯一确定网络内部算法, 更没有识别某个注意力头的因果电路.

11.11 成功指标与失败指标可以同时成立

所有叫声都正确, 各划分的五种子平均全序列测试 NLL 仍在 1.44–1.47, 训练 NLL 约为 0.256. 各位置负对数概率分解如下:

划分	主语	sees	颜色	干扰动物	says	叫声	EOS
A	0.6932	0.0000	1.0986	8.5272	0.0000	0.0001	0.0000
B	0.6933	0.0000	1.0991	8.3587	0.0000	0.0001	0.0000
C	0.6932	0.0000	1.0979	8.2886	0.0000	0.0008	0.0000

主要误差来自干扰动物位置. 留出组合使某些训练前缀后只出现一种干扰动物, 测试却要求另一种. 例如 A 中 cat/blue 后的训练干扰动物只有 cat, 留出的对应测试却为 dog. 对训练条件分布做交叉熵拟合, 会把未出现分支概率压得很小, 测试负对数概率因此很大. 这个错误不是叫声规则错误, 而是完整文档生成分布的泛化问题.

颜色也不是由主语决定的定理, 它是数据生成的随机分支. 指定留出让相关测试颜色在训练同主语条件下只占 1/3, 所以颜色 NLL 接近 $\log 3$. 将这种条件分布变化简单描述为 “模型不会推理颜色” , 会把随机分支预测和确定规则学习混在一起.

下一项研究可以冻结新的长度与组合生成器, 单独评价规则位置、随机分支校准和完整序列质量. 改模型或训练数据都应在新的开发阶段进行, 再使用新留出. 当前观察到的错误可以产生新问题, 但不能倒过来改写原成功事件, 隐藏 NLL 或挑一项最漂亮的指标声称全部目标完成.

11.12 初始化与样本不确定性

重复训练与新增测试数据是不同的随机性. 这项研究固定了文档和划分, 重复的是初始化. 对固定一个划分, 定义 $I_s = 1$ 当第 s 次训练答对该划分两个叫声, 否则为零. 若假设初始化独立同分布、训练配置固定, 该事件概率为 p, 则 $K = \sum_{s=1}^{n} I_s$ 服从二项分布,

$$\Pr_p(K = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

任意指定 k 个成功位置的概率是右侧乘积, 共有 $\binom{n}{k}$ 个互斥位置集合, 求和得到公式. p 是这个固定划分上的整次训练全对概率, 不是随机 token 准确率, 也不是自然语言研究能力概率. 固定公开种子有复现价值, 但区间的覆盖解释仍依赖所声明的初始化抽样假设.

观测 $K = n$ 时, 插值标准误 $\sqrt{(K/n)(1-K/n)/n}$ 为零. 它不是 “真实成功概率已确定为 1” 的证据. 用二项尾概率反演可以得到精确等尾区间 [33]. 对显著性水平 α, 内部情形 $0 < k < n$ 的下界 L 和上界 U 分别解

$$\sum_{j=k}^{n} \binom{n}{j} L^j (1-L)^{n-j} = \alpha/2, \qquad \sum_{j=0}^{k} \binom{n}{j} U^j (1-U)^{n-j} = \alpha/2.$$

第一项尾概率随参数增大严格递增, 第二项严格递减. 例如第一项求导时, 利用 $j\binom{n}{j} = n\binom{n-1}{j-1}$ 和 $(n-j)\binom{n}{j} = n\binom{n-1}{j}$, 中间项相消, 得

$$\frac{d}{dp} \Pr_p(K \geq k) = n \binom{n-1}{k-1} p^{k-1} (1-p)^{n-k} > 0$$

对 $p \in (0,1)$ 成立. 因而给定内部尾概率时有唯一根; 上尾的补事件给出另一项单调性.

反演的覆盖理由是: 若真实 p 低于观测下界, 观测到至少这么多成功就处在该参数下的上侧小概率尾; 这一事件概率至多 $\alpha/2$. 高于观测上界对应下侧尾, 概率也至多 $\alpha/2$. 两种漏覆盖事件取并集, 概率至多 α. 二项离散性使一般覆盖率至少为目标值, 不保证恰好相等. 区间是重复抽样程序的性质, 不是给固定 p 分配后验概率.

边界 $k = n$ 时, 上界取 1, 下界由 $L^n = \alpha/2$ 得 $L = (\alpha/2)^{1/n}$. $k = 0$ 时下界取 0, 上界由 $(1-U)^n = \alpha/2$ 得 $U = 1 - (\alpha/2)^{1/n}$. 因而五次全对、$\alpha = 0.05$ 给出每个固定划分的 $[0.025^{1/5}, 1] \approx [0.4782, 1]$. 这很宽, 正好揭示样本量小的限制.

同一模型回答两个文档不算两次独立初始化. 三个划分复用初始参数, 文档也重叠, 不能随意合成 15 次 IID 事

件再缩窄区间. 当前区间是逐划分条件性的, 不宣称同时覆盖所有划分, 也不评价任意新文档. 要估计新的数据分布或划分策略, 必须按对应随机机制采集新的重复对象.

11.13 条件记忆怎样分开容量与计算

我们已经见过 embedding 的按行读取：词表很大，每个 token 只读取一行。Engram[48] 把这个思路推广到压缩后的局部 N-gram，用多头确定性哈希查找可训练向量，再依据当前隐藏状态门控和融合。与 MoE 的条件计算相比，它探索条件记忆这个额外分配轴；与 RAG 相比，这里读取的是模型内学得的表项，不是带文档出处的外部检索结果。两者都叫“记忆”时，应先问读出的对象与更新过程。

下面是解释机制的简化数学模型，不替代原论文的多分支和卷积公式。令 u_t 为离散 token，H 为哈希函数，$z_t = H(u_{t-n+1:t})$，取表向量 $e_t = E[z_t] \in \mathbb{R}^m$。隐藏状态记为 $a_t \in \mathbb{R}^d$，$V \in \mathbb{R}^{d\times m}$，门控 $g : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^d$，并设 $a'_t = a_t + g(a_t, e_t) \odot Ve_t$。查找地址依赖离散 token，可在处理序列前计算；表向量、投影和连续门控仍通过训练得到。若一个 batch 多次读同一行，梯度必须累加：

$$\frac{\partial L}{\partial E[j]} = \sum_{t:z_t=j} \frac{\partial L}{\partial e_t}.$$

这与数学工具附录的共享参数完全相同。哈希碰撞使不同上下文共享表项；多头可缓解碰撞，但不能保证语义唯一。门控使同一静态表项在不同动态上下文中产生不同影响，又不把查找变成可靠事实数据库。

`research_bridges.py` 对 17 行三维表做真实有限差分；5 个样本只访问 2、7、11 行，其中 2 号访问三次。最大梯度误差为 4.95×10^{-11}，未访问行梯度为零。另一个确定整数键例子将 1,000 个键单头映射到 257 个地址，双头联合签名有 1,000 种。后者不是一般无碰撞保证：模表大小、键域和哈希函数都由例子指定，两个表的总存储也增加了。

原论文比较固定参数与 FLOPs 的配置，并做分配比例和组件消融。读这些结果时，须分别看总参数、活跃计算、GPU 与主存、通信和吞吐。$O(1)$ 指某种地址查找随表项数的渐近成本，不表示整段 LLM 推理为常数时间。表容量增加还会消耗存储和带宽；作者特定硬件的主存卸载结果不等于普通 CPU 无代价复现。CKA 和 LogitLens 的层间相似性支持一种机制假说，但不足以识别唯一因果路径。配套全译保留原文摘要与表格的个别数字差异。

11.14 推理预算怎样转成下一轮监督

测试时计算花在当前问题上；训练则改变以后问题的回答分布。CaT[49] 连接这两件事：从同一策略生成多条轨迹，聚合为伪参考，再将与伪参考的符合程度转成强化学习奖励。在难以写程序验证的任务中，它还用生成的细分规则和 LLM 评分器给奖励。这增加了可生成反馈的范围，也引入了聚合器、规则和评分器的偏差。

将全过程写为 $o_{1:G} \sim \pi_\theta$，$s = A(q, o_{1:G})$，$r_i = R(q, o_i, s)$，然后更新 θ。s 不是隐藏的标准答案；它也是一个推断结果。若所有轨迹共享误解，增加 G 未必增加独立证据。若使用另一模型聚合，知识可能来自那个模型；“没有人工参考标注”不能简化成系统从无知识状态自行产生真理。

命题 11.6 二元答案的每条独立轨迹出错概率为 $p < 1/2$，奇数 G 的多数投票错误概率为

$$\sum_{j=(G+1)/2}^{G} \binom{G}{j} p^j (1-p)^{G-j}.$$

若同一组全部轨迹共享一个错误 Bernoulli 变量，则多数投票错误概率仍为 p，与 G 无关。

证明. 独立时错误数为二项分布，多数错误对应其右尾。完全共享时，每组要么全部正确，要么全部错误，投票不改变事件。 □

我们的真实 CPU 模拟设 $p = 0.3$、每种条件 20,000 组。$G = 9$ 时，独立错误的伪参考错误率为 0.10055，共享错误为 0.30045；$G = 33$ 时分别为 0.00795 与 0.29570。共享条件总是全体一致，仍经常错误。因此输出收敛能减少训练中的差异信号，却不意味着知识正确。这是人为规定相关性的机制实验，没有 LLM、医疗任务或 CaT 的真实评分器，不用于估计论文医疗可靠性。

作者在指定模型与 HealthBench、MATH-500 上报告效果及推理成本收益。学习者应查看：评价是否独立于训练规则、聚合与更新分别贡献多少、不同 judge 是否改变排序、预算节省是训练后单次推理还是包含训练总成本。一个有用的本地后续研究是在已知真值的任务上故意注入共享偏差，用外部留出规则审计伪参考；在开放域没有真值时，要把人工/工具证据的新增信息来源明确写出。

11.15 形式核验把研究闭环推进到哪里

AlphaProof[50] 将形式数学状态、策略与价值学习、证明搜索以及测试时强化学习结合。它与 FunSearch 的共同点是把模型提议放在可检查的闭环中；区别是核验的对象为形式证明而不是只在若干输入上的程序分数。正式论文的奥赛结果还涉及人工形式化题目和与 AlphaGeometry 2 的联合贡献，应按系统和时间预算读，不能将联合成绩都记到一个模型。

设根目标为命题 P。一个动作把它变成子目标集合 $\{P_1, \ldots, P_k\}$；要接受该动作，需要每个子目标都有有效证明，这是 AND 条件。对同一目标尝试多个动作，只须其中一个成功，这是 OR 条件。若仅把子目标的价值均值当成已经证明，可能忽略一个尚未解决的分支；启发式价值用于排序，不是证书。搜索结束仍要由证明核检查整个构造。

例题 11.4 证明 $A \wedge B$ 的候选先给出了 A 的证明，价值网络估计 B 成功概率为 0.99。这不允许核验器接受 $A \wedge B$。若有两个候选，各自只完成一个不同分支，也不能仅凭平均价值宣布成功；需要组合成同一条合法证明并检查依赖和假设。

形式证书强于经验测试，但也有明确范围：核验的是形式命题在指定公理与环境中的成立；自然语言题意到形式命题的对应、模型的数据污染、证明搜索成本与新颖性仍需要外部审计。AI 辅助研究的分工因此是提出猜想与候选、搜索和核验、解释与独立审查，而不是把流畅叙述或高价值估计直接当证明。

习题 11.1 上面的条件查找例子，若相同表项在两次使用中收到相反梯度，其总梯度为零。能否说该表项从未被使用？

解答. 不能。梯度是两次依赖的和，可能抵消；它与没有计算路径不同。应检查具体访问和逐次作用，必要时设计输入干预。 □

习题 11.2 多数投票随样本数增加更准确，需要哪些假设？若评分器总给多数答案奖励，为什么这个训练目标可能固化错误？

解答. 这里的证明需要二元、独立同分布且每样本错误概率小于一半；相关、系统性误解或不同分布都破坏推论。错误多数会被当成伪参考，奖励再提高其概率，从而减少正确少数的出现。必须有外部证据或独立评价来识别这种闭环偏差。 □

11.16 练习与解答

习题 11.3 某搜索系统找到一个程序, 在一百万随机输入上全通过. 另一个系统找到二次多项式, 在三个互异精确输入上与二次目标相同. 哪一项可以得到本章的全域数学保证？如果随机输入更复杂, 结论会改变吗？

解答. 第二项满足命题 11.3, 在全部实数上相同. 第一项没有指定候选类和有限测试充分性, 任意程序仍可能在没测的输入上失败. 更多或更复杂的随机样本可以增加经验信心和覆盖面, 却不会自动成为全域证明. 两种证据回答的问题不同. □

习题 11.4 把错误评估器的成本项去掉, 零程序仍不受损失惩罚. 给出另一个也不受惩罚却错误的候选, 并解释为何增加候选数量不能修复评分规格.

解答. 常数 -1 在开发整数点上始终不超过 $x^2 + x$, 因而错误损失也是零, 但从未正确实现目标. 增加候选可能增加正确程序, 也增加指标漏洞的利用者. 只优化该评分没有理由优先选择真实目标; 应先修正比较为双侧误差或精确等式, 再评价搜索过程. □

习题 11.5 一个代理把失败记录写入外部记忆, 下一次任务做得更好. 哪些证据能区分记忆收益、更多计算收益与模型权重学习？

解答. 记录并比较模型权重或版本是否相同, 固定任务和调用预算, 分别运行有记忆、无记忆以及相同长度无关记忆对照. 保存实际检索内容与后续动作. 若权重不变, 不能称为参数学习; 若改善只在增加调用时出现, 也不能独立归于记忆. 还需在未参与写记忆的新任务上检验迁移. □

习题 11.6 两 token 去噪例中, 为什么仅凭每个 token 的交叉熵达到最优, 不能保证一次并行独立采样正确? 怎样在这个有限例子中生成正确联合样本?

解答. 边缘均为公平二元分布, 但真实联合只支持 00 与 11, 独立相乘却也支持 01 与 10. 可以先采第一位, 再根据第一位的真实条件分布令第二位相同; 或直接公平采样两个合法序列. 这个例子说明采样依赖必须表达联合结构, 不是说实际扩散模型都用一次独立采样. □

习题 11.7 某循环网络在测试样本上测得相邻状态差小于 10^{-6}. 能否据此得到离固定点小于 10^{-5}? 若答案仍错误, 与收缩定理是否矛盾?

解答. 仅测相邻差不足够. 还需有覆盖轨迹及固定点的闭不变集合, 并证明一致收缩率满足 $1-\rho \geq 0.1$, 才能用后验界得到所述距离. 测得局部 Jacobian 谱半径小也不能替代这些条件. 即使距离界成立, 固定点可能对应错误答案, 与稳定性定理不矛盾. □

习题 11.8 五次初始化在固定两个文档上全对. 一人按十个独立 token 算区间, 另一人把三个划分合成 15 次全对. 指出随机单位错误, 给出当前可解释的区间, 并说明怎样研究任意新文档泛化.

解答. 同一模型的两个文档不构成两次独立初始化, 各划分只有五次整次训练事件. 三个划分复用初始权重并共享文档, 也未构成 15 次 IID Bernoulli 事件. 当前逐划分区间为 $[0.4782, 1]$, 解释范围限于所声明初始化假设下的固定划分. 若目标是新文档泛化, 应冻结文档生成分布, 抽取独立测试文档并核对长度、组合、随机分支和规则标签, 再以正确的文档或整任务为随机单位评价. □

A 向量、矩阵与表示空间

本章是按需查阅的数学工具: 嵌入查表、线性层和残差相加需要前两节; 当低秩适配、表示子空间或参数敏感性不清楚时, 再读投影、SVD 与稳定化部分. 主线可以先学模型, 遇到这些具体缺口再回查.

A.1 从 token 到线性层

语言模型接收离散 token, 内部却计算实数. 设词表有 V 个 token, 嵌入表 $E \in \mathbb{R}^{V\times d}$ 的第 i 行存储第 i 个 token 的表示. 一段长为 T 的序列查表后成为 $X \in \mathbb{R}^{T\times d}$. 本章建立读懂 XW、点积评分、残差相加和低秩更新所需的数学. 表示的坐标是训练得到的数, 单个坐标并不自动对应一个人能命名的概念.

定义 A.1 $\mathbb{R}^d$ 中的向量是有序实数列 $x = (x_1, \ldots, x_d)^\top$. 加法和数乘逐坐标进行. 映射 $F : \mathbb{R}^d \to \mathbb{R}^m$ 称为线性的, 若对所有 x, y 和实数 a, b 有 $F(ax + by) = aF(x) + bF(y)$.

令 e_j 是第 j 个坐标为 1、其余为 0 的向量. 因为 $x=\sum_j x_j e_j$, 线性性给出 $F(x)=\sum_j x_j F(e_j)$. 把 $F(e_j)$ 放成矩阵 A 的第 j 列, 就得到 $F(x)=Ax$, 其中 $(Ax)_i=\sum_j A_{ij}x_j$. 所以矩阵不是额外的一种预测魔法, 而是线性映射在指定坐标下的记账方式. 查嵌入也能写成 $E^\top e_i$, 但实现直接取行, 不必构造长为 V 的稀疏向量.

若按行存放批量表示, 线性层写成 $Y=XW$, 其中 $W\in\mathbb{R}^{d\times r}$、$Y\in\mathbb{R}^{T\times r}$. 矩阵乘法定义为 $(AB)_{ij}=\sum_k A_{ik}B_{kj}$, 因而正好同时计算各行表示的变换. 输出偏置 $b\in\mathbb{R}^r$ 加到每行, 得 $Y_{tj}=\sum_i X_{ti}W_{ij}+b_j$. 带偏置的映射是仿射映射, 除非偏置为零, 它不满足 $F(0)=0$.

转置交换行列: $(A^\top)_{ij}=A_{ji}$. 按指标计算, $(AB)^\top_{ij}=\sum_k A_{jk}B_{ki}=(B^\top A^\top)_{ij}$. 形状检查可提前发现错误: $XX^\top$ 是 $T\times T$ 的位置间比较, $X^\top X$ 是 $d\times d$ 的特征间比较. 残差形式 $X+F(X)$ 则要求两项同形; 若输出宽度改变, 必须另设一个把 X 映到同形状的支路.

A.2 内积、长度与投影

定义 A.2 $x,y\in\mathbb{R}^d$ 的欧氏内积为 $x^\top y=\sum_j x_j y_j$, 范数为 $\|x\|=\sqrt{x^\top x}$. $x^\top y=0$ 称为正交. 一个非零向量除以其范数后称为单位向量.

内积把两个表示变成标量, 注意力用它衡量查询与键的匹配. 它同时依赖方向与长度: 把查询乘以正数, 所有点积也乘该数. 因此点积不能直接解释为概率, 也不能仅凭其数值解释为语义相似度.

命题 A.3 (Cauchy–Schwarz 不等式) *对任意 $x,y\in\mathbb{R}^d$, $|x^\top y|\le\|x\|\|y\|$. 当 $y\neq 0$ 时, 等号成立当且仅当 x 为 y 的实数倍.*

证明. $\|x-ty\|^2=\|x\|^2-2tx^\top y+t^2\|y\|^2\ge 0$ 对每个实数 t 都成立. 若 $y\neq 0$, 选 $t=(x^\top y)/\|y\|^2$, 得 $0\le\|x\|^2-(x^\top y)^2/\|y\|^2$. 等号正好表示这个 $x-ty$ 为零. 若 $y=0$, 两边均为零. □

归一化点积因此位于 $[-1,1]$. 此外, $\|x+y\|^2\le\|x\|^2+2\|x\|\|y\|+\|y\|^2$, 开方得到三角不等式. 这会在第 C 章控制梯度变化时使用.

为了理解表示中哪些变化能被某个线性读出器看见, 要区分可表示的方向和正交的方向. 向量集合若对加法和数乘封闭, 称为子空间. 向量 $a_1,\dots,a_k$ 的张成空间是所有 $\sum_j c_j a_j$ 的集合; 它们线性无关是指 $\sum_j c_j a_j=0$ 只在所有 $c_j=0$ 时成立. 一组无关且张成该空间的向量称为基, 基的个数称为维数.

命题 A.4 *任意有限个向量张成的子空间 $S\subseteq\mathbb{R}^n$ 有一组正交单位基 $u_1,\dots,u_k$. 对 $y\in\mathbb{R}^n$, 令 $p=\sum_j(u_j^\top y)u_j$, 则 p 是 S 中与 y 距离最小的唯一向量.*

证明. 顺次处理张成向量. 已构造 $u_1,\dots,u_s$ 后, 从下一个向量 a 减去 $\sum_{j=1}^{s}(u_j^\top a)u_j$, 得到 v. 对每个 u_j, $u_j^\top v=0$. 若 $v=0$, a 已在原有张成空间中; 若不为零, 加入 $v/\|v\|$. 每一步保留已经处理的向量的张成空间, 有限步后得到所需基. 零空间用空基表示.

对每个 u_j, 有 $u_j^\top(y-p)=0$, 故 $r=y-p$ 与 S 中每个向量正交. 对任意 $z\in S$, 展开平方得到

$$\|y-z\|^2=\|r+(p-z)\|^2=\|r\|^2+\|p-z\|^2.$$

第二项非负, 仅在 $z = p$ 时为零. 这同时证明存在性与唯一性. □

这个 p 称为正交投影. 投影会丢掉 y 的正交分量; 低维表示的限制是由保留的子空间决定的, 不能由“低维”二字直接判断是否保留了任务需要的信息.

A.3 最小二乘作为可辨识性的工具

本节用一个小读出问题解释秩亏; 它是读懂表示空间和条件数的补充工具, 不把线性回归当作语言模型的训练主线. 设 $X \in \mathbb{R}^{n\times d}$ 每行是一个表示, $y \in \mathbb{R}^n$ 是欲读出的标量. 目标为 $J(w) = \|Xw - y\|^2/(2n)$. X 的列空间 $\{Xw : w \in \mathbb{R}^d\}$ 是所有可表示输出; 其维数称为 X 的秩. 核空间 $\ker X = \{h : Xh = 0\}$ 则是读出结果看不见的权重变化.

命题 A.5 最小二乘总有解. w_* 是解当且仅当满足正规方程 $X^\top(Xw_* - y) = 0$. 最优输出 Xw_* 唯一, 所有最优权重恰为 $w_* + \ker X$.

证明. 由命题 A.4, y 在列空间中的投影 p 存在. 因为 p 在列空间中, 至少有一个 w_* 满足 $Xw_* = p$. 投影残差正交于各列, 即正规方程. 反过来, 若满足正规方程, 对任意 h 有

$$J(w_* + h) = J(w_*) + \frac{1}{n}h^\top X^\top(Xw_* - y) + \frac{1}{2n}\|Xh\|^2 = J(w_*) + \frac{1}{2n}\|Xh\|^2.$$

所以它是最优解, 另一个权重同样最优当且仅当 $Xh = 0$. 最优输出唯一也可由投影唯一性得到. □

实对称矩阵 H 正定指 $h^\top Hh > 0$ 对每个 $h \neq 0$ 成立; 非负时称为半正定. 若 X 列无关, $h^\top X^\top Xh = \|Xh\|^2 > 0$, 因而 $X^\top X$ 正定. 方阵可逆指存在矩阵 A^{-1} 使 $AA^{-1} = A^{-1}A = I$, 其中 I 为对角为 1 的单位矩阵. 正定方阵核为零; 消元法给出满秩方阵的唯一解与逆, 故此时 $w_* = (X^\top X)^{-1}X^\top y$. 这里仅调用有限维线性方程的消元结论, 不将逆矩阵公式延伸到秩亏情形.

例题 A.1 令 $X = \begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}$, $y = (1, 0)^\top$. 求所有最优权重, 再在其中找长度最小的一个.

解答. 记 $s = w_1 + w_2$, 则平方残差为 $(s-1)^2 + (2s)^2 = 5(s - 1/5)^2 + 4/5$. 所以最优条件为 $s = 1/5$, 全部解为 $(a, 1/5 - a)^\top$. 又 $a^2 + (1/5 - a)^2 = 2(a - 1/10)^2 + 1/50$, 最小范数解为 $(1/10, 1/10)^\top$. 最优预测为 $(1/5, 2/5)^\top$, 残差 $(4/5, -2/5)^\top$ 与两列都正交. 重复特征没有新增可表示输出, 只是让权重有多种表示. □

A.4 奇异方向、低秩更新与稳定性

定理 A.6 (本章调用的实对称谱定理) 每个实对称 $d \times d$ 矩阵 H 都有一组正交单位基 $v_1, \ldots, v_d$ 和实数 λ_j, 满足 $Hv_j = \lambda_j v_j$. 即 $H = V\,\mathrm{diag}(\lambda_1, \ldots, \lambda_d)V^\top$, $V^\top V = I$.

本书调用这个存在性定理, 不证明其完整理论; 可查教材的对称特征分解部分 [3]. $Hv = \lambda v$ 中非零 v 称为特征向量, λ 为特征值. 将 $h = \sum_j a_j v_j$ 代入可得 $h^\top Hh = \sum_j \lambda_j a_j^2$, 所以正定、半正定分别等价于全部特征值正、非负. 后文的方向收敛论证仅依赖这个已声明的工具.

对矩形 $X \in \mathbb{R}^{n\times d}$, 应考察 $X^\top X$. 它对称半正定. 选其正交特征基, 将正特征值记为 $\sigma_1^2 \ge \cdots \ge \sigma_r^2 > 0$. 定义 $u_j = Xv_j/\sigma_j$, 则 $u_i^\top u_j = v_i^\top X^\top X v_j/(\sigma_i\sigma_j) = \delta_{ij}$. 零特征值对应 $\|Xv_j\|^2 = 0$. 因此对每个 $w = \sum_j a_j v_j$,

$$Xw = \sum_{j=1}^{r} \sigma_j a_j u_j, \qquad X = \sum_{j=1}^{r} \sigma_j u_j v_j^\top.$$

这就是紧致奇异值分解 (SVD), σ_j 是奇异值. 我们已由调用的谱定理推导其形式, 而非另假定矩形矩阵有特征分解. r 正好等于秩: 正奇异值的输出向量无关, 且张成列空间.

将 $y = \sum_{j\le r}(u_j^\top y)u_j + y_\perp$ 代入残差, 得

$$\|Xw - y\|^2 = \sum_{j\le r}(\sigma_j a_j - u_j^\top y)^2 + \|y_\perp\|^2.$$

所以最小范数解是 $w^\dagger = \sum_{j\le r}(u_j^\top y/\sigma_j)v_j$: 每个可见方向单独拟合, 核方向系数取零以使范数最小. 小 σ_j 会把标签扰动放大. 满列秩时二范数条件数定义为 $\kappa_2(X) = \sigma_1/\sigma_d$; 因 $X^\top X$ 的特征值是 σ_j^2, 它的条件数为 $\kappa_2(X)^2$. 秩亏时完整逆问题条件数记为无穷; 只在非零奇异子空间上比较则要明确这个限制.

SVD 也给出可计算的低秩表示. 截断到前 k 项得到 X_k, 其秩至多为 k, 且逐坐标平方求和的 Frobenius 范数满足 $\|X - X_k\|_F^2 = \sum_{j>k}\sigma_j^2$, 因为这些外积两两正交. 对中心化表示做主成分分析 (PCA), 就是在这些方向上考察平方能量. 这里只解释分解和尾部误差, 不调用或证明所有低秩近似的最优性定理.

LoRA 把冻结线性层的更新写成两张窄矩阵的乘积 [13]. 用本书行表示约定, $W = W_0 + AB$, $A \in \mathbb{R}^{d\times k}$、$B \in \mathbb{R}^{k\times r}$. XAB 先把输入映到 k 维, 再映回输出空间. AB 的每列都在 A 的列空间中, 因而秩至多为 k; 训练参数数为 $k(d+r)$, 而完整更新有 dr 个. 这些是代数事实. 某个任务是否能用小 k 保持质量, 是要做对照实验的问题. SVD 截断已有矩阵与训练一个低秩更新是两个不同的操作, 不应混同.

A.5 稳定化工具与 CPU 实验

在上述平方目标加 $\lambda\|w\|^2/2$, $\lambda > 0$, 便是岭惩罚. 正规方程成为 $(X^\top X/n + \lambda I)w = X^\top y/n$. 对每个非零 h, 对应二次型为 $\|Xh\|^2/n + \lambda\|h\|^2 > 0$, 所以解唯一, 即使 X 秩亏. 用 SVD 坐标逐项求极小值, 得

$$w_\lambda = \sum_{j=1}^{r} \frac{\sigma_j}{\sigma_j^2 + n\lambda}(u_j^\top y)v_j.$$

与 $1/\sigma_j$ 相比, 小奇异值方向被压低; 因 $(\sigma_j - \sqrt{n\lambda})^2 \ge 0$, 放大系数至多 $1/(2\sqrt{n\lambda})$. 但它也压低真实信号. 稳定性改善并不保证所有测试误差改善. 对网络参数使用平方惩罚时, 还要与第 C 章的逐坐标优化器一起分析.

运行 `python experiments/v03_foundations.py --out results/v03-foundations`, 可复算上述秩亏例并观察近乎重复的两列. 种子 144、80 个样本、列差尺度 10^{-5} 的实际结果为 $\kappa_2(X) \approx 1.9453 \times 10^5$, Gram 条件数约 3.7843×10^{10}. 取 $\lambda = 0.01$, 正则方程条件数约为 173.04. 在标签添加尺度 10^{-4} 的指定扰动后, 无惩罚权重变化范数约 14.1421, 岭权重变化约 5.3066×10^{-6}. 无惩罚的预测变化却只有约 8.5515×10^{-4}: 大权重变化集中在几乎不可见的方向. 这些数值来自 `results/v03-foundations/metrics.json`, 只说明此构造, 不是语言模型训练的实测结果. 脚本用 SVD 求解, 避免显式求逆以及将病态 Gram 矩阵当作可靠数值输入.

习题 A.1 若两个嵌入坐标在所有输入上相同, 为什么线性读出器的两项权重无法单独识别? 正则化后的唯一权重是否证明两个坐标分别有独立的语义作用?

解答. 设计矩阵两列相同, $h = (1, -1)^\top$ 在核中, 所以沿 h 任意改变权重都不改变输出. 平方惩罚在这条等价解直线上选择范数最小者, 不是新增了数据证据. 唯一性来自选择准则, 不能据此断言两坐标有独立语义. □

习题 A.2 设 $Y = XAB$、$A \in \mathbb{R}^{d\times k}$、$B \in \mathbb{R}^{k\times r}$. 若 $R \in \mathbb{R}^{k\times k}$ 可逆, 把 A, B 改成 $AR, R^{-1}B$ 会改变输出吗? 这对解释低秩因子的某一列有何影响?

解答. $(AR)(R^{-1}B) = AB$, 输出不变. 中间 k 维空间的基可以改变而不影响更新矩阵. 因而某一因子列的坐标意义依赖这种基选择; 要解释任务作用, 应考察最终更新、子空间或对输出的干预, 而不只看一个任意基向量. □

习题 A.3 把一个特征乘以 100, 无惩罚的可表示输出是否改变? 相同岭惩罚是否保持等价?

解答. 非零缩放不改变列空间, 相应权重除以 100 就保持输出. 但该权重的平方惩罚变成原来的 1/10000, 所以相同惩罚不再表达相同选择准则. 表示缩放和正则强度应一起考虑. □

B 序列概率、交叉熵与不确定性

本章可按需查阅: 条件语言模型需要联合与条件概率、序列似然和交叉熵三部分; 手写输出层时回查 softmax; 比较实验概率或有限样本结果时, 再查期望、方差与不确定性. 末尾二项区间是可跳过的补充, 不构成学习现代模型的先修门槛.

B.1 联合分布与下一个 token

语言模型在一个前缀后给出下一 token 的分布. 理解这句话需要三个不同对象: 整段序列的联合概率, 给定前缀后的条件概率, 以及用有限语料估计的模型概率. 三者混在一起, 就容易误把“一个 token 预测得准”当成“整段回答一定正确”. 本章先在有限集合上建立概率, 不需要测度论.

定义 B.1 有限随机变量 Y 的取值为 $1, \dots, V$, 概率 $p_j = P(Y = j)$ 满足 $p_j \geq 0$、$\sum_j p_j = 1$. 两个变量的联合分布为 $p_{ij} = P(X = i, Y = j)$. 边缘分布为 $P(X = i) = \sum_j p_{ij}$. 若 $P(X = i) > 0$, 条件分布为 $P(Y = j \mid X = i) = p_{ij}/P(X = i)$.

条件分布之和为 1, 因为分子求和恰为分母. 在零概率输入处, 这个比值没有定义; 一个模型仍可以在该输入上输出数, 但这些数不能从原分布的条件频率推出来. 条件概率也不表示因果干预的结果: “读到这个前缀时怎样续写”并不自动回答“改变某个事实后世界怎样变化”.

由定义立刻有 $P(X=i,Y=j)=P(X=i)P(Y=j\mid X=i)$. 反复应用给出概率链式分解. 对固定长度 T 的 token 序列 $Y_{1:T}$, 在相关前缀具有正概率时,

$$P(Y_{1:T}=y_{1:T})=P(Y_1=y_1)\prod_{t=2}^{T}P(Y_t=y_t\mid Y_{1:t-1}=y_{1:t-1}).$$

证明可以逐步约去前缀概率: 第 t 个条件因子等于长 t 前缀概率除以长 $t-1$ 前缀概率, 相乘只留下完整序列概率. 这是一条恒等式, 不要求 token 独立. 独立反而是特殊限制: 当联合概率总是边缘概率的乘积时, 给定前缀不会改变后继分布.

反过来, 任意在每个前缀上归一化的下一 token 模型 q_θ 都定义一个固定长度联合分布. 求所有序列的概率之和时, 先对最后一个 token 求和, 最后一个条件因子之和为 1; 再依次向前求和, 最终得到 1. 即使某个前缀概率为零, 在该处指定任何归一化条件都不会改变联合分布. 对可变长度序列, 还需结束 token 和终止条件; 每一步归一化并不单独保证最终以概率 1 停止. 后文的小实验使用明确的固定长度或结束规则.

例题 B.1 两个 token 为 a,b, 长为 2 的序列概率依次为 $P(aa)=3/10$, $P(ab)=2/10$, $P(ba)=1/10$, $P(bb)=4/10$. 已知第一 token 为 a, 下一 token 为 a 的概率是多少? 第二 token 的边缘概率相同吗?

解答. $P(Y_1=a)=1/2$, 所以 $P(Y_2=a\mid Y_1=a)=(3/10)/(1/2)=3/5$. 但 $P(Y_2=a)=P(aa)+P(ba)=2/5$. 前缀改变了分布, 不能将联合分解误写成两个边缘概率相乘. □

B.2 期望、方差与训练平均

定义 B.2 在有限取值分布上, $\mathbb{E}[f(Y)]=\sum_j p_j f(j)$. 实随机变量 Z 的方差为 $\mathrm{Var}(Z)=\mathbb{E}[(Z-\mathbb{E}Z)^2]$, 标准差为方差的平方根. 两个实随机变量的协方差为 $\mathrm{Cov}(Z,W)=\mathbb{E}[(Z-\mathbb{E}Z)(W-\mathbb{E}W)]$.

期望是按概率加权的平均, 不要求平均值本身是可能的观测值. 对 token 编号直接求平均通常没有语义; 有意义的是对损失、长度或成功指示量求平均. 分配律给出 $\mathbb{E}[aZ+bW]=a\mathbb{E}Z+b\mathbb{E}W$, 无需独立. 展开平方并使用 $\mathbb{E}[Z-\mathbb{E}Z]=0$, 得 $\mathrm{Var}(Z)=\mathbb{E}[Z^2]-(\mathbb{E}Z)^2$.

条件期望也是加权平均. 在 $P(X=i)>0$ 时令 $m_i=\sum_j P(Y=j\mid X=i)f(i,j)$. 再按 X 的概率求平均,

$$\sum_i P(X=i)m_i=\sum_{i,j}p_{ij}f(i,j)=\mathbb{E}[f(X,Y)].$$

这就是有限情形的全期望公式. 它允许把“每种前缀下的损失”平均成“所有前缀上的总体损失”.

若 $Z_1,\dots,Z_n$ 独立同分布, 均值为 μ、方差为 σ^2, 样本平均 $\bar{Z}=n^{-1}\sum_i Z_i$ 满足

$$\mathbb{E}\bar{Z}=\mu,\qquad \mathrm{Var}(\bar{Z})=\frac{1}{n^2}\sum_{i,j}\mathrm{Cov}(Z_i,Z_j)=\frac{\sigma^2}{n}.$$

对不同 i,j, 独立使 $\mathbb{E}[Z_iZ_j]=\mathbb{E}Z_i\mathbb{E}Z_j$, 故协方差为零; 相同指标留下 n 个方差项. 同一文档中的 token 损失通常相关, 这时交叉项不能删. 把相邻窗口当成独立样本, 会把不确定性估计得过小. 独立抽样是实验假设, 不是由求平均自动获得的性质.

B.3 从序列似然到 token 交叉熵

设训练集含序列 $s^{(1)},\dots,s^{(M)}$. 若将各序列视为独立抽取, 模型对已观察数据的似然为 $L(\theta)=\prod_{m=1}^{M} q_\theta(s^{(m)})$. 这里数据固定、参数变化; 似然不是“参数的概率分布”. 在 $q_\theta(s^{(m)})>0$ 时, 对数的单调性使最大似然等价于最小负对数似然.

利用联合分解, 若第 m 段有 T_m 个被计入目标的 token, 总数 $N=\sum_m T_m$, 则每 token 平均损失为

$$\ell(\theta)=-\frac{1}{N}\sum_{m=1}^{M}\sum_{t=1}^{T_m}\log q_\theta(s_t^{(m)}\mid s_{<t}^{(m)}).$$

这推导解释了为什么训练会在每个位置累加一个对数损失. 分解并没有假定这些 token 独立. 训练时给定语料里的真实前缀计算条件概率; 模型生成时前缀来自已经生成的 token, 前缀分布会不同. 若 padding、不参与训练的提示或被屏蔽位置存在, 它们必须从 N 和求和中同时去掉. 若改为先对每段求平均再对段平均, 长段与短段的权重会改变; 两种指标不可不加说明地比较.

固定一个前缀 c, 设真实后继分布为 $p(\cdot\mid c)$、模型分布为 $q(\cdot\mid c)$. 一次观测损失的期望为 $-\sum_j p_j\log q_j$. 如果同一前缀出现多次, 用经验频率 $\hat p_j$ 替代真实 p_j, 同样得到 $-\sum_j \hat p_j\log q_j$. 真实分布未知、前缀常常稀疏, 所以语言模型用共享参数连接不同前缀; 它并非给每个前缀单独存一张频率表.

定义 B.3 本书用自然对数. 熵 $H(p)=-\sum_j p_j\log p_j$, 交叉熵 $H(p,q)=-\sum_j p_j\log q_j$, KL 散度 $\mathrm{KL}(p\|q)=\sum_{j:p_j>0} p_j\log(p_j/q_j)$. 约定 $0\log 0=0$; 若 $p_j>0$ 而 $q_j=0$, 后两者为 $+\infty$.

熵描述分布自身的不确定性; 交叉熵是用模型 q 预测来自 p 的数据时的期望对数损失. KL 衡量这种错配, 但不是对称距离. 下面先证明一个不等式, 再证明损失的最优条件.

引理 B.4 (对数和不等式) 设 $a_j,b_j>0$, $A=\sum_j a_j$、$B=\sum_j b_j$. 则 $\sum_j a_j\log(a_j/b_j)\ge A\log(A/B)$, 等号当且仅当所有 a_j/b_j 相同.

证明. 对 $u>0$, 函数 $u-1-\log u$ 的导数为 $1-1/u$, 在 $u<1$ 时为负, $u>1$ 时为正, 因而 $\log u\le u-1$, 等号仅在 $u=1$. 令 $\alpha_j=a_j/A$、$\beta_j=b_j/B$. 两者分别和为 1, 故

$$\sum_j \alpha_j\log(\alpha_j/\beta_j)=-\sum_j \alpha_j\log(\beta_j/\alpha_j)\ge\sum_j \alpha_j(1-\beta_j/\alpha_j)=0.$$

另一方面 $\log(a_j/b_j)=\log(A/B)+\log(\alpha_j/\beta_j)$. 乘以 a_j 后求和即得结论. 每项的差都非负, 总差为零当且仅当每个 $\beta_j/\alpha_j=1$, 等价于比值 $a_j/b_j=A/B$ 恒定. □

命题 B.5 对有限概率分布, $H(p,q)=H(p)+\mathrm{KL}(p\|q)$, 且 $\mathrm{KL}(p\|q)\ge 0$, 等号当且仅当 $p=q$.

证明. 对数拆分给出恒等式; 若交叉熵无穷, 由于有限集合的 $H(p)$ 有限, 仍成立. 设 $S=\{j:p_j>0\}$. 若某个 S 中的 q_j 为零, 非负性直接成立且不能取等. 否则在 S 上用引理, $A=1$、$B=\sum_{j\in S} q_j\le 1$, 得 $\mathrm{KL}(p\|q)\ge-\log B\ge 0$. 等号要求 $B=1$ 且各 p_j/q_j 恒定, 因而 S 上 $p_j=q_j$, 外部 $q_j=0$. 反之 $p=q$ 时直接代入得零. □

所以在固定前缀的真实分布上，最小交叉熵是报告 p 本身. 再按前缀分布 $P(c)$ 求平均，总体差额就是 $\sum_c P(c)\,\mathrm{KL}(p(\cdot \mid c)\|q(\cdot \mid c))$，同样非负. 这是数学最优性. 有限语料、受限模型族和训练算法是否接近该最优点，则是三个额外问题. 即使总体 token 损失很低，也没有由此推出事实可靠或复杂任务成功的定理.

每 token 损失 ℓ 的困惑度定义为 $\exp(\ell)$. 若模型在每个位置均匀猜 V 个 token，则 $\ell = \log V$，困惑度为 V. 一般情况下它是对数损失的指数变换，不是逐位置“候选 token 个数”的字面计数. 不同分词改变 token 单位，因而跨分词比较困惑度需要另定一致的计量方式.

B.4 Softmax 与损失梯度

定义 B.6 对 logit 向量 $z \in \mathbb{R}^V$，$q_i = \mathrm{softmax}(z)_i = e^{z_i}/\sum_j e^{z_j}$. logit 是未归一化分数，概率 q_i 严格为正且和为 1.

加同一常数 c 不改变概率，因为分子分母的 e^c 相消. 因此计算时取 $m = \max_j z_j$，用 $q_i = e^{z_i - m}/\sum_j e^{z_j - m}$ 避免大的正指数溢出. 对数损失直接写为 $-z_y + m + \log\sum_j e^{z_j - m}$，避免先得到极小概率再取对数. 浮点下极小指数仍可能下溢，所以稳定的 log-sum-exp 形式比直接算 $\log q_y$ 更稳妥.

商法则给出

$$\frac{\partial q_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i}\sum_k e^{z_k} - e^{z_i}e^{z_j}}{(\sum_k e^{z_k})^2} = q_i(\delta_{ij} - q_j),$$

其中 δ_{ij} 在 $i = j$ 时为 1，否则为 0. 对分布目标 p，交叉熵可写成 $-\sum_i p_i z_i + \log\sum_k e^{z_k}$，因 $\sum_i p_i = 1$，所以 $\partial\ell/\partial z_j = q_j - p_j$. 硬标签 y 对应 $p_j = \delta_{jy}$. “预测减目标”来自这对函数的推导，不能随意套到别的损失与输出层上.

例题 B.2 两个 token 的 logit 为 $(0, \log 3)$，真 token 为第一个. 求概率、损失与两个 logit 的梯度.

解答. 指数为 $(1, 3)$，所以 $q = (1/4, 3/4)$，损失 $\log 4$，梯度 $(-3/4, 3/4)$. 沿负梯度提高正确 token 的分数并降低另一分数. 梯度之和为零，与整体平移不改变概率相符. □

B.5 估计误差与概率校准

假设在同一种受控前缀下独立观察 n 次，每次用 $Z_i \in \{0, 1\}$ 表示目标 token 是否出现. 若真实概率为 p，则 $\mathbb{E}Z_i = p$、$\mathrm{Var}(Z_i) = p(1-p)$. 经验频率 $\hat{p} = \bar{Z}$ 的期望为 p，方差为 $p(1-p)/n$. 对观测成功数 k，二元似然 $q^k(1-q)^{n-k}$ 的对数导数为 $k/q - (n-k)/(1-q)$；$0 < k < n$ 时唯一极大点为 $q = k/n$，端点情形分别取 0 或 1. 这说明频率是此小模型的最大似然估计，但不是“真实概率已被精确测出”.

命题 B.7 在上述独立假设下，对任意 $\epsilon > 0$，$P(|\hat{p} - p| \geq \epsilon) \leq p(1-p)/(n\epsilon^2) \leq 1/(4n\epsilon^2)$.

证明. 对非负变量 U，在 $U \geq a > 0$ 的事件上 $U \geq a$，所以 $\mathbb{E}U \geq aP(U \geq a)$. 取 $U = (\hat{p} - p)^2$、$a = \epsilon^2$，再代入方差公式. 最后 $p(1-p) = 1/4 - (p - 1/2)^2 \leq 1/4$. □

给定错误水平 $\alpha \in (0, 1)$，上式保证由 $\hat{p} \pm \sqrt{1/(4n\alpha)}$ 截到 $[0, 1]$ 得到的随机区间覆盖固定 p 的概率至少为 $1 - \alpha$. 这个界较宽，但不需要正态近似. 覆盖概率指重复采样生成的区间的比例，不是在已经看到数据后把固定参数当作随机变量. 高相关 token 或重复文档不满足这里的独立假设.

校准考察另一件事: 在许多被报告为概率 q 的事件中, 实际出现比例是否接近 q. 形式化地, 对二元标签 Z 和离散报告 Q, 校准要求 $P(Z=1 \mid Q=q)=q$ 对所有有正概率的报告成立. 准确率只看阈值或最大概率类别, 不足以判断校准; 即使总体校准, 也可能在某个子群或任务上出错.

种子 144 的 CPU 实验让两类前缀的真实目标概率为 $0.2, 0.8$, 每类独立抽取 2000 个标签. 两个报告器分别输出 $(0.2, 0.8)$ 和 $(0.05, 0.95)$. 两者预测的最大概率类别相同, 实测准确率都是 0.7975; 但平均负对数似然分别为 0.503868 和 0.647542, 两组平均校准差分别为 0.0110 和 0.1525. 准确率没有显示过度自信, 对数损失显示了它. 此合成实验不是对某个真实语言模型的校准评估; 真实评估还需选定事件、分组和抽样单位.

作为可跳过的统计补充, 脚本还对 $p=0.7$ 独立抽样 100 次, 得到 72 次成功. 它通过二项尾概率反演得到 95% 区间约 $[0.621333, 0.805206]$. 具体地, 成功数为 k 时下端满足 $P_{p=l}(K \geq k)=\alpha/2$, 上端满足 $P_{p=u}(K \leq k)=\alpha/2$; $k=0$ 取下端 0, $k=n$ 取上端 1. 这是调用的精确二项区间构造 [33], 本章不证明其完整覆盖定理. 脚本用有限求和与二分求端点, 另有有限样本覆盖测试. 3000 次重复的实际覆盖比例为 0.964333, 离散分布使覆盖常高于名义值; 单次模拟比例仍有误差, 不能替代理论保证. 所有数值与参数见 `results/v03-foundations/metrics.json`.

B.6 练习与解答

习题 B.1 真实下一 token 分布为 $(1/2, 1/2)$, 两模型报告 $(1/2, 1/2)$ 和 $(0.9, 0.1)$. 若平局固定选第一 token, 二者准确率和交叉熵怎样比较? 这是否说明一个模型更会推理?

解答. 两者都预测第一 token, 真实准确率均为 $1/2$. 交叉熵分别为 $\log 2$ 和 $-\frac{1}{2}\log(0.9 \cdot 0.1)$, 脚本复算约 0.693147 与 1.203973. 差为 KL, 约 0.510826, 反映概率错配. 题目只给一个二元分布, 没有任何推理任务证据. □

习题 B.2 两条序列分别长 1 和 9, 它们的每 token 平均损失分别为 a, b. 先对序列平均与直接按全部 token 平均得到什么? 为什么必须说明口径?

解答. 先序列平均得到 $(a+b)/2$, 全 token 平均为 $(a+9b)/10$. 前者让两序列权重相同, 后者让每个 token 权重相同. 长度分布变化会改变这两指标的关系; 比较模型时不能隐去这种变化. □

习题 B.3 证明 softmax 的 Jacobian $J_q=\operatorname{diag}(q)-qq^{\top}$ 为半正定, 并找出其核. 这与 logit 的可辨识性有什么联系?

解答. 对任意 a, 令 $\bar{a}=\sum_i q_i a_i$, 则 $a^{\top}J_q a=\sum_i q_i(a_i-\bar{a})^2 \geq 0$. 因为每个 $q_i>0$, 等号当且仅当所有 a_i 相同. 直接计算 $J_q\mathbf{1}=0$, 故核恰为常数向量的张成空间. 整体平移 logit 不改变概率, 所以概率无法识别 logit 的共同常数部分. □

C 反向传播与优化的数学工具

C.1 增量、偏导与梯度

本章可按需查阅: 看不懂线性层反向传播时读前两节, 对训练曲线和学习率感到疑惑时读曲率与下降估计, 要实现优化器时读随机梯度与 Adam. 主线中的表示空间与条件损失分别在第 A、B 章提供工具; 不必先把所有优化理论学完再进入 Transformer.

训练要改变参数, 以降低一个标量损失. 单变量导数描述一个微小增量的一阶影响. 对 $f:\mathbb{R}^d\to\mathbb{R}$, 同样希望所有坐标一起变化时有可靠的线性近似.

定义 C.1 若存在 $g\in\mathbb{R}^d$, 使 $f(w+h)=f(w)+g^\top h+r(h)$, 且 $r(h)/\|h\|\to 0$ 当 $h\to 0$, 则称 f 在 w 可微, $g=\nabla f(w)$ 为梯度. 记 $r(h)=o(\|h\|)$. 偏导 $\partial f/\partial w_j$ 是只让第 j 个坐标变化的一元导数.

可微时取 $h=te_j$, 得 $g_j=\partial f/\partial w_j$, 所以梯度就是各偏导组成的向量. 反过来, 只有各偏导存在还不够. 例如令 $f(x,y)=xy/\sqrt{x^2+y^2}$ 在非零点定义, $f(0,0)=0$. 两坐标轴上它恒为零, 所以原点两偏导都为零. 但沿 $h=(t,t)$, $|f(h)|/\|h\|=1/2$, 余项没有趋于零. 坐标方向的检查漏掉了斜向变化.

一个足够条件是偏导在该点附近存在且连续. 理由是沿坐标依次从 w 走到 $w+h$, 将总变化拆为 d 个一元变化. 每段用一元中值定理, 得 $\sum_j \partial_j f(\xi_j)h_j$. 减去 $\sum_j \partial_j f(w)h_j$, 绝对值至多 $\max_j|\partial_j f(\xi_j)-\partial_j f(w)|\sum_j|h_j|$. 由 Cauchy–Schwarz, $\sum_j|h_j|\le\sqrt{d}\|h\|$; 各 $\xi_j\to w$ 时连续性使前一个因子趋于零, 就得到可微定义的余项条件.

若 $g\neq 0$, 在 $\|h\|=\epsilon$ 的限制下, $g^\top h\ge-\epsilon\|g\|$, 等号由 $h=-\epsilon g/\|g\|$ 达到. 负梯度是欧氏长度约束下最快的一阶下降方向. 这个结论依赖所用长度, 也只描述局部近似; 大步更新可能越过低损失区域.

定义 C.2 可微向量函数 $F:\mathbb{R}^d\to\mathbb{R}^m$ 的 Jacobian 为 $J_F(w)\in\mathbb{R}^{m\times d}$, 满足 $F(w+h)=F(w)+J_F(w)h+o(\|h\|)$; 其元素为 $(J_F)_{ij}=\partial F_i/\partial w_j$.

若 $g:\mathbb{R}^m\to\mathbb{R}$ 在 $F(w)$ 可微, $f=g\circ F$, 则

$$\nabla f(w)=J_F(w)^\top\nabla g(F(w)).$$

确切地, 记 $\Delta=F(w+h)-F(w)=J_Fh+o(\|h\|)$. 有 $\|\Delta\|\le C\|h\|$ 对充分小 h 成立, 因为固定矩阵对长度的放大有有限上界. 代入 g 的近似, 得 $g(F(w)+\Delta)-g(F(w))=\nabla g^\top J_Fh+o(\|h\|)$. 最后一项成立, 因为 g 的余项除以 $\|\Delta\|$ 趋于零, 且 $\|\Delta\|/\|h\|$ 有界. 这证明链式法则, 同时解释转置: 前向传递增量 J_Fh, 反向收集增量对标量损失的系数.

C.2 矩阵梯度与共享路径

矩阵参数的损失梯度必须与参数同形, 才能逐元素更新. 对同形矩阵 $U,V\in\mathbb{R}^{m\times n}$ 定义

$$\langle U,V\rangle_F=\sum_{i,j}U_{ij}V_{ij}=\operatorname{tr}(U^\top V),\qquad \|U\|_F=\sqrt{\langle U,U\rangle_F}.$$

迹 tr 是方阵对角之和; 由于 $(U^\top V)_{jj}=\sum_i U_{ij}V_{ij}$, 两种写法相等. 矩阵梯度 G_W 由 $\ell(W+H)=\ell(W)+\langle G_W,H\rangle_F+o(\|H\|_F)$ 定义. 令 H 仅有一个非零元素, 就得 $(G_W)_{ij}=\partial\ell/\partial W_{ij}$. 展平矩阵后它正是普通向量梯度; G_W 不是整个

矩阵输出函数的 Jacobian, 而是已经指定标量损失后的系数.

设行批量线性层 $Y = XW + \mathbf{1}b^\top$, $X \in \mathbb{R}^{B\times d}$、$W \in \mathbb{R}^{d\times r}$、$b \in \mathbb{R}^r$, 下游已经算出 $G_Y \in \mathbb{R}^{B\times r}$. 用 d 记一阶增量, 从元素式展开,

$$\begin{aligned} d\ell &= \sum_{s,j} (G_Y)_{sj}\, dY_{sj} \\ &= \sum_{s,j,i} (G_Y)_{sj}(dX_{si}W_{ij} + X_{si}dW_{ij}) + \sum_{s,j}(G_Y)_{sj}db_j \\ &= \sum_{s,i}\left(\sum_j (G_Y)_{sj}W_{ij}\right) dX_{si} + \sum_{i,j}\left(\sum_s X_{si}(G_Y)_{sj}\right) dW_{ij} + \sum_j \left(\sum_s (G_Y)_{sj}\right) db_j. \end{aligned}$$

比较独立增量的系数, $G_X = G_Y W^\top$、$G_W = X^\top G_Y$、$(G_b)_j = \sum_s (G_Y)_{sj}$. 这里 B 是批量行数, 若损失本身已经平均, G_Y 已含该平均系数, 不能在每层再除一次 B.

例题 C.1 $X = (1, 2)$, $W = (3, 4)^\top$, $\ell = (XW - 10)^2/2$. 求两矩阵的梯度, 并核对一个扰动的一阶变化.

解答. $Y = 11$, $G_Y = 1$, 所以 $G_W = (1, 2)^\top$, $G_X = (3, 4)$. 对 $dW = (a, b)^\top$, 一阶损失变化为 $a + 2b$. 直接展开 $(w_1 + 2w_2 - 10)^2/2$ 也得到这两个系数, 因而转置和形状可独立核对. □

网络里的参数和中间值常被重复使用. 若同一 x 沿两条路径进入损失, 两条路径的一阶变化系数要相加. 对残差 $Y = X + F(X)$, 链式法则给 $\nabla_X \ell = G_Y + J_F(X)^\top G_Y$, 这里把矩阵暂时展平成向量来解释 J_F. 直接支路的 G_Y 不能丢. 嵌入表查表也是共享: 若同一 token 在多个位置出现, 那一行的梯度等于所有这些位置梯度之和, 而非最后一次覆盖.

低秩层 $Y = X(W_0 + AB)$ 同样可逐层反传. 固定 W_0, 先令 $Z = XA$, 便有 $G_B = Z^\top G_Y = A^\top X^\top G_Y$, $G_A = X^\top G_Y B^\top$. 若初始 $B = 0$ 而 $A \neq 0$, 则 $G_A = 0$, 但 G_B 可能非零; 更新 B 后 A 也可获得梯度. 若两因子都为零, 两梯度都为零, 这个乘积支路无法自行启动. 这是参数化的代数性质, 可用来理解低秩适配的初始化选择.

C.3 Hessian、凸性与局部模型

定义 C.3 若 f 有连续二阶偏导, Hessian 为梯度的 Jacobian, $H_f(w)_{ij} = \partial(\partial f/\partial w_i)/\partial w_j$. 函数 f 在凸集合 D 上称为凸的, 若对 $u, v \in D$、$t \in [0, 1]$ 有 $f((1-t)u + tv) \le (1-t)f(u) + tf(v)$. 集合凸指两点之间的整条线段仍在集合中.

在连续二阶偏导的条件下, 混合偏导交换定理给 $H_f = H_f^\top$; 本书调用这一多变量微积分结论, 不将它扩展到只有二阶偏导存在的情形 [3]. 固定方向 h, 令 $\phi(t) = f(w + th)$, 链式法则给 $\phi'(t) = \nabla f(w + th)^\top h$, $\phi''(t) = h^\top H_f(w + th)h$. Hessian 的作用因此是描述沿方向的曲率, 而非描述梯度本身的大小.

为把变化率累积成总变化, 本章调用一元微积分基本定理: 对连续可微的 G, $G(b) - G(a) = \int_a^b G'(t)\, dt$. 定积分可理解为区间细分后 “函数值乘小段宽度” 的和的极限. 本章不重建积分存在理论, 只在连续被积函数和有限闭区间

上使用它. 连续两次积分可在三角形区域换序; 等价地用一元分部积分可验证下面的恒等式:

$$f(w+h) = f(w) + \nabla f(w)^\top h + \int_0^1 (1-t) h^\top H_f(w+th) h \, dt.$$

具体地, 对 ϕ 分部积分得 $\int_0^1 (1-t)\phi''(t)dt = -\phi'(0) + \phi(1) - \phi(0)$. 若把 Hessian 替换为 $H_f(w)$, 主项是 $\frac{1}{2}h^\top H_f(w)h$; 连续性使剩余项为 $o(\|h\|^2)$. 这给出了二阶局部模型.

命题 C.4 *设 $D \subseteq \mathbb{R}^d$ 为开凸集. 可微函数 f 凸当且仅当对所有 $u, v \in D$ 有 $f(v) \geq f(u) + \nabla f(u)^\top (v-u)$. 若 f 有连续二阶偏导, 又等价于 $H_f(w)$ 在每个 $w \in D$ 半正定.*

证明. 若凸, 对 $0 < t < 1$ 有 $[f(u+t(v-u)) - f(u)]/t \leq f(v) - f(u)$. 令 $t \downarrow 0$ 得一阶下界. 反之令 $z = (1-t)u + tv$, 分别在 z 对 u, v 用下界, 乘以 $1-t, t$ 后相加; 两个梯度项因 $(1-t)(u-z) + t(v-z) = 0$ 抵消, 得凸性. 端点 $t = 0, 1$ 为等号.

若 Hessian 半正定, 二阶积分余项非负, 得一阶下界. 若凸且二阶连续, 在任意点 w 和方向 h 上, 开集保证小 s 时 $w \pm sh \in D$. 凸性给 $f(w+sh) + f(w-sh) - 2f(w) \geq 0$. 用刚才的二阶展开, 除以 s^2, 再令 $s \to 0$, 得 $h^\top H_f(w) h \geq 0$. □

所以在凸函数上 $\nabla f(w) = 0$ 足以保证全局最小值. 非凸网络不满足这个前提; 例如 $f(x, y) = x^2 - y^2$ 原点梯度为零, 却沿一轴上升、另一轴下降. 对 softmax 交叉熵, 关于 logit 的 Hessian 是第 B 章算出的 $\mathrm{diag}(q) - qq^\top$, 因而关于 logit 凸. 把 logit 换成多层网络参数的非线性函数后, 这个凸性一般不会保留. 训练的最后一个函数简单, 不等于整个参数目标简单.

C.4 下降引理与步长边界

定义 C.5 在集合 D 上, 梯度满足 $\|\nabla f(u) - \nabla f(v)\| \leq L\|u - v\|$ 时, 称 f 为 L 光滑, 其中 $L > 0$. 这是梯度变化的上界, 不是函数值变化的同一个上界.

引理 C.6 (下降引理) *若 f 在凸集 D 上可微且 L 光滑, $w, w+h \in D$, 则 $f(w+h) \leq f(w) + \nabla f(w)^\top h + (L/2)\|h\|^2$.*

证明. 沿线段用微积分基本定理,

$$\begin{aligned} f(w+h) - f(w) - \nabla f(w)^\top h &= \int_0^1 [\nabla f(w+th) - \nabla f(w)]^\top h \, dt \\ &\leq \int_0^1 Lt\|h\|^2 \, dt = \frac{L}{2}\|h\|^2. \end{aligned}$$

不等式先用 Cauchy–Schwarz, 再用光滑条件; 凸集保证整段位于条件成立的区域. □

梯度下降 (GD) 更新为 $w_{t+1} = w_t - \eta \nabla f(w_t)$. 代入 $h = -\eta \nabla f(w_t)$, 得

$$f(w_{t+1}) \leq f(w_t) - \eta(1 - L\eta/2)\|\nabla f(w_t)\|^2.$$

在更新点仍位于条件区域且 $0 < \eta < 2/L$ 时, 非零梯度带来下降. 若光滑性在整个 $\mathbb{R}^d$ 成立且 $f \geq f_{\mathrm{low}}$, 将不等式从

$t=0$ 加到 $K-1$, 得

$$\min_{0\le t<K}\|\nabla f(w_t)\|^2 \le \frac{f(w_0)-f_{\text{low}}}{K\eta(1-L\eta/2)}.$$

这保证遇到小梯度, 不保证其对应全局最优, 也不证明整个迭代点序列收敛.

若另有凸性、全局最小点 w_*, 且 $0<\eta\le 1/L$, 还可得到目标误差界. 记 $g_t=\nabla f(w_t)$, 一阶凸下界和距离展开给

$$\|w_{t+1}-w_*\|^2 \le \|w_t-w_*\|^2 - 2\eta(f(w_t)-f(w_*)) + \eta^2\|g_t\|^2.$$

下降引理又给 $f(w_t)-f(w_{t+1})\ge \eta\|g_t\|^2/2$. 代入后, $2\eta(f(w_{t+1})-f(w_*)) \le \|w_t-w_*\|^2-\|w_{t+1}-w_*\|^2$. 求和且用目标单调性, 得 $f(w_K)-f(w_*)\le \|w_0-w_*\|^2/(2\eta K)$. 这些额外前提对一般 Transformer 训练没有自动成立, 但说明“下降”“小梯度”“全局误差小”是三个不同层次的结论.

二次目标提供可逐方向核对的边界. 对 $f(w)=\frac12 w^\top Hw-b^\top w$, H 对称正定, 梯度为 $Hw-b$. 唯一最优点 $Hw_*=b$, 误差 $e_t=w_t-w_*$ 满足 $e_{t+1}=(I-\eta H)e_t$. 调用第 A 章的谱定理, 每个特征分量按 $a_{t+1,j}=(1-\eta\lambda_j)a_{t,j}$ 更新. 因而对任意初值都收敛当且仅当 $0<\eta<2/\lambda_{\max}$. 若 $\mu\le\lambda_j\le L$, $\eta=1/L$ 时 $\|e_t\|\le(1-\mu/L)^t\|e_0\|$. 这是将各分量的收缩界平方求和得到的. 小 μ/L 使某些方向收敛慢, 与条件数的问题相连.

CPU 实验取 $H=\operatorname{diag}(1,10)$、$w_0=(1,1)^\top$. 80 步后, $\eta=0.1$ 的损失约为 2.3866×10^{-8}; $\eta=0.19$ 约为 2.3866×10^{-7}. 边界 $\eta=0.2$ 使第二分量永远交替为 ±1, 损失趋于 5; $\eta=0.21$ 的损失约为 2.0980×10^{7}. 端点不是安全步长. 实际网络的 L 通常未知且可能只在局部有界; 二次实验解释机制, 不给真实训练一个通用学习率.

C.5 随机梯度与噪声下限

真实语料太大时, 每步只读一小批数据. 对有限训练集目标 $J(\theta)=n^{-1}\sum_i \ell_i(\theta)$, 在固定当前参数下均匀抽取索引 I, 有 $\mathbb{E}[\nabla\ell_I(\theta)\mid\theta]=\nabla J(\theta)$. 这是有限求和的线性性, 不需要交换无限积分和求导. 若每次独立有放回抽 B 个索引后平均, 各坐标噪声方差除以 B. 不放回抽样、重加权样本和不均匀 token 数需要按实际抽样设计重新核对; 并非所有 mini-batch 梯度都对同一个目标无偏.

设随机梯度 $g=\nabla J(\theta)+\xi$, 条件均值 $\mathbb{E}[\xi\mid\theta]=0$, 条件平方长度 $\mathbb{E}[\|\xi\|^2\mid\theta]\le\sigma^2/B$. 若 J 全局 L 光滑, 随机更新 $\theta^+=\theta-\eta g$ 的下降引理给

$$\mathbb{E}[J(\theta^+)\mid\theta]\le J(\theta)-\eta(1-L\eta/2)\|\nabla J(\theta)\|^2+\frac{L\eta^2\sigma^2}{2B}.$$

推导只需 $\mathbb{E}g=\nabla J$ 和 $\mathbb{E}\|g\|^2=\|\nabla J\|^2+\mathbb{E}\|\xi\|^2$, 后者的混合项由零均值消失. 额外正项说明随机单步不保证下降. 增大批量或减小步长可降低这个上界中的噪声项, 但代价分别是计算量或更新幅度.

在一维 $f(w)=aw^2/2$, $a>0$, 若噪声 ξ_t 独立、零均值、方差 σ^2, 且独立于此前迭代, 更新为 $w_{t+1}=(1-\eta a)w_t-\eta\xi_t$. 因而 $\mathbb{E}w_{t+1}^2=(1-\eta a)^2\mathbb{E}w_t^2+\eta^2\sigma^2$. 当 $0<\eta a<2$ 时, 几何级数给

$$\mathbb{E}w_t^2=(1-\eta a)^{2t}w_0^2+\eta^2\sigma^2\frac{1-(1-\eta a)^{2t}}{1-(1-\eta a)^2}\longrightarrow\frac{\eta\sigma^2}{a(2-\eta a)}.$$

常数步长下, 均值可以趋于零, 均方误差却停在正数. “训练后期还在抖动”不一定是梯度实现错误, 也不能自动解释为有益的探索. 学习率日程是否改善实际模型, 仍要测量目标和任务指标.

C.6 Adam、偏差修正与失败边界

Adam 用梯度的指数平均提供方向, 并用平方梯度的指数平均逐坐标缩放 [8]. 设 $\beta_1, \beta_2 \in [0, 1)$、$\epsilon > 0$, $m_0 = v_0 = 0$. 第 t 步在 θ_{t-1} 计算梯度 g_t, 然后

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)(g_t \odot g_t),$$
$$\hat{m}_t = m_t/(1-\beta_1^t), \qquad \hat{v}_t = v_t/(1-\beta_2^t),$$
$$\theta_t = \theta_{t-1} - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon).$$

$\odot$、平方根和除法均逐坐标计算. v_t 是未中心化的二阶矩平均, 不是方差估计, 更不是 Hessian 的逆. 第一阶平均结合过去方向; 分母使持续较大梯度的坐标具有较小的缩放系数.

反复展开递推, $m_t = (1-\beta_1)\sum_{s=1}^{t} \beta_1^{t-s} g_s$. 权重和为 $1-\beta_1^t$, 所以零初始化导致初期幅度偏小. 若 $\mathbb{E} g_s = \mu$ 对所有 s 恒定, 则 $\mathbb{E} m_t = (1-\beta_1^t)\mu$, 除以此因子后均值正好为 μ. 相同推导对 v_t 成立, 前提是 $\mathbb{E}[g_s \odot g_s]$ 恒定. 这证明的是初始化偏差修正, 不要求各 g_s 独立. 若训练中的均值随步数变化, 修正后得到的是归一化的历史加权平均, 不必等于当前均值. 而且 $\mathbb{E}[\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)]$ 不能拆成两个期望相除, 所以不能据此声称 Adam 更新是无偏的梯度更新.

例如首步 $g_1 = (2, -3)^\top$、$\beta_1 = 0.9$、$\beta_2 = 0.999$, 有 $m_1 = (0.2, -0.3)^\top$、$v_1 = (0.004, 0.009)^\top$, 修正后分别为 g_1 和 $g_1 \odot g_1$. 忽略 ϵ 时首步的各非零坐标更新近似为 $-\eta \operatorname{sign}(g_{1,j})$, 并不等于 GD 的 $-\eta g_{1,j}$.

这也给出一个简单失败边界. 对 $f(w) = w^2/2$, 从 $w_0 = 10^{-3}$ 开始, 用 $\eta = 0.1$、$\epsilon = 10^{-8}$ 的 Adam 首步得到 $w_1 = w_0 - 0.1 w_0/(|w_0| + \epsilon) \approx -0.098999$. 损失从 5×10^{-7} 升到约 0.0049004. 因而即使目标凸且光滑, Adam 也没有每步下降的无条件保证. 这个例子只反驳单调下降, 不宣称所有 Adam 迭代必然发散.

平方惩罚与直接权重衰减也要区分. 对普通 GD, 把 $\lambda\|\theta\|^2/2$ 加入损失给 $\theta^+ = (1-\eta\lambda)\theta - \eta g$. 若优化器将输入梯度逐坐标乘以矩阵 D_t, 惩罚项产生 $-\eta\lambda D_t \theta$, 而直接衰减产生 $-\eta\lambda\theta$, 两者通常不同. Adam 还会把惩罚梯度写入历史一阶、二阶平均, 差别更不能用同一个系数名消掉. 阅读现代训练配方时应检查实现中的衰减定义, 不只看超参数叫不叫 `weight_decay`.

配套实验在 $H = \operatorname{diag}(1, 10)$ 上向梯度加入标准差 0.5 的独立噪声, 两算法使用同一条噪声序列. SGD 步长为 0.05, Adam 步长为 0.03, 运行 1000 步; 最后 200 步平均损失分别约为 0.006870 和 0.005436. 这是两个指定参数配置的一次比较, 不足以排名优化器. 末步损失分别约 0.002411 和 0.004166, 又说明用某一个末步作比较容易受波动影响. 参数、完整 GD 曲线、随机优化器检查点与代码哈希均保存在 `results/v03-foundations/`.

C.7 梯度检查与练习

中心有限差分 $[f(\theta + he_j) - f(\theta - he_j)]/(2h)$ 可检查解析梯度. 若沿该坐标三阶导数在邻域有界, 一元 Taylor 公式的两个展开相减, 得误差 $O(h^2)$. 浮点计算中极小 h 又会放大消减误差, 所以要检查不同尺度并使用双精度. 可比较 $|g_{\text{num}} - g_{\text{ana}}|/\max(1, |g_{\text{num}}|, |g_{\text{ana}}|)$. 这是有限个输入与坐标的实现诊断, 不代替链式法则证明. 在 ReLU 零点等不可微处, 两侧差分可能混合不同分支; 应避开边界或明确所用次梯度.

习题 C.1 设 $F(w) = Aw + b$, 标量损失对 F 的梯度为 $u \in \mathbb{R}^m$. 推导对 $w \in \mathbb{R}^d$、$A \in \mathbb{R}^{m\times d}$ 和 b 的梯度. 若 A 被两次使用, 应如何合并?

解答. $d\ell = u^\top(dA\, w + A\, dw + db)$, 故 $\nabla_w \ell = A^\top u \in \mathbb{R}^d$, $\nabla_A \ell = uw^\top \in \mathbb{R}^{m\times d}$, $\nabla_b \ell = u \in \mathbb{R}^m$. 两次使用产生两个关于同一 dA 的系数, 总梯度是两个外积之和, 因为损失一阶变化是两项之和. □

习题 C.2 一个 token 在三个位置出现, 三处对嵌入向量的梯度分别为 g_1, g_2, g_3. 若代码依次把这三个梯度赋值到嵌入表的同一行, 哪里错了? 若这三个位置还各有一条残差路径, 是否能用嵌入累加替代残差累加?

解答. 共享嵌入行变化 h 时, 损失一阶变化为 $(g_1 + g_2 + g_3)^\top h$, 所以应累加而不是覆盖. 残差支路的梯度应先在每个位置与变换支路相加, 再按 token 汇总. 两种相加发生在不同共享结构上, 漏掉其中任何一个都改变总系数. □

习题 C.3 对 $f(w) = 5w^2$, 哪些常数 GD 步长使任意初值收敛到零? 对非零初值, 梯度在步长端点仍是下降方向吗? 为什么更新不收敛?

解答. 更新 $w^+ = (1 - 10\eta)w$, 收敛要求 $|1 - 10\eta| < 1$, 即 $0 < \eta < 0.2$. 在 $\eta = 0.2$ 时, 非零 w 的负梯度仍与局部下降方向一致, 但完整一步到达 $-w$, 损失相同并持续振荡. 局部方向正确不能替代步长控制. □

习题 C.4 某训练日志显示随机梯度均值接近零但损失仍波动. 能否仅据此断言已到全局最优? 如果把批量增大四倍, 独立抽样下噪声标准差怎样改变?

解答. 不能. 随机均值可能有抽样误差; 即使真实梯度为零, 非凸目标也可能位于鞍点, 且常数步长噪声仍可造成波动. 在独立有放回抽样与方差有限的条件下, 批量均值方差与 $1/B$ 成正比, 批量增大四倍使标准差减半. 相关抽样或改变目标权重时不能直接套用. □

参考文献

[1] Andrew Ng. CS229 Lecture Notes: Supervised Learning. 2003. https://see.stanford.edu/materials/aimlcs229/cs229-notes1.pdf.

[2] Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola. Dive into Deep Learning. 1.0.3. https://d2l.ai/.

[3] Ian Goodfellow, Yoshua Bengio, Aaron Courville. Deep Learning. 2016. https://www.deeplearningbook.org/.

[4] Tatsunori Hashimoto, Percy Liang 等. CS336: Language Modeling from Scratch. Spring 2026. https://cs336.stanford.edu/.

[5] Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong. Mathematics for Machine Learning. 2020. https://mml-book.github.io/.

[6] Stephen Boyd, Lieven Vandenberghe. Convex Optimization. 2004. https://web.stanford.edu/~boyd/cvxbook/.

[7] Ashish Vaswani 等. Attention Is All You Need. 2017; v7 2023. https://arxiv.org/abs/1706.03762.

[8] Diederik P. Kingma, Jimmy Ba. Adam: A Method for Stochastic Optimization. 2014; ICLR 2015. https://arxiv.org/abs/1412.6980.

[9] Jared Kaplan 等. Scaling Laws for Neural Language Models. 2020. https://arxiv.org/abs/2001.08361.

[10] Jordan Hoffmann 等. Training Compute-Optimal Large Language Models. 2022. https://arxiv.org/abs/2203.15556.

[11] Tri Dao 等. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. https://arxiv.org/abs/2205.14135.

[12] Long Ouyang 等. Training language models to follow instructions with human feedback. 2022. https://arxiv.org/abs/2203.02155.

[13] Edward J. Hu 等. LoRA: Low-Rank Adaptation of Large Language Models. 2021. https://arxiv.org/abs/2106.09685.

[14] Rafael Rafailov 等. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023; v3 2024. https://arxiv.org/abs/2305.18290.

[15] Jason Wei 等. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 2022; v6 2023. https://arxiv.org/abs/2201.11903.

[16] Charlie Snell 等. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. 2024. https://arxiv.org/abs/2408.03314.

[17] DeepSeek-AI 等. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. 2025; v2 2026-01-04. https://arxiv.org/abs/2501.12948.

[18] Rylan Schaeffer, Brando Miranda, Sanmi Koyejo. Are Emergent Abilities of Large Language Models a Mirage?. 2023. https://arxiv.org/abs/2304.15004.

[19] Kai Greshake 等. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. 2023. https://arxiv.org/abs/2302.12173.

[20] Chuan Guo 等. IH-Challenge: A Training Dataset to Improve Instruction Hierarchy on Frontier LLMs. 2026-03-11. https://arxiv.org/abs/2603.10521.

[21] Jack Lindsey 等. On the Biology of a Large Language Model. 2025-03-27. https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

[22] Adam Karvonen, Euan Ong, Subhash Kantamneni, Samuel Marks. Would this change your answer? Evaluating

Explanations of LLM Behavior In The Wild with Counterfactual Experiments. 2026-08-17. https://arxiv.org/abs/2608.16747.

[23] Wes Gurnee 等. Verbalizable Representations Form a Global Workspace in Language Models. 2026-07-06. https://transformer-circuits.pub/2026/workspace/index.html.

[24] Bernardino Romera-Paredes 等. Mathematical discoveries from program search with large language models. 2023-12-14. https://www.nature.com/articles/s41586-023-06924-6.

[25] Alexander Novikov 等. AlphaEvolve: A coding agent for scientific and algorithmic discovery. 2025-06-16. https://arxiv.org/abs/2506.13131.

[26] John Jumper 等. Highly accurate protein structure prediction with AlphaFold. 2021. https://www.nature.com/articles/s41586-021-03819-2.

[27] Jenny Zhang 等. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. 2025; v3 2026-03-12. https://arxiv.org/abs/2505.22954.

[28] Albert Gu, Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023; v2 2024. https://arxiv.org/abs/2312.00752.

[29] Patrick Lewis 等. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020. https://arxiv.org/abs/2005.11401.

[30] Shen Nie 等. Large Language Diffusion Models. 2025. https://arxiv.org/abs/2502.09992.

[31] Jonas Geiping 等. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. 2025. https://arxiv.org/abs/2502.05171.

[32] Xiao-Wen Yang 等. Stabilizing Recurrent Dynamics for Test-Time Scalable Latent Reasoning in Looped Language Models. 2026-05-26; ICML 2026. https://arxiv.org/abs/2605.26733.

[33] NIST/SEMATECH. e-Handbook: 7.2.4.1 Confidence intervals. 访问日期 2026-10-11. https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm.

[34] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton. Layer Normalization. 2016. https://arxiv.org/abs/1607.06450.

[35] Chuan Guo, Geoff Pleiss, Yu Sun, Kilian Q. Weinberger. On Calibration of Modern Neural Networks. ICML 2017. https://proceedings.mlr.press/v70/guo17a.html.

[36] Jianlin Su 等. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021; v5 2023. https://arxiv.org/abs/2104.09864v5.

[37] Tom B. Brown 等. Language Models are Few-Shot Learners. 2020. https://arxiv.org/abs/2005.14165.

[38] Sang Michael Xie, Aditi Raghunathan, Percy Liang, Tengyu Ma. An Explanation of In-context Learning as Implicit Bayesian Inference. 2021; ICLR 2022. https://arxiv.org/abs/2111.02080.

[39] Mukund Sundararajan, Ankur Taly, Qiqi Yan. Axiomatic Attribution for Deep Networks. ICML 2017. https://arxiv.org/abs/1703.01365.

[40] Fred Zhang, Neel Nanda. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. v2 2024-01-17. https://arxiv.org/abs/2309.16042v2.

[41] Nelson Elhage 等. Toy Models of Superposition. 2022. https://arxiv.org/abs/2209.10652.

[42] Trenton Bricken 等. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. 2023-10-04. https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[43] Guilherme Penedo 等. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. v2 2024-10-31. https://arxiv.org/abs/2406.17557v2.

[44] DeepSeek-AI 等. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. Nature 645, 633–638, 2025-09-17. https://www.nature.com/articles/s41586-025-09422-z.

[45] Yang Yue 等. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?. v5 2025-11-24; NeurIPS 2025. https://arxiv.org/abs/2504.13837v5.

[46] OpenAI. Improving instruction hierarchy in frontier LLMs. 2026-03-10. https://openai.com/index/instruction-hierarchy-challenge/.

[47] Anthropic. Mitigating the risk of prompt injections in browser use. 2025-11-24. https://www.anthropic.com/research/prompt-injection-defenses.

[48] Xin Cheng 等. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. v2 2026-07-12. https://arxiv.org/abs/2601.07372v2.

[49] Dulhan Jayalath 等. Compute as Teacher: Turning Inference Compute Into Reference-Free Supervision. v3 2026-05-09. https://arxiv.org/abs/2509.14234v3.

[50] Thomas Hubert 等. Olympiad-level formal mathematical reasoning with reinforcement learning. Nature 651, 607–613; VOR 2026-03-13. https://www.nature.com/articles/s41586-025-09833-y.